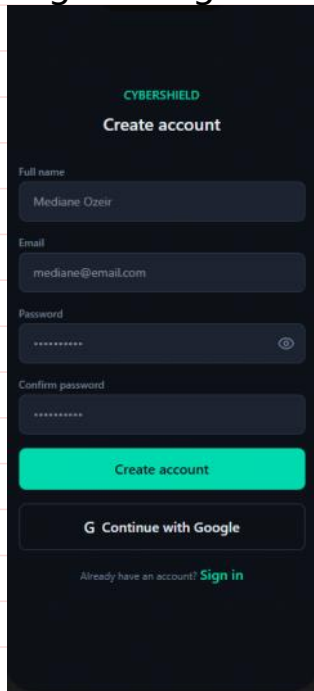


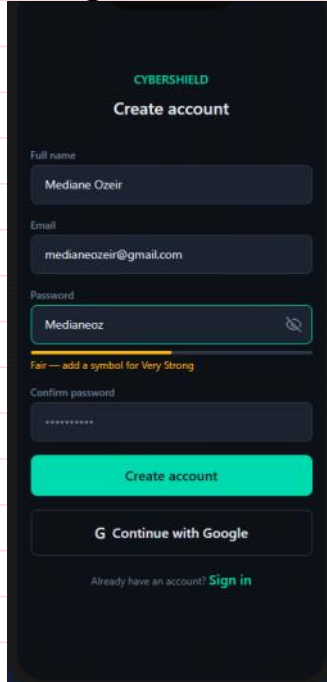
UI/UX Design

Register Page:



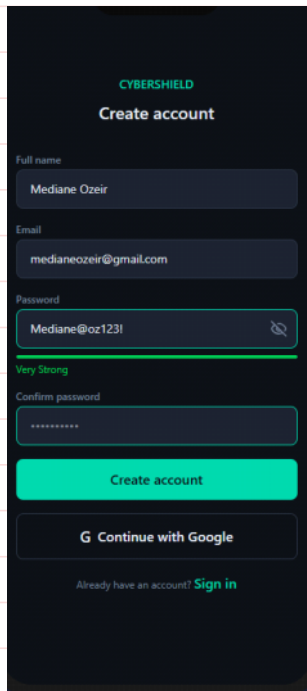
A mobile app registration screen with a dark background. At the top, the text 'CYBERSHIELD' is in green, and 'Create account' is in white. Below are four input fields: 'Full name' with 'Mediane Ozeir', 'Email' with 'mediane@email.com', 'Password' with masked characters and an eye icon, and 'Confirm password' with masked characters. A large green 'Create account' button is below the inputs. Underneath is a button with the Google logo and 'Continue with Google'. At the bottom, a link says 'Already have an account? Sign in'.

Filling info:



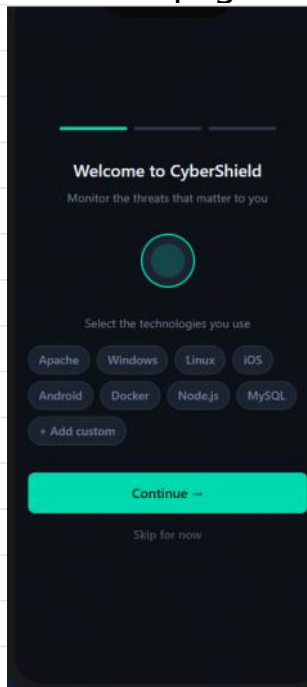
A mobile app registration screen, similar to the first one, but with additional feedback. The 'Full name' field contains 'Mediane Ozeir', the 'Email' field contains 'medianezeir@gmail.com', and the 'Password' field contains 'Medianeoz'. Below the password field, a green progress bar is partially filled, and the text 'Fair — add a symbol for Very Strong' is displayed in green. The 'Confirm password' field contains masked characters. The 'Create account' button is green, and the 'Continue with Google' button is below it. At the bottom, a link says 'Already have an account? Sign in'.

Password security check:



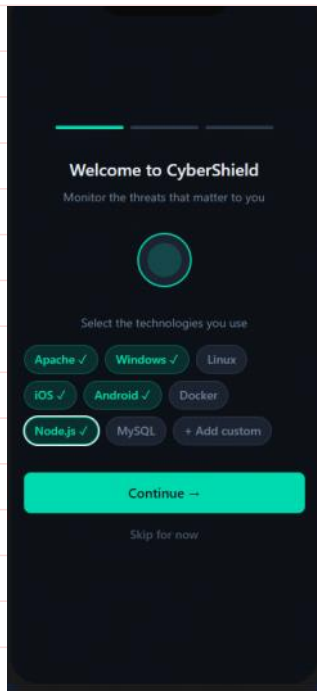
The image shows the 'Create account' screen of the CyberShield application. At the top, the 'CYBERSHIELD' logo is displayed in green, followed by the title 'Create account'. Below this, there are four input fields: 'Full name' (containing 'Mediane Ozeir'), 'Email' (containing 'medianeozeir@gmail.com'), 'Password' (containing 'Mediane@oz123!', with a strength indicator 'Very Strong' in green below it), and 'Confirm password' (containing '*****'). A green 'Create account' button is positioned below the confirm password field. Underneath the button is a 'Continue with Google' option, featuring the Google 'G' logo. At the bottom, there is a link that says 'Already have an account? Sign in'.

Welcome page:

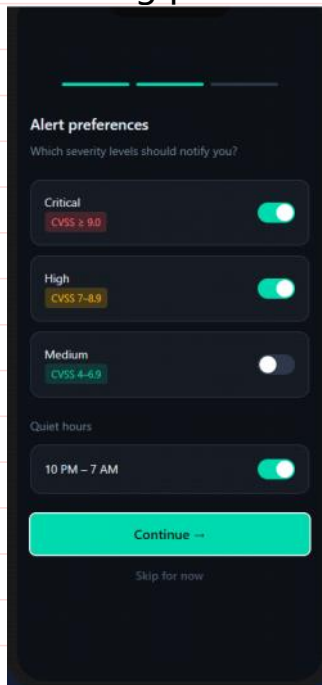


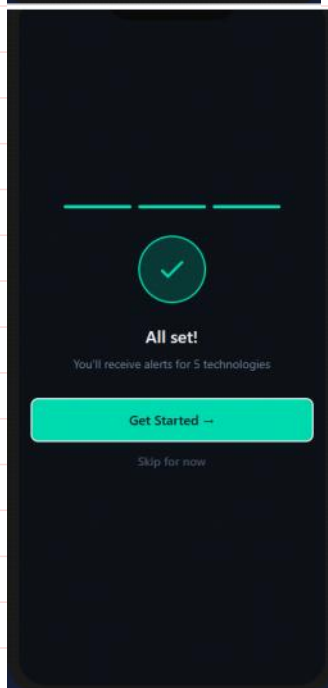
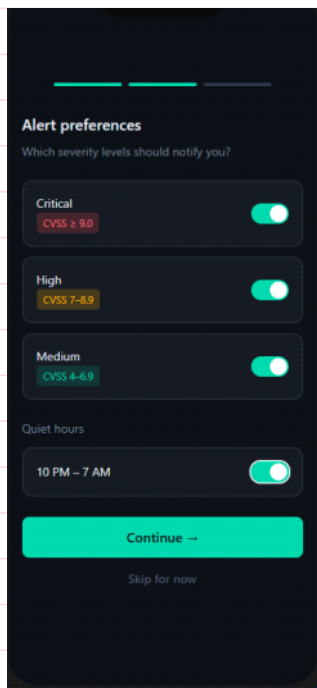
The image shows the 'Welcome to CyberShield' screen. At the top, there is a progress indicator with three segments, the first of which is green. Below this, the text 'Welcome to CyberShield' is displayed, followed by the tagline 'Monitor the threats that matter to you'. A large green circular icon with a white dot in the center is shown. Below the icon, the text 'Select the technologies you use' is displayed. There are eight buttons arranged in two rows: 'Apache', 'Windows', 'Linux', 'IOS' in the first row, and 'Android', 'Docker', 'Node.js', 'MySQL' in the second row. Below these buttons is an 'Add custom' button with a plus icon. A large green 'Continue' button with a right-pointing arrow is located at the bottom. Below the 'Continue' button is a link that says 'Skip for now'.

Choosing Technologies:

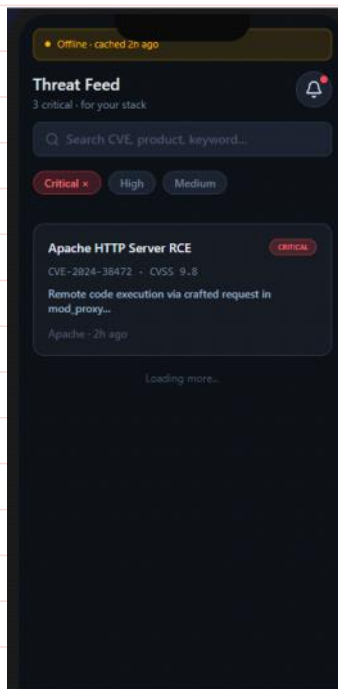


Choosing preferences (will be saved):





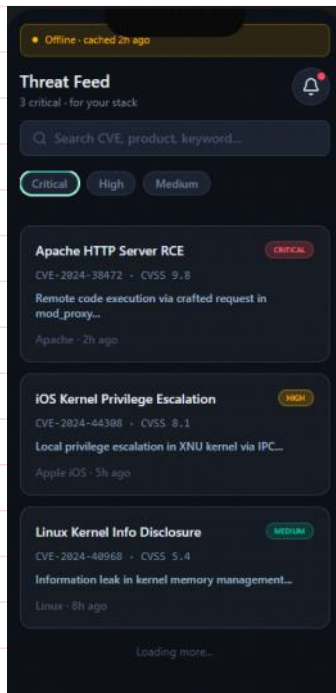
Feed Page:



Filtering by Severity:



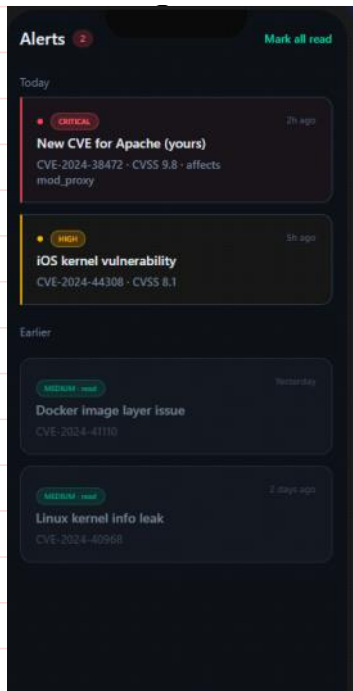
Of course, I will add an "All" option



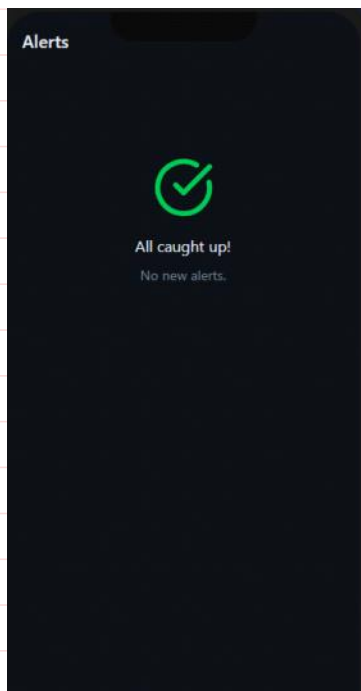
CVE card details:



Alerts Page:



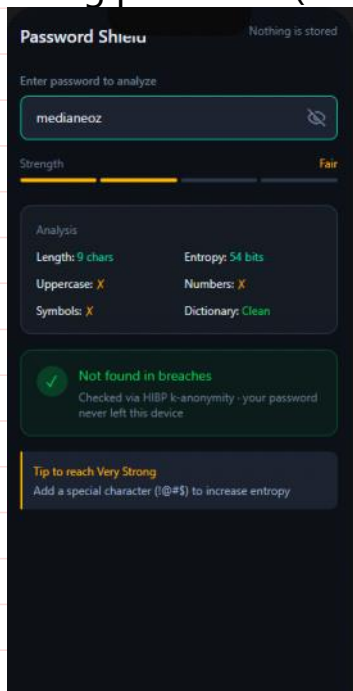
Read all alerts:



Password Shield Page:



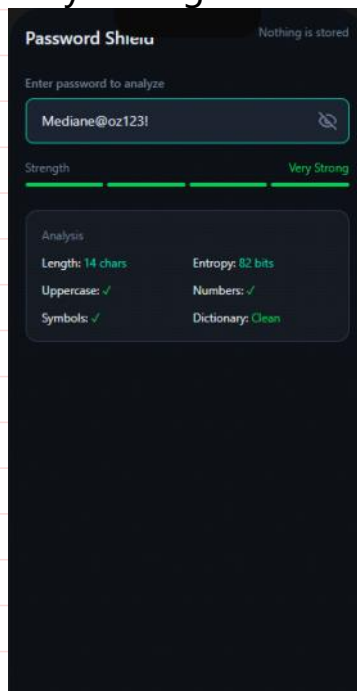
Filling password: (I will add a button for breaches checking, it will not be automatically)



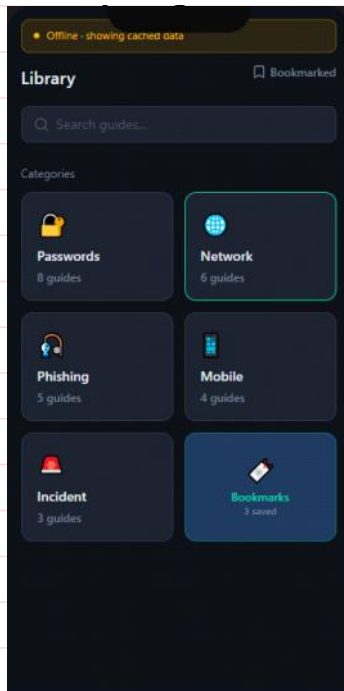
Strong Pass:



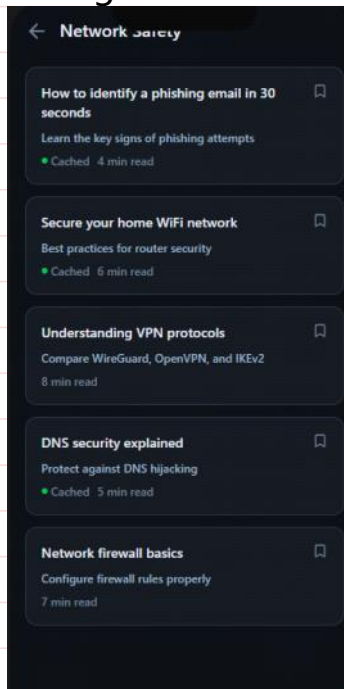
Very Strong Pass:



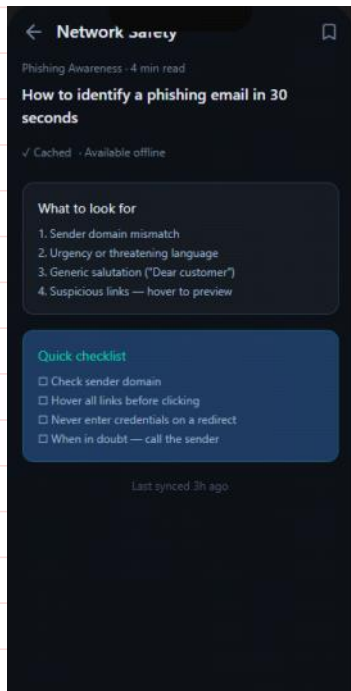
Library Page:



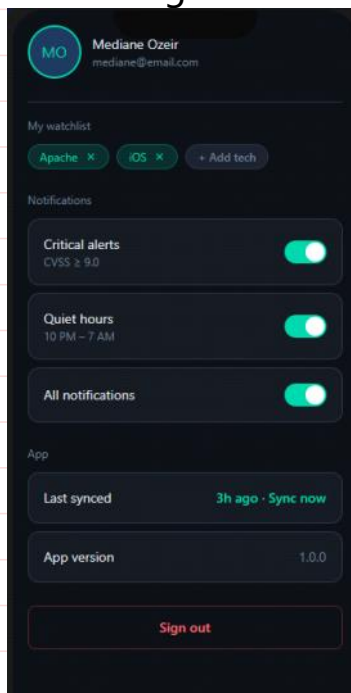
Listing Articles of a specific category:



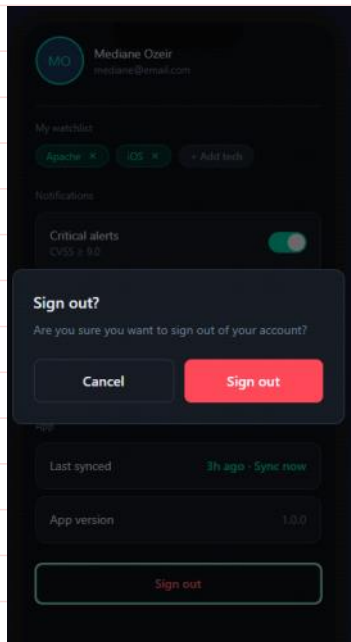
Article View:



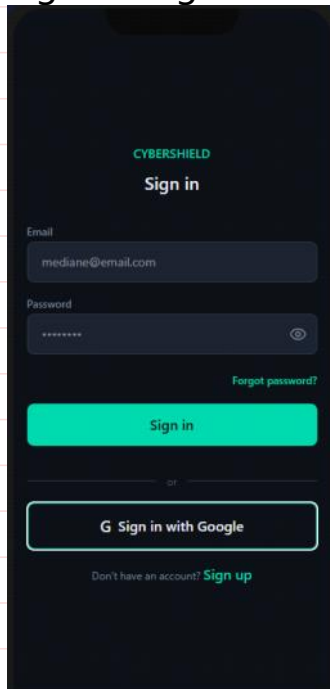
Profile Page:



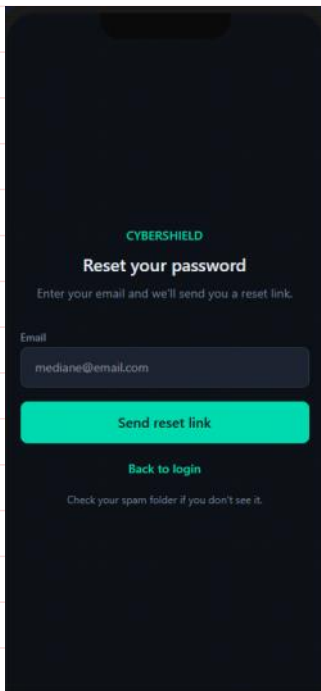
Sign out:



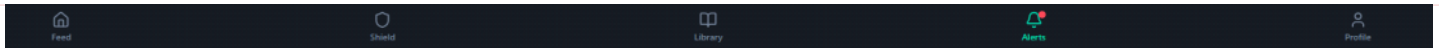
Sign-in Page:



Reset Password:



Footer:



Github Repository

Flutter project and Github repository creation:

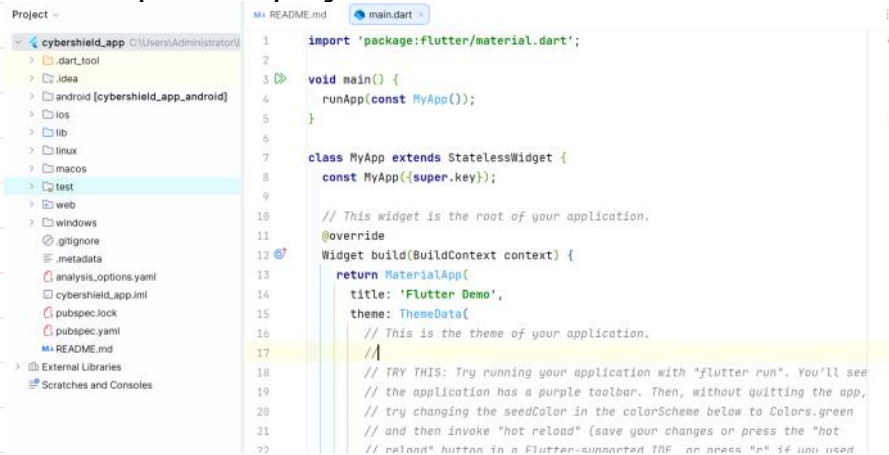
Creating the flutter project:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project
$ flutter create cybershield_app

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

Creating project cybershield_app...
Resolving dependencies in 'cybershield_app'...
Downloading packages...
```

Then I open the project in android studio:



Creating the MVC Folder Structure:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development
$ mkdir -p lib/app/models
$ mkdir -p lib/app/controllers
$ mkdir -p lib/app/views/auth
$ mkdir -p lib/app/views/home
$ mkdir -p lib/app/views/threat_feed
$ mkdir -p lib/app/views/password_checker
$ mkdir -p lib/app/views/knowledge_base
$ mkdir -p lib/app/views/notifications
$ mkdir -p lib/app/bindings
$ mkdir -p lib/app/routes
$ mkdir -p lib/app/services
$ mkdir -p lib/app/utils
```

I will create the basic models, views and controllers files. (I will add others if needed):

- Models:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development
$ touch lib/app/models/threat_advisory.dart
$ touch lib/app/models/user_profile.dart
$ touch lib/app/models/knowledge_article.dart
$ touch lib/app/models/password_check_result.dart
```

- Controllers:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship
$ touch lib/app/controllers/auth_controller.dart
touch lib/app/controllers/threat_feed_controller.dart
touch lib/app/controllers/password_controller.dart
touch lib/app/controllers/knowledge_controller.dart
touch lib/app/controllers/notification_controller.dart
```

- Bindings:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship
$ touch lib/app/bindings/home_binding.dart
touch lib/app/bindings/auth_binding.dart
touch lib/app/bindings/profile_binding.dart
```

- Routes:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship
$ touch lib/app/routes/app_pages.dart
touch lib/app/routes/app_routes.dart
```

- Services:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship
$ touch lib/app/services/api_service.dart
touch lib/app/services/auth_service.dart
touch lib/app/services/notification_service.dart
touch lib/app/services/storage_service.dart
```

- Utils:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship
$ touch lib/app/utils/constants.dart
touch lib/app/utils/validators.dart
touch lib/app/utils/password_analyzer.dart
```

- View Placeholder:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship
$ touch lib/app/views/home/home_view.dart
```

Now I will adjust some files.

lib/main:

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'app/routes/app_pages.dart';
import 'app/routes/app_routes.dart';

void main() {
  runApp(const CyberShieldApp());
}

class CyberShieldApp extends StatelessWidget {
  const CyberShieldApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'CyberShield',
      debugShowCheckedModeBanner: false,
      initialRoute: AppRoutes.INITIAL,
      getPages: AppPages.routes,
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
    );
  }
}
```

The main() function launches CyberShieldApp.

It's the entry point that loads the app and tells GetX which screens exist and which one to show first.

lib/app/routes/app_routes:

```
abstract class AppRoutes {  
  static const INITIAL = '/';  
  static const HOME = '/home';  
  static const LOGIN = '/login';  
}
```

The basic routes.

lib/app/routes/app_pages:

```
import 'package:get/get.dart';  
import '../views/home/home_view.dart';  
import '../bindings/home_binding.dart';  
import 'app_routes.dart';  
  
class AppPages {  
  static final routes = [  
    GetPage(  
      name: AppRoutes.INITIAL,  
      page: () => const HomeView(),  
      binding: HomeBinding(),  
    ), GetPage  
  ];  
}
```

It is a list of all screens with their route names, the widget to show, and optionally a Binding

lib/app/views/home/home_view:

```
import 'package:flutter/material.dart';  
  
class HomeView extends StatelessWidget {  
  const HomeView({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('CyberShield Home'),  
      ),  
      body: const Center(  
        child: Text('Welcome to CyberShield'),  
      ),  
    );  
  }  
}
```

only contains the design and logic for the home page.

pubspec.yaml:

```

name: cybershield_app
description: CyberShield Security Intelligence App

publish_to: 'none'

version: 1.0.0+1

environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter

  get: ^4.6.6
  dio: ^5.4.0
  hive: ^2.2.3
  hive_flutter: ^1.1.0
  firebase_core: ^3.6.0
  firebase_auth: ^5.3.0
  firebase_messaging: ^15.0.0
  flutter_secure_storage: ^9.0.0
  connectivity_plus: ^6.0.3
  google_sign_in: ^6.2.1

```

Adding all dependencies needed.

Running the flutter pub get, to download all dependencies.

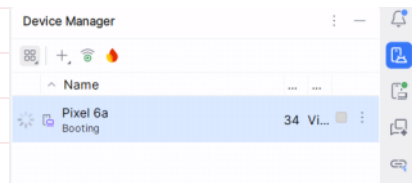
```

Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile
$ flutter pub get
Resolving dependencies...
Downloading packages...
+ _flutterfire_internals 1.3.59 (1.3.69 available)pubspec.yaml
+ args 2.7.0
+ code_assets 1.0.0
+ connectivity_plus 6.1.5 (7.1.1 available)dependencies (GetX, Dio, Firebase, e
+ connectivity_plus_platform_interface 2.1.0
+ crypto 3.0.7
+ dbus 0.7.12
+ dio 5.9.2
+ dio_web_adapter 2.1.2
+ ffi 2.2.0
+ file 7.0.1

```

Now I will run the flutter project.

But to be able to run the project, we need to run an emulator.



Now:

```

Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Proj
$ flutter run
Launching lib\main.dart on sdk gphone64 x86 64 in debug mode...
Running Gradle task 'assembleDebug'...

```

The project runs successfully:

Welcome to CyberShield

+
-
1/1
2/2

I will initialize the github repo:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/pro
$ git init
Initialized empty Git repository in C:/Users/Administrator/Desktop/Internship-2025/Internship-Projects/Mobile-Dev
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/pro
$ git add .
warning: in the working copy of 'metadata'. LF will be replaced by CRLF the next time Git touches it
```

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Project
$ git commit -m "Initial project setup with MVC structure"
[master (root-commit) 3130100] Initial project setup with MVC structure
155 files changed, 5477 insertions(+)
create mode 100644 .flutter-plugins-dependencies
create mode 100644 .gitignore
```

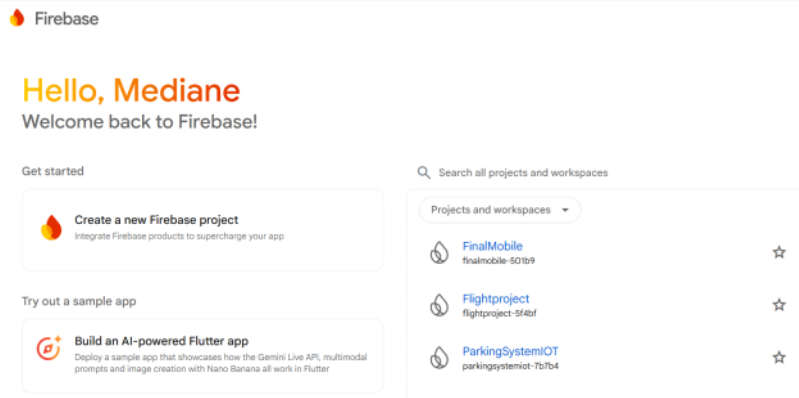
```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mob
$ git branch -M main
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mob
$ git remote add origin https://github.com/MedianeOz/mobileInternshipProj.git
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mob
$ git push -u origin main
Enumerating objects: 214, done.
Counting objects: 100% (214/214), done.
Delta compression using up to 4 threads
Compressing objects: 100% (166/166), done.
Writing objects: 100% (214/214), 273.59 KiB | 2.68 MiB/s, done.
Total 214 (delta 19), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (19/19), done.
To https://github.com/MedianeOz/mobileInternshipProj.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mob
$ git checkout -b develop
Switched to a new branch 'develop'
```

Authentication Flow

We'll proceed in this order:

- Firebase project setup
 - AuthService
 - AuthController + AuthBinding + routes updates
 - Login, Register, Forgot Password views
- Firebase setup connects the app to a real authentication backend (sign-up, sign-in, password reset).
 - AuthService wraps all Firebase calls in one clean, reusable class.
 - AuthController manages the UI state (loading, errors, current user) with GetX, so the views react automatically.
 - Routes & bindings wire the new screens into the app navigation and inject the controller where needed.
 - Login / Register / Forgot Password views turn the Figma designs into actual code, with proper form validation and error handling.

Firebase Setup:



I will create a new Firebase project:

× Create a project

Let's start with a name for
your project[?]

Project name

CyberShield-MobileInternship

 cybershield-mobileintern-78da8

Already have a Google Cloud project?
[Add Firebase to Google Cloud project](#)

Continue

× Create a project

Configure Google Analytics

Choose or create a Google Analytics account[?]

 Default Account for Firebase

Automatically create a new property in this account 

Upon project creation, a new Google Analytics property will be created in your chosen Google Analytics account and linked to your Firebase project. This link will enable data flow between the products. Data exported from your Google Analytics property into Firebase is subject to the Firebase Terms of Service, while Firebase data imported into Google Analytics is subject to the Google Analytics Terms of Service. [Learn more](#)

[Previous](#)

Create project



Preparing your project, please wait.
CyberShield-Mobileinternship

× Add Firebase to your Android app

1 Register app

Android package name[?]

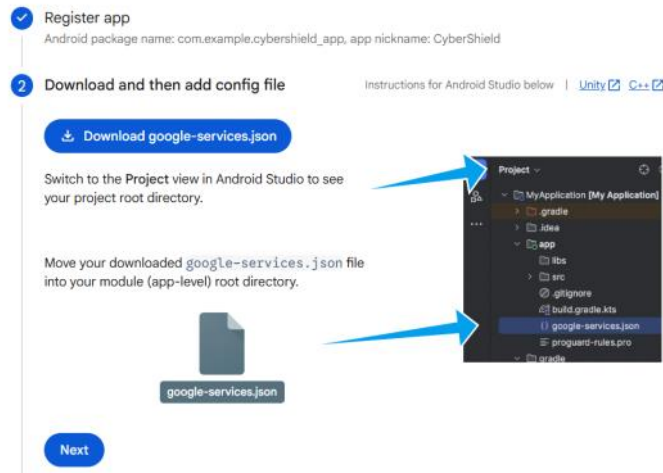
com.example.cybershield_app

App nickname(optional)[?]

CyberShield

Register app

I will download this google-services.json file



I will put in the android/app folder

- src
- build.gradle
- google-services

In android/build.gradle.kts

```
plugins {  
    id("com.android.application") version "8.1.0" apply false  
    id("org.jetbrains.kotlin.android") version "1.8.22" apply false  
    id("com.google.gms.google-services") version "4.4.4" apply false  
}
```

In android/app/build.gradle.kts

```
plugins {  
    id("com.android.application")  
    id("kotlin-android")  
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.  
    id("dev.flutter.flutter-gradle-plugin")  
    id("com.google.gms.google-services")  
}  
  
android {  
    ...  
}  
  
dependencies {  
    // Firebase BoM  
    implementation(platform("com.google.firebase:firebase-bom:34.12.0"))  
    // You do not need to add individual Firebase SDKs here—Flutter packages handle that.  
}
```

To sync the project with gradle files:

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project/cybershield_app (develop)
$ cd android/

Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project/cybershield_app/android (develop)
$ ./gradlew --refresh-dependencies

welcome to Gradle 8.14!

Here are the highlights of this release:
- Java 24 support
- GraalVM Native Image toolchain selection
- Enhancements to test reporting
- Build Authoring improvements

For more details see https://docs.gradle.org/8.14/release-notes.html

Starting a Gradle Daemon, 1 incompatible Daemon could not be reused, use --status for details
-----> 0% INITIALIZING [3s]
> Evaluating settings > compiling settings file 'settings.gradle.kts'
```

4 Next steps

You're ready!

Make sure that you take a look at the [documentation](#) to learn how to get started with each Firebase product that you want to use in your app.

You can also explore [sample Firebase apps](#).

Or, continue to the console to explore Firebase.

Previous

Continue to the console

The same for ios app:

1 Register app

Apple bundle ID ⓘ

com.example.cybershieldApp

App nickname(optional) ⓘ

CyberShield

App Store ID (optional) ⓘ

123456789

Register app

2 Download config file

Instructions for Xcode below | [Unity](#) | [C++](#)

Download GoogleService-Info.plist

Move the GoogleService-Info.plist file that you just downloaded into the root of your Xcode project and add it to all targets.



Next

Put it in ios/Runner/

- Assets.xcassets
- Base.lproj
- AppDelegate
- GeneratedPluginRegistrant
- GeneratedPluginRegistrant
- Info
- Runner-Bridging-Header
- SceneDelegate
- GoogleService-Info

```
Waiting for authentication...
+ Success! Logged in as medianeozeir@gmail.com
C:\Users\Administrator>
```

Woohoo!

Firebase CLI Login Successful

You are logged in to the Firebase Command-Line interface. You can immediately close this window and continue using the CLI.

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/pro
$ flutterfire configure --project=cybershield-mobileintern-78da8 --platforms=android,ios
i Found 4 Firebase projects. Selecting project cybershield-mobileintern-78da8.
i Selected platforms: android,ios
i Firebase android app com.example.cybershield_app registered.
i Firebase ios app com.example.cybershieldApp registered.

Firebase configuration file lib\firebase_options.dart generated successfully with the following Firebase apps:

Platform  Firebase App Id
android   1:730430895323:android:fafb7d4f0b1253961208fe
ios       1:730430895323:ios:1a4d7641e5232b931208fe

Learn more about using this file and next steps from the documentation:
> https://firebase.google.com/docs/flutter/setup
```

Created automatically:

- lib
 - app
 - firebase_options.dart
 - main.dart

In the main.dart

```
import 'app/routes/app_routes.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const CyberShieldApp());
}
```

Change the gradle from old version to new one:

☐ Download external annotations for dependencies

Gradle

Distribution: Wrapper

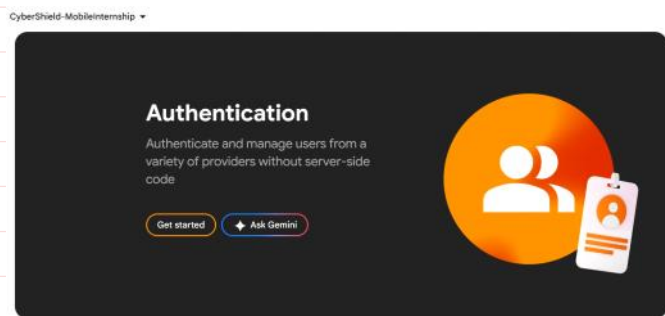
Gradle JDK: jbr-21 JetBrains Runtime 21.0.6 C:/Program Files/Android/

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project/cybershield_app (develop)
$ flutter clean
Deleting build... 5.2s
Deleting .dart_tool... 29ms
Deleting ephemeral... 1ms
Deleting Generated.xcconfig... 0ms
Deleting flutter_export_environment.sh... 2ms
Deleting ephemeral... 5ms
Deleting ephemeral... 1ms
Deleting ephemeral... 3ms
Deleting .flutter-plugins-dependencies... 0ms

Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project/cybershield_app (develop)
$ flutter pub get
Resolving dependencies...
Downloading packages...

Then run:
flutter run
```

Now Enable Firebase Authentication:



I will use the email and password as a provider:

Sign-in providers

Get started with Firebase Auth by adding your first sign-in method

Native providers	Additional providers	Custom providers
Email/Password	Google	OpenID Connect

Sign-in providers

Email/Password Enable

Allow users to sign up using their email address and password. Our SDKs also provide email address verification, password recovery and email address change primitives. [Learn more](#)

Email link (passwordless sign-in) Enable

Cancel Save

Also sign-in with google:

Google Enable

Important: To enable Google Sign-In for your Android apps, you must provide the [SHA-1 release fingerprint](#) for each app (go to [Project settings](#) > [Your apps](#) section).

Update the [project-level setting](#) below to continue

Public-facing name for project ⓘ

project-730430895323

Support email for project ⓘ

medianeozair@gmail.com

I'll take the SHA-1 key (required in the previous picture)

```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project/cybershield_app/android (develop)
$ ./gradlew signingReport

> Task :app:signingReport
Variant: debug
Config: debug
Store: C:\Users\Administrator\.android\debug.keystore
Alias: AndroidDebugKey
MD5: E7:A9:31:5B:31:82:E3:8E:3D:14:0E:65:3F:14:15:7A
SHA1: EC:8B:4A:CA:37:8A:A2:84:88:61:92:0D:A3:54:3D:AD:FE:82:02:01
SHA-256: 3B:4C:77:CF:A3:43:55:06:8F:A1:D2:8F:29:0F:9B:28:97:24:F0:12:65:D0:9A:5B:35:89:88:D7:0A:42:E7:24
```

And insert it in the firebase console project

Certificate fingerprint

EC:8B:4A:CA:37:8A:A2:84:88:61:92:0D:A3:54:3D:AD:FE:82:02:01 ✓ SHA1 SHA256

Cancel Save

Download the latest configuration file

Enabling Google sign-in for the first time creates new OAuth clients, which are automatically added to the config files for your apps.

Download and replace the configuration files in your apps so that you can use Google sign-in for your apps.

After replacing your config files, finish setting up Google sign-in for [Android](#) and [Apple](#) apps.

Search

Your apps	Configuration file
CyberShield	google-services.json
CyberShield	GoogleService-Info.plist

Done

I will replace the new google services (containing oauth_client) with the old ones.

Sign-in providers Add new provider

Provider	Status
Email/Password	✓ Enabled
Google	✓ Enabled

AuthService:

Generates the backend connection layer. This class wraps all Firebase calls (sign up, sign in, Google login, password reset, sign out) and manages the authentication state stream.

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:google_sign_in/google_sign_in.dart';

class AuthService {
  final FirebaseAuth _firebaseAuth;
  final GoogleSignIn _googleSignIn;
  final FlutterSecureStorage _storage;

  AuthService({
    FirebaseAuth? firebaseAuth,
    GoogleSignIn? googleSignIn,
    FlutterSecureStorage? storage,
  }) : _firebaseAuth = firebaseAuth ?? FirebaseAuth.instance,
       _googleSignIn = googleSignIn ?? GoogleSignIn(),
       _storage = storage ?? const FlutterSecureStorage();

  // Get current user
  User? get currentUser => _firebaseAuth.currentUser;

  // Auth state changes stream
  Stream<User?> get authStateChanges => _firebaseAuth.authStateChanges();
}
```

Sign up with email:

```
// Sign up with email
Future<UserCredential> signUpWithEmail(String email, String password) async {
  try {
    final credential = await _firebaseAuth.createUserWithEmailAndPassword(
      email: email,
      password: password,
    );

    await _storage.write(key: 'is_logged_in', value: 'true');

    return credential;
  } on FirebaseAuthException catch (e) {
    throw FirebaseAuthException(
      code: e.code,
      message: e.message ?? 'Sign up failed',
    );
  } catch (e) {
    throw FirebaseAuthException(
      code: 'unknown',
      message: e.toString(),
    );
  }
}
```

Sign in with email

```
// Sign in with email
Future<UserCredential> signInWithEmail(String email, String password) async {
  try {
    final credential = await _firebaseAuth.signInWithEmailAndPassword(
      email: email,
      password: password,
    );

    await _storage.write(key: 'is_logged_in', value: 'true');

    return credential;
  } on FirebaseAuthException catch (e) {
    throw FirebaseAuthException(
      code: e.code,
      message: e.message ?? 'Sign in failed',
    );
  } catch (e) {
    throw FirebaseAuthException(
      code: 'unknown',
      message: e.toString(),
    );
  }
}
```

Sign in with Google

```
// Sign in with Google
Future<UserCredential> signInWithGoogle() async {
  try {
    final GoogleSignInAccount? googleUser =
      await _googleSignIn.signIn();

    if (googleUser == null) {
      throw FirebaseAuthException(
        code: 'google-sign-in-aborted',
        message: 'Google sign-in was cancelled',
      );
    }

    final GoogleSignInAuthentication googleAuth =
      await googleUser.authentication;

    final OAuthCredential credential = GoogleAuthProvider.credential(
      accessToken: googleAuth.accessToken,
      idToken: googleAuth.idToken,
    );

    final userCredential =
      await _firebaseAuth.signInWithCredential(credential);

    await _storage.write(key: 'is_logged_in', value: 'true');

    return userCredential;
  } on FirebaseAuthException catch (e) {
    throw FirebaseAuthException(
      code: e.code,
      message: e.message ?? 'Google sign-in failed',
    );
  } catch (e) {
    throw FirebaseAuthException(
      code: 'unknown',
      message: e.toString(),
    );
  }
}
}
```

Password Reset:

```
// Send password reset email
Future<void> sendPasswordResetEmail(String email) async {
  try {
    await _firebaseAuth.sendPasswordResetEmail(email: email);
  } on FirebaseAuthException catch (e) {
    throw FirebaseAuthException(
      code: e.code,
      message: e.message ?? 'Password reset failed',
    );
  } catch (e) {
    throw FirebaseAuthException(
      code: 'unknown',
      message: e.toString(),
    );
  }
}
}
```

Sign Out:

```
// Sign out
Future<void> signOut() async {
  try {
    await _googleSignIn.signOut();
    await _firebaseAuth.signOut();
    await _storage.delete(key: 'is_logged_in');
  } on FirebaseAuthException catch (e) {
    throw FirebaseAuthException(
      code: e.code,
      message: e.message ?? 'Sign out failed',
    );
  } catch (e) {
    throw FirebaseAuthException(
      code: 'unknown',
      message: e.toString(),
    );
  }
}
}
```

AuthController:

Generates the state manager (GetX controller). It holds all reactive variables (isLoading, errorMessage, currentUser) and methods that the UI calls. When a user taps "Sign in" , the controller calls the AuthService and updates the state, making the UI react automatically. It also handles error messages from Firebase and listens for auth changes (e.g., if the user logs out elsewhere).

```
import 'package:get/get.dart';
import 'package:firebase_auth/firebase_auth.dart';
import '../services/auth_service.dart';

class AuthController extends GetxController {
  final AuthService _authService = Get.find<AuthService>();

  RxBool isLoading = false.obs;
  RxString errorMessage = ''.obs;
  Rxn<User> currentUser = Rxn<User>();

  @override
  void onInit() {
    super.onInit();

    // Listen to auth state changes
    _authService.authStateChanges.listen((user) {
      currentUser.value = user;
    });

    // Check if already signed in
    currentUser.value = _authService.currentUser;
  }
}
```

Register

```
// — Register —
Future<void> register(String email, String password) async {
  errorMessage.value = '';
  isLoading.value = true;

  try {
    final credential =
      await _authService.signUpWithEmail(email, password);
    currentUser.value = credential.user;
  } on FirebaseAuthException catch (e) {
    errorMessage.value = _mapError(e.code);
  } catch (e) {
    errorMessage.value = 'An unexpected error occurred.';
  } finally {
    isLoading.value = false;
  }
}
```

Login and Google Login

```
// — Login —
Future<void> login(String email, String password) async {
  errorMessage.value = '';
  isLoading.value = true;

  try {
    final credential =
      await _authService.signInWithEmail(email, password);
    currentUser.value = credential.user;
  } on FirebaseAuthException catch (e) {
    errorMessage.value = _mapError(e.code);
  } catch (e) {
    errorMessage.value = 'An unexpected error occurred.';
  } finally {
    isLoading.value = false;
  }
}
```


Google Login

```
// — Google Login —  
// Note: Google sign-in can throw PlatformException (not FirebaseAuthException)  
// when the user cancels, so we catch both here.  
Future<void> loginWithGoogle() async {  
  errorMessage.value = '';  
  isLoading.value = true;  
  
  try {  
    final credential = await _authService.signInWithGoogle();  
    currentUser.value = credential.user;  
  } on FirebaseAuthException catch (e) {  
    if (e.code != 'google-sign-in-aborted') {  
      // Aborted = user cancelled, not an error to surface  
      errorMessage.value = _mapError(e.code);  
    }  
  } catch (e) {  
    errorMessage.value = 'Google sign-in failed. Please try again.';  
  } finally {  
    isLoading.value = false;  
  }  
}
```

Reset Password and Logout

```
// — Reset Password —  
Future<void> resetPassword(String email) async {  
  errorMessage.value = '';  
  isLoading.value = true;  
  
  try {  
    await _authService.sendPasswordResetEmail(email);  
  } on FirebaseAuthException catch (e) {  
    errorMessage.value = _mapError(e.code);  
  } catch (e) {  
    errorMessage.value = 'Failed to send reset email.';  
  } finally {  
    isLoading.value = false;  
  }  
}
```

Logout

```
// — Logout —  
Future<void> logout() async {  
  errorMessage.value = '';  
  isLoading.value = true;  
  
  try {  
    await _authService.signOut();  
    currentUser.value = null;  
  } on FirebaseAuthException catch (e) {  
    errorMessage.value = _mapError(e.code);  
  } catch (e) {  
    errorMessage.value = 'Sign out failed.';  
  } finally {  
    isLoading.value = false;  
  }  
}
```

Error Handling:

```
// — Error Mapping —
String _mapError(String code) {
  switch (code) {
    // — Email / password —
    case 'invalid-email':
      return 'Please enter a valid email address.';
    case 'user-not-found':
      return 'No account found with this email.';
    case 'wrong-password':
      return 'Incorrect password. Please try again.';
    case 'invalid-credential':
      // Firebase v10+ merges user-not-found + wrong-password into this
      return 'Invalid email or password.';
    case 'email-already-in-use':
      return 'An account with this email already exists.';
    case 'weak-password':
      return 'Password is too weak. Use at least 6 characters.';
    case 'operation-not-allowed':
      return 'This sign-in method is not enabled.';

    // — Account state —
    case 'user-disabled':
      return 'This account has been disabled.';
    case 'too-many-requests':
      return 'Too many attempts. Please wait a moment and try again.';

    // — Network —
    case 'network-request-failed':
      return 'No internet connection. Please check your network.';

    // — Google —
    case 'google-sign-in-aborted':
      return ''; // User cancelled — do not show an error

    // — Fallback —
    default:
      return 'Something went wrong ($code). Please try again.';
  }
}
```

AuthBinding + Route updates + placeholder views

- AuthBinding – injects AuthService and AuthController so they are available to the login/register pages automatically (GetX dependency injection).
- Placeholder views – creates empty login_view.dart, register_view.dart, forgot_password_view.dart files with a basic Scaffold, so the app compiles and I can start building the actual UI.

Bindings/AuthBinding

```
import 'package:get/get.dart';
import '../services/auth_service.dart';
import '../controllers/auth_controller.dart';

class AuthBinding extends Bindings {
  @override
  void dependencies() {
    Get.put(AuthService());
    Get.put(AuthController());
  }
}
```

routes/app_routes

```

abstract class AppRoutes {
  static const INITIAL = '/';
  static const HOME = '/home';
  static const LOGIN = '/login';
  static const REGISTER = '/register';
  static const FORGOT_PASSWORD = '/forgot-password';
  static const ONBOARDING = '/onboarding';
}

```

routes/app_pages

```

class AppPages {
  static final routes = [
    GetPage(
      name: AppRoutes.INITIAL,
      page: () => const HomeView(),
      binding: HomeBinding(),
    ),
    GetPage(
      name: AppRoutes.LOGIN,
      page: () => const LoginView(),
      binding: AuthBinding(),
    ),
    GetPage(
      name: AppRoutes.REGISTER,
      page: () => const RegisterView(),
      binding: AuthBinding(),
    ),
    GetPage(
      name: AppRoutes.FORGOT_PASSWORD,
      page: () => const ForgotPasswordView(),
      binding: AuthBinding(),
    ),
  ];
}

```

views/auth/login_view

```

import 'package:flutter/material.dart';

class LoginView extends StatelessWidget {
  const LoginView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: const Center(child: Text('Login Screen')),
    );
  }
}

```

views/auth/register_view

```

import 'package:flutter/material.dart';

class RegisterView extends StatelessWidget {
  const RegisterView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Register')),
      body: const Center(child: Text('Register Screen')),
    );
  }
}

```

views/auth/forgot_password_view

```
import 'package:flutter/material.dart';

class ForgotPasswordView extends StatelessWidget {
  const ForgotPasswordView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Forgot Password')),
      body: const Center(child: Text('Forgot Password Screen')),
    );
  }
}
```

Full Login & Register Views

Now I'll adjust the views based on the figma designed created.

Part of the views/auth/login_view.dart code

```
class LoginView extends StatelessWidget {
  const LoginView({super.key});

  @override
  Widget build(BuildContext context) {
    final AuthController controller = Get.find<AuthController>();
    final emailController = TextEditingController();
    final passwordController = TextEditingController();
    final RxBool obscurePassword = true.obs;

    return Scaffold(
      backgroundColor: const Color(0xFF000F14),
      body: SafeArea(
        child: Obx(() {
          if (controller.currentUser.value != null) {
            Future.microtask(() => Get.offAllNamed(AppRoutes.HOME));
          }

          return Padding(
            padding: const EdgeInsets.symmetric(horizontal: 24),
            child: ListView(
              children: [
                const SizedBox(height: 64),

                // — Brand —
                const Center(
                  child: Text(
                    'CYBERSHIELD',
                    style: TextStyle(
                      color: Color(0xFF00E5A0),
                      fontSize: 12,
                      fontWeight: FontWeight.w600,
                      letterSpacing: 2.5,
                    ),
                  ),
                ),

                const SizedBox(height: 12),

                const Center(
                  child: Text(
                    'Sign in',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 26,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ),

                const SizedBox(height: 40),
              ],
            ),
          );
        })
      ),
    );
  }
}
```

Part of the register_view.dart code (the password security check)

```
// --- Strength Model ---
class _StrengthResult {
  final List<Color> bars;
  final String label;
  final Color labelColor;

  const _StrengthResult({
    required this.bars,
    required this.label,
    required this.labelColor,
  });
}

// --- Strength Evaluator ---
_StrengthResult _evaluatePassword(String value) {
  if (value.isEmpty) {
    return _StrengthResult(
      bars: List.generate(4, (_) => const Color(0xFF2A2D3A)),
      label: '',
      labelColor: Colors.transparent,
    );
  }

  final bool hasLower = value.contains(RegExp(r'[a-z]'));
  final bool hasUpper = value.contains(RegExp(r'[A-Z]'));
  final bool hasMultipleDigits = value.contains(RegExp(r'\d{2,}')) ||
    (value.split('').where((c) => RegExp(r'\d').hasMatch(c)).length >= 2);
  final bool hasSymbol =
    value.contains(RegExp(r'[!@#$$%^&*(),.?":{}|<>]'));

  final int len = value.length;

  int score = 0;

  // Length scoring
  if (len >= 10) score++;

  // Complexity scoring
  if (hasLower && hasUpper) score++;
  if (hasMultipleDigits) score++;
  if (hasSymbol) score++;

  // Clamp
  // if (score > 4) score = 4;

  // Fair
  if (score == 2) {
    return _StrengthResult(
      bars: [
        const Color(0xFFE5A000),
        const Color(0xFFE5A000),
        const Color(0xFF2A2D3A),
        const Color(0xFF2A2D3A),
      ],
      label: 'Fair',
      labelColor: const Color(0xFFE5A000),
    );
  }

  // Strong
  if (score == 3) {
    return _StrengthResult(
      bars: [
        const Color(0xFF00E5A0),
        const Color(0xFF00E5A0),
        const Color(0xFF00E5A0),
        const Color(0xFF2A2D3A),
      ],
      label: 'Strong',
      labelColor: const Color(0xFF00E5A0),
    );
  }

  // Very Strong
  return _StrengthResult(
    bars: List.generate(4, (_) => const Color(0xFF00C853)),
    label: 'Very Strong',
    labelColor: const Color(0xFF00C853),
  );
}
```

Part of the forgot_password_view.dart code

```
class ForgotPasswordView extends StatelessWidget {  
  const ForgotPasswordView({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    final AuthController controller = Get.find<AuthController>();  
    final emailController = TextEditingController();  
  
    // Track whether the reset email was sent successfully  
    final RxBool emailSent = false.obs;  
  
    return Scaffold(  
      backgroundColor: const Color(0xFF0D0F14),  
      body: SafeArea(  
        child: Obx(() {  
          return Padding(  
            padding: const EdgeInsets.symmetric(horizontal: 24),  
            child: ListView(  
              children: [  
                const SizedBox(height: 80),
```

```
child: Column(  
  children: [  
    const Icon(Icons.mark_email_read_outlined,  
      color: Color(0xFF00E5A0), size: 36),  
    const SizedBox(height: 12),  
    const Text(  
      'Reset link sent!',  
      style: TextStyle(  
        color: Colors.white,  
        fontSize: 16,  
        fontWeight: FontWeight.w600,  
      ),  
    ),  
    const SizedBox(height: 6),  
    Text(  
      'Check your inbox at ${emailController.text.trim()}',  
      textAlign: TextAlign.center,  
      style: const TextStyle(  
        color: Color(0xFF8A8F9E),  
        fontSize: 13,  
      ),  
    ),  
  ],  
),
```

I've made a mistake. The `firebase_options.dart` and `google-services.json` are created automatically. But it contains the private key of both the firebase and google services. So when I've pushed the commits to the github repo, I've got a an alert email:

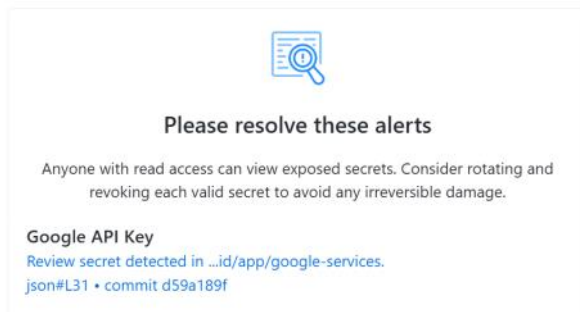
Action needed: Secrets detected in **MedianeOz/mobileInternshipProj**



Please resolve these alerts

Anyone with read access can view exposed secrets. Consider rotating and revoking each valid secret to avoid any irreversible damage.

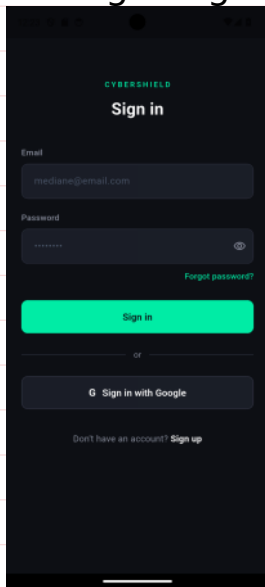
Google API Key
[Review secret detected in lib/firebase_options.dart#L61 • commit 447e7839](#)



I deleted these 2 files, and redo the setup (the previous steps) and before I push it to the github repo, I put it in the .gitignore file.
So now I have 2 new private keys that are only on my local device.

The login and Registration Page

The login Page:



Registration Page:

Weak Password

Fair (Upper and Lower Case + Symbols)

CYBERSHIELD

Create account

Full name

Mediane Ozeir

Email

mediane@gmail.com

Password

meDo@

1 bar

Confirm password

.....

Create account

Continue with Google

Already have an account? [Sign in](#)

Strong (Upper and Lower + Symbols + 2 numbers)

CYBERSHIELD

Create account

Full name

Mediane Ozeir

Email

mediane@gmail.com

Password

medO@12

3 bars

Strong

Confirm password

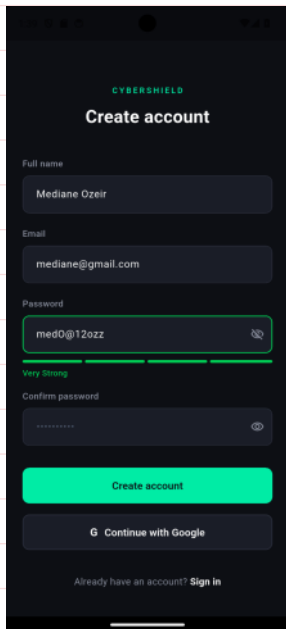
.....

Create account

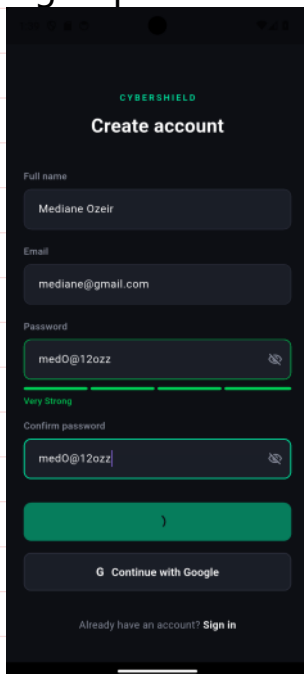
Continue with Google

Already have an account? [Sign in](#)

Very strong (Upper and Lower + Symbols + 2 numbers + length ≥ 10)



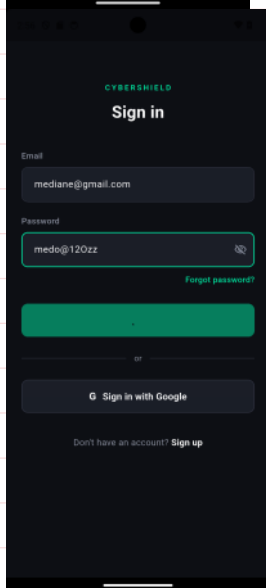
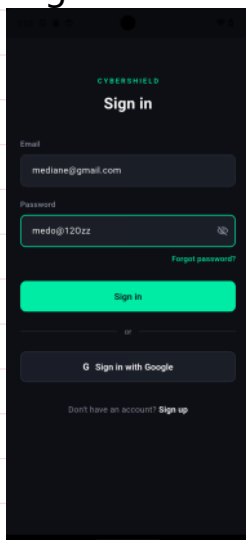
Sign Up:



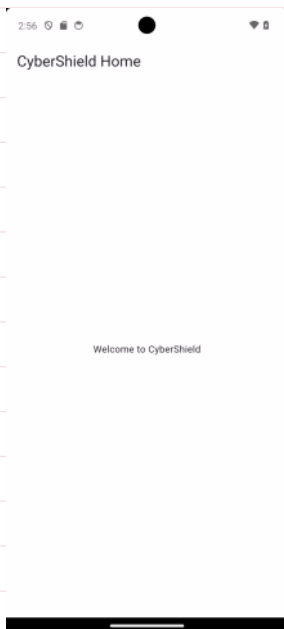
Account Created:



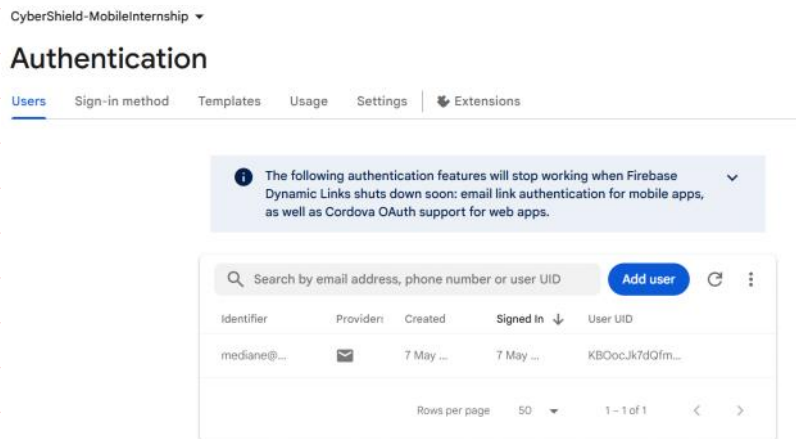
Login:



Logged-in Successfully:



In the Firebase Authentication:



I made some changes for the home_view page.

I added a logout button, that takes me to the login page

I added the email that the user logged-in with, a text and change the layout of the page.

The home_view.dart:

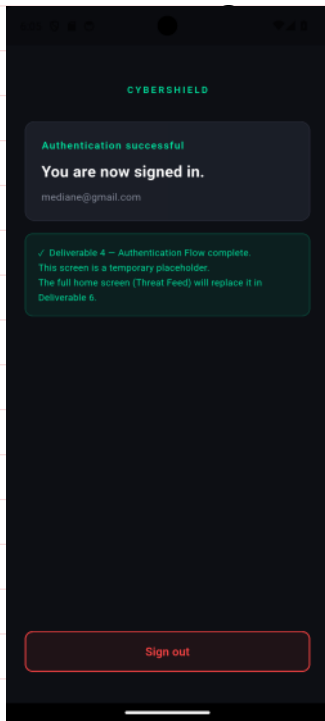
```

    'Authentication successful',
    style: TextStyle(
      color: Color(0xFF00E5A0),
      fontSize: 13,
      fontWeight: FontWeight.w600,
      letterSpacing: 1.2,
    ),
  ),
),
const SizedBox(height: 10),
const Text(
  'You are now signed in.',
  style: TextStyle(
    color: Colors.white,
    fontSize: 20,
    fontWeight: FontWeight.bold,
  ),
),
const SizedBox(height: 8),

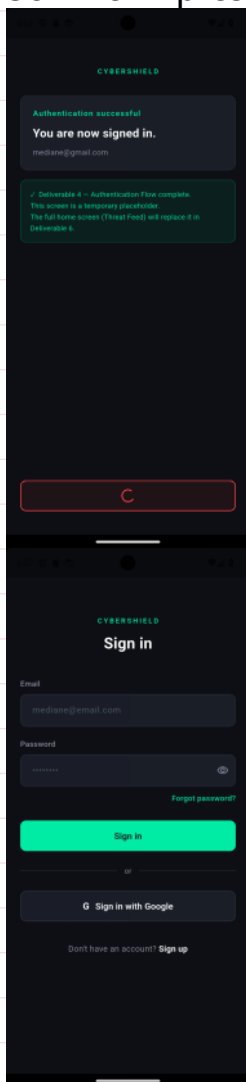
// — Show logged-in email —————
Obx(() {
  final email =
    auth.currentUser.value?.email ?? 'Unknown';
  return Text(
    email,
    style: const TextStyle(
      color: Color(0xFF8A8F9E),
      fontSize: 13,
    ),
  ),
),
Obx(() => SizedBox(
  width: double.infinity,
  height: 52,
  child: OutlinedButton(
    onPressed: auth.isLoading.value
      ? null
      : () async {
        await auth.logout();
        Get.offAllNamed(AppRoutes.LOGIN);
      },
    style: OutlinedButton.styleFrom(
      foregroundColor: const Color(0xFFFF4444),
      side: const BorderSide(
        color: Color(0xFFFF4444), width: 1.5),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(10),
      ),
      backgroundColor: const Color(0xFFFF4444)
        .withOpacity(0.07),
    ),
    child: auth.isLoading.value
      ? const SizedBox(
        width: 20,
        height: 20,
        child: CircularProgressIndicator(
          strokeWidth: 2,
          color: Color(0xFFFF4444),
        ),
      ),
  ),
),

```

The home Page:



So when I press the sign out button, it takes me to the sign-in page:



Enhancements:

1- AuthController:

- **authStateChanges** stream subscription leak:
onInit subscribes to **_authService.authStateChanges** but the **StreamSubscription** is never stored or cancelled. GetX will clean up the controller, but if the controller is put/found on multiple routes , the subscriptions can accumulate.

```
late StreamSubscription<User?> _authSub; // ← store the subscription

@override
void onInit() {
  super.onInit();

  // Check if already signed in
  currentUser.value = _authService.currentUser;

  // Listen auth state changes
  _authSub = _authService.authStateChanges.listen((user) {
    currentUser.value = user;
  });
}

@override
void onClose() {
  _authSub.cancel(); // ← cancel on controller disposal
  super.onClose();
}
```

- Add **clearError()** method:
When a user triggers an error on the login screen (e.g., "Invalid email or password"), then navigates to Register and back, **errorMessage** still holds the previous value and the error banner re-appears immediately.
"Single place to clear stale errors".

```
// — Clear error (call when navigating between auth screens) —
void clearError() => errorMessage.value = '';
```

- Add missing **_mapError** codes:
Defensive coverage for future features

```
case 'requires-recent-login':
  return 'For security, please sign out and sign in again before making this change.';
case 'account-exists-with-different-credential':
  return 'An account already exists with this email using a different sign-in method.';
case 'expired-action-code':
  return 'This reset link has expired. Please request a new one.';
case 'invalid-action-code':
  return 'This reset link is invalid or has already been used.';
```

2- login_view:

- Convert to **StatefulWidget** (prevents controller re-creation on rebuild)
- Add **_validateFields()** guard (catches empty fields before hitting Firebase)
- Call **clearError()** on navigation (Prevents stale error banner on return)

```
class LoginView extends StatefulWidget { // ← was StatelessWidget
  const LoginView({super.key});
```

```

@override
void initState() {
  super.initState();
  emailController = TextEditingController();
  passwordController = TextEditingController();
  obscurePassword = true.obs;

  // — Clear any stale error from a previous auth screen —
  WidgetsBinding.instance.addPostFrameCallback((_) {
    Get.find<AuthController>().clearError();
  });
}

```

```

@override
void dispose() {
  emailController.dispose();
  passwordController.dispose();
  super.dispose();
}

```

```

// — Empty-field guard —
bool _validateFields() {
  if (emailController.text.trim().isEmpty ||
      passwordController.text.trim().isEmpty) {
    Get.snackbar(
      'Missing fields',
      'Please enter your email and password.',
      backgroundColor: const Color(0xFFFA1D26),
      colorText: Colors.white,
    );
    return false;
  }
  return true;
}

```

```

// — Forgot Password —
Align(
  alignment: Alignment.centerRight,
  child: GestureDetector(
    onTap: () {
      controller.clearError(); // + clear before navigating
      Get.toNamed(AppRoutes.FORGOT_PASSWORD);
    },
    child: const Text(
      'Forgot password?',
      style: TextStyle(
        color: Color(0xFF00E5A0),
        fontSize: 13,
        fontWeight: FontWeight.w500,
      ),
    ),
  ),
),
),

```

```

// — Sign In Button —
Obx(() => _CyberPrimaryButton(
  label: 'Sign in',
  isLoading: controller.isLoading.value,
  onPressed: () {
    if (!_validateFields()) return; // + empty-field guard
    controller.login(
      emailController.text.trim(),
      passwordController.text.trim(),
    );
  },
)),
),

```



```
// — Footer —————
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    const Text("Don't have an account? ",
      style: TextStyle(
        color: Color(0xFF8ABF9E), fontSize: 14)),
    GestureDetector(
      onTap: () {
        controller.clearError(); // ← clear before navigating
        Get.toNamed(AppRoutes.REGISTER);
      },
      child: const Text('Sign up',
        style: TextStyle(
          color: Colors.white,
          fontSize: 14,
          fontWeight: FontWeight.bold,
        )),
    ),
  ],
),
),
```

3- forgot_password_view:

- Convert to StatefulWidget
- Move emailSent + emailController to initState (prevents state reset on rebuild)
- Call clearError() on back-to-login tap

```
class ForgotPasswordView extends StatefulWidget { // ← was StatelessWidget
  const ForgotPasswordView({super.key});

  @override
  State<ForgotPasswordView> createState() => _ForgotPasswordViewState();
}

class _ForgotPasswordViewState extends State<ForgotPasswordView> {
  late TextEditingController emailController;
  late RxBool emailSent; // ← moved out of build()

  @override
  void initState() {
    super.initState();
    emailController = TextEditingController();
    emailSent = false.obs;

    // — Clear any stale error from a previous auth screen ———
    WidgetsBinding.instance.addPostFrameCallback((_) {
      Get.find<AuthController>().clearError();
    });
  }
}
```

```
// — Back to Login —————
Center(
  child: GestureDetector(
    onTap: () {
      controller.clearError(); // ← clear before navigating
      Get.toNamed(AppRoutes.LOGIN);
    },
    child: const Text(
      'Back to login',
      style: TextStyle(
        color: Color(0xFF00E5A0),
        fontSize: 14,
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
),
),
```

Last Step:

- 1- Adding the check of the email format, password, full name in the utils/validators.
- 2- Adding the password security analyze in the utils/password_analyzer.

3- Adding hardcoded constants in utils/constants.

1-

```
class Validators {
  Validators._(); // static-only class

  static final RegExp _emailRegex =
    RegExp(r'^[\\w.+-]+@[\\w-]+\\.([a-z]){2,}$', caseSensitive: false);

  static String? email(String value) {
    final t = value.trim();
    if (t.isEmpty) return 'Email is required.';
    if (!_emailRegex.hasMatch(t)) return 'Please enter a valid email address.';
    return null;
  }

  static String? password(String value) {
    if (value.isEmpty) return 'Password is required.';
    if (value.length < 6) return 'Password must be at least 6 characters.';
    return null;
  }

  static String? fullName(String value) {
    final t = value.trim();
    if (t.isEmpty) return 'Full name is required.';
    if (t.length < 2) return 'Please enter your full name.';
    return null;
  }
}
```

2-

```
enum PasswordStrength { empty, weak, fair, strong, veryStrong }

class PasswordStrengthResult {
  final PasswordStrength level;
  final List<Color> bars;
  final String label;
  final Color labelColor;
  final Color borderColor;

  const PasswordStrengthResult({
    required this.level,
    required this.bars,
    required this.label,
    required this.labelColor,
    required this.borderColor,
  });
}

class PasswordAnalyzer {
  PasswordAnalyzer._();

  static const Color _empty = Color(0xFF2A2D3A);
  static const Color _red = Color(0xFFFF4444);
  static const Color _amber = Color(0xFFE8A000);
  static const Color _teal = Color(0xFF00E8A0);
  static const Color _green = Color(0xFF00C853);

  static PasswordStrengthResult evaluate(String value) {
    if (value.isEmpty) {
      return PasswordStrengthResult(
        level: PasswordStrength.empty,
        bars: List.generate(4, (_) => _empty),
        label: '',
        labelColor: Colors.transparent,
        borderColor: _empty,
      );
    }

    int score = 0;
    if (value.length >= 10) score++;
    if (value.contains(RegExp(r'[a-z]')) &&
      value.contains(RegExp(r'[A-Z]'))) score++;
    if (value.split('').where((c) => RegExp(r'\d').hasMatch(c)).length >= 2) score++;
    if (value.contains(RegExp(r'[!@#$%^&*().,?":{}|<>]'))) score++;

    if (score <= 1) {
      return PasswordStrengthResult(
        level: PasswordStrength.weak,
        bars: [_red, _empty, _empty, _empty],
        label: 'Weak',
        labelColor: _red,
        borderColor: _red,
      );
    }
  }
}
```

3-

```
// lib/app/utils/constants.dart

import 'package:flutter/material.dart';

class AppColors {
  AppColors._();

  static const background = Color(0xFF000F14);
  static const surface = Color(0xFF1A1D26);
  static const border = Color(0xFF2A2D3A);
  static const textMuted = Color(0xFF8A8F9E);
  static const textHint = Color(0xFF4A5568);
  static const primary = Color(0xFF00E5A0);
  static const primaryDark = Color(0xFF00C853);
  static const danger = Color(0xFFFF4444);
  static const warning = Color(0xFFE5A000);
}
```

Now I'll update the login and register view:
Register view:

```
late Rx<PasswordStrengthResult> strength; // ← now uses PasswordAnalyzer type
late RxBool confirmMatches;

@override
void initState() {
  super.initState();
  nameController = TextEditingController();
  emailController = TextEditingController();
  passwordController = TextEditingController();
  confirmController = TextEditingController();
  obscurePassword = true.obs;
  obscureConfirm = true.obs;
  strength = PasswordAnalyzer.evaluate('').obs; // ← PasswordAnalyzer
  confirmMatches = false.obs;
}
```

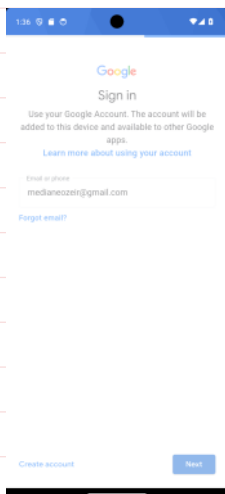
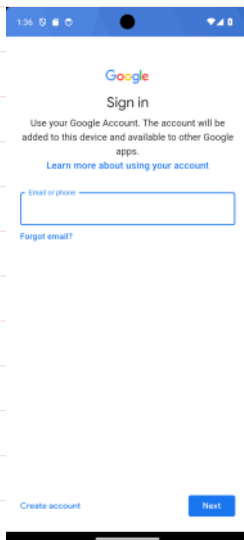
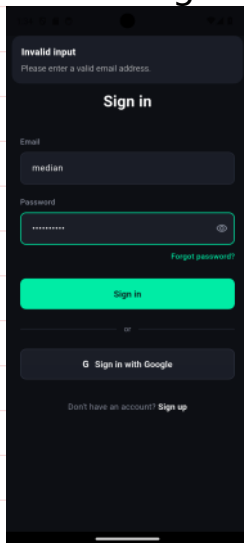
```
// — Password —
_AuthLabel(label: 'Password'),
const SizedBox(height: 8),
Obx(() => _AuthTextField(
  controller: passwordController,
  hintText: '.....',
  obscureText: obscurePassword.value,
  onChanged: (val) {
    strength.value = PasswordAnalyzer.evaluate(val); // ← PasswordAnalyzer
    confirmMatches.value =
      val == confirmController.text && val.isNotEmpty;
  },
),
```

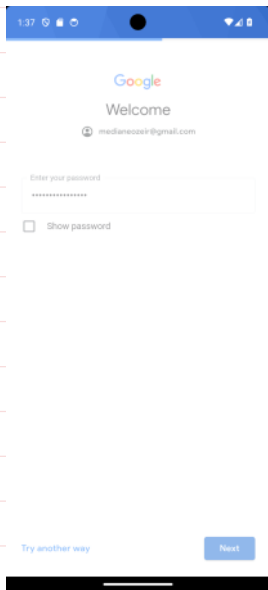
login_view:

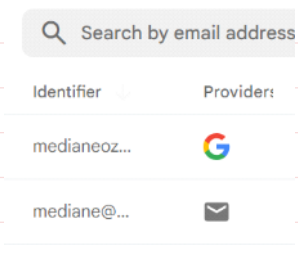
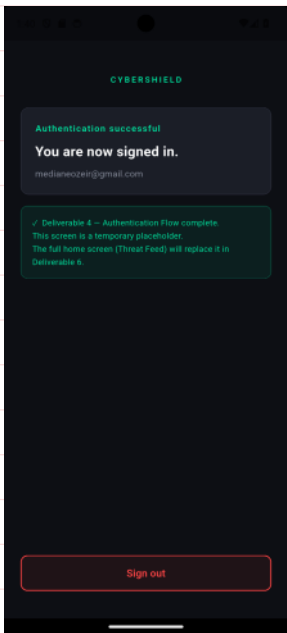
```
bool _validateFields() {
  final emailError = Validators.email(emailController.text);
  final passwordError = Validators.password(passwordController.text);
  final firstError = emailError ?? passwordError;

  if (firstError != null) {
    Get.snackbar(
      'Invalid input',
      firstError,
      backgroundColor: const Color(0xFF1A1D26),
      colorText: Colors.white,
    );
    return false;
  }
  return true;
}
```

It's working successfully:







API Integration

In this step I implemented Deliverable 5: API Integration for the CyberShield Flutter mobile app

The goal of this step is to connect the app to real external APIs and display live cybersecurity data inside the app.

The two APIs used are:

- **NVD API**: used to fetch real CVE vulnerability advisories.
- **Have I Been Pwned Passwords API**: used to check whether a password appeared in known data breaches.

The user is already authenticated before reaching these functionalities, so this step focuses on the authenticated part of the app.

I will proceed in this order:

- Complete constants and design tokens
- Create the data models
- Build the API service using Dio
- CreateGetX controllers
- Create the threat feed screen
- Create the password checker screen
- Add the bottom navigation shell
- Update routes and bindings
- Test and analyze the project

~API Intregation Overview

The app communicates with two external services:

1-NVD:

- Fetches CVE advisories
- Supports pagination
- Supports search by keyword
- Supports severity filtering

- Returns vulnerability data like CVE Id, description, severity, CVSS score, published date, and references.

2-HIBP:

- Checks if a password was exposed in a breach
- Uses the k-anonymity model
- The full password is never sent to the server
- Only the first 5 characters of the SHA-1 hash are sent

#Pubspec.yaml

I added the crypto package as a direct dependency

This package is used to generate the SHA-1 hash required by the Have I Been Pwned password API

I also added flutter_lints because already referenced it

```
crypto: ^3.0.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

Run flutter pub get successfully

```
Administrator@Medizine-M2NOW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project/cybershield_app (develop)
$ flutter pub get

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

Resolving dependencies...
Downloading packages...
  flutterfire_internals 1.3.59 (1.3.71 available) required by flutterfire_core
  cloud_firestore 5.6.12 (6.4.1 available)
  cloud_firestore_platform_interface 6.6.12 (8.0.1 available)
  cloud_firestore_web 4.4.12 (5.4.1 available)
  connectivity_plus 6.1.5 (7.1.1 available)
  firebase_auth 5.7.0 (6.5.1 available)
```

#utils/constants.dart

I completed the constants file

This file now contains the CyberShield color and API constants used across the app.

AppColors:


```

class AppColors {
    AppColors._();

    static const background = Color(0xFF000F14);
    static const surface = Color(0xFF1A1D26);
    static const border = Color(0xFF2A2D3A);
    static const primary = Color(0xFF00E5A0);
    static const primaryDark = Color(0xFF00C853);
    static const danger = Color(0xFFFF4444);
    static const warning = Color(0xFFE5A000);
    static const textMuted = Color(0xFF8A8F9E);
    static const textHint = Color(0xFF4A5568);
    static const white = Color(0xFFFFFFFF);
}

```

AppStrings:

```

class AppStrings {
    AppStrings._();

    static const appName = 'CYBERSHIELD';
    static const nvdBaseUrl = 'https://services.nvd.nist.gov/rest/json/cves/2.0';
    static const hibpBaseUrl = 'https://api.pwnedpasswords.com/range';
    static const nvdApiKey = 'YOUR_KEY_HERE';
    static const nvdResultsPerPage = 20;
}

```

I need to get the API key

Now I will set up NVD API Key:

<https://nvd.nist.gov/developers/request-an-api-key>

Request an API Key

To request an NVD API Key, please provide your organization name and a valid email address, and indicate your organization type. You must scroll to end of the Terms of Use Agreement and check "I agree to the Terms of Use" to obtain an API Key. Upon submitting the request, you will receive an email containing a single-use hyperlink that is used to activate and view your API Key. If your key is not activated within seven days, a new request for an API Key must be submitted.

Organization Name:	Sekureup
Email Address:	medianeozeir@gmail.com
Organization Type:	Academia

I received the email:

Thank you for your request for an NVD API Key. Please navigate to the following page to activate you

<https://nvd.nist.gov/developers/confirm-api-key>

On that page, please verify your email address and use the following UUID: **B9EC1FEE-A052-F111-8**

For additional information on both the CVE and CPE API, please visit <https://nvd.nist.gov/developers>.

If you did not request an API Key from the NVD, please contact NVD Support at nvd@nist.gov.

V/r
National Vulnerability Database Team

Generating Your API Key

Do not leave this page until API key generation and validation are completed.

5%

Validating your API key... (0s elapsed)

```
class AppStrings {
  AppStrings._();

  static const appName = 'CYBERSHIELD';
  static const nvdBaseUrl = 'https://services.nvd.nist.gov/rest/json/cves/2.0';
  static const h1bpBaseUrl = 'https://api.pwnedpasswords.com/range';
  static const nvdApiKey = '22c34afc-c129-4fa6-896f-a4676276fdb6';
  static const nvdResultsPerPage = 20;
}
```

Models

#models/threat_advisory.dart

I adjusted the ThreatAdvisory model

This model represents one CVE advisory from the NVD API

The fields:

```
class ThreatAdvisory {
  final String id;
  final String description;
  final double baseScore;
  final String severity;
  final DateTime publishedDate;
  final DateTime lastModifiedDate;
  final List<String> referenceUrls;

  const ThreatAdvisory({
    required this.id,
    required this.description,
    required this.baseScore,
    required this.severity,
    required this.publishedDate,
    required this.lastModifiedDate,
    required this.referenceUrls,
  });
}
```

The model includes a fromJson factory

The NVD response is nested, so the model extracts:

- CVE ID from cve.id
- English description from descriptions
- CVSS score and severity from metrics
- Reference links from references
- Published and last modified dates

```

factory ThreatAdvisory.fromJson(Map<String, dynamic> json) {
  final descriptions = _asList(json['descriptions']);
  final englishDescription = descriptions.cast<Map?>().firstWhere(
    (item) => item?['lang'] == 'en',
    orElse: () => descriptions.isNotEmpty ? descriptions.first : null,
  );

  final metric = _extractMetric(json['metrics']);
  final references = _asList(json['references'])
    .map((item) => item is Map ? item['url']?.toString() : null)
    .whereType<String>()
    .where((url) => url.isNotEmpty)
    .toList();

  return ThreatAdvisory(
    id: json['id']?.toString() ?? 'Unknown CVE',
    description: englishDescription?['value']?.toString() ??
      'No description is available for this advisory.',
    baseScore: metric.score,
    severity: metric.severity,
    publishedDate: _parseDate(json['published']),
    lastModifiedDate: _parseDate(json['lastModified']),
    referenceUrls: references,
  );
}

```

```

factory ThreatAdvisory.fromCache(Map<dynamic, dynamic> json) {
  return ThreatAdvisory(
    id: json['id']?.toString() ?? 'Unknown CVE',
    description: json['description']?.toString() ??
      'No description is available for this advisory.',
    baseScore: (json['baseScore'] as num?)?.toDouble() ?? 0,
    severity: _normalizeSeverity(json['severity']?.toString()),
    publishedDate: _parseDate(json['publishedDate']),
    lastModifiedDate: _parseDate(json['lastModifiedDate']),
    referenceUrls: _asList(json['referenceUrls'])
      .map((item) => item.toString())
      .where((url) => url.isNotEmpty)
      .toList(),
  );
}

```

I also added graceful null handling.

This is important because some NVD vulnerabilities may not include metrics, descriptions, or references.

If a field is missing, the app will not crash

```

Color get severityColor {
  switch (severity.toUpperCase()) {
    case 'CRITICAL':
      return AppColors.danger;
    case 'HIGH':
      return AppColors.warning;
    case 'MEDIUM':
      return AppColors.primary;
    case 'LOW':
      return AppColors.textMuted;
    default:
      return AppColors.textHint;
  }
}

static List<dynamic> _asList(dynamic value) {
  return value is List ? value : <dynamic>[];
}

static DateTime _parseDate(dynamic value) {
  return DateTime.tryParse(value?.toString() ?? '') ??
    DateTime.fromMillisecondsSinceEpoch(0);
}

```

```

static _MetricSummary _extractMetric(dynamic metrics) {
  if (metrics is! Map) {
    return const _MetricSummary(score: 0, severity: 'UNKNOWN');
  }

  final metricGroups = [
    metrics['cvssMetricV31'],
    metrics['cvssMetricV30'],
    metrics['cvssMetricV2'],
  ];

  for (final group in metricGroups) {
    final entries = _asList(group);
    if (entries.isEmpty || entries.first is! Map) continue;

    final entry = entries.first as Map;
    final data = entry['cvssData'];
    if (data is! Map) continue;

    final score = (data['baseScore'] as num??).toDouble() ?? 0;
    final severity = data['baseSeverity']?.toString() ??
      entry['baseSeverity']?.toString() ??
      'UNKNOWN';

    return _MetricSummary(
      score: score,
      severity: _normalizeSeverity(severity),
    );
  }
}

```

```

static String _normalizeSeverity(String? value) {
  final severity = (value ?? 'UNKNOWN').toUpperCase();
  const supported = {'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'};
  return supported.contains(severity) ? severity : 'UNKNOWN';
}

class _MetricSummary {
  final double score;
  final String severity;

  const _MetricSummary({
    required this.score,
    required this.severity,
  });
}

```

#models/password_check_result.dart

This model stores the result of the HIBP password check

```

class PasswordCheckResult {
  final bool isPwned;
  final int breachCount;
  final String password;

  const PasswordCheckResult({
    required this.isPwned,
    required this.breachCount,
    required this.password,
  });
}

```

The password is kept in memory only.

It is never stored in Hive, Firebase, logs, or local files.

#models/user_profiles.dart

I completed the UserProfile model

This model stores user profile data and preferences.

```

class UserProfile {
  final String uid;
  final String email;
  final String displayName;
  final List<String> watchList;
  final bool criticalAlertsEnabled;
  final bool quietHoursEnabled;
  final bool allNotificationsEnabled;

  const UserProfile({
    required this.uid,
    required this.email,
    required this.displayName,
    required this.watchList,
    this.criticalAlertsEnabled = true,
    this.quietHoursEnabled = true,
    this.allNotificationsEnabled = true,
  });
}

```

I also added fromJson and toJson.

This will help later when saving profile preferences to Firebase or local storage

```

factory UserProfile.fromJson(Map<String, dynamic> json) {
  return UserProfile(
    uid: json['uid']?.toString() ?? '',
    email: json['email']?.toString() ?? '',
    displayName: json['displayName']?.toString() ?? '',
    watchList: (json['watchList'] as List?)
      ?.map((item) => item.toString())
      .where((item) => item.isNotEmpty)
      .toList() ??
      <String>[],
    criticalAlertsEnabled: json['criticalAlertsEnabled'] as bool? ?? true,
    quietHoursEnabled: json['quietHoursEnabled'] as bool? ?? true,
    allNotificationsEnabled: json['allNotificationsEnabled'] as bool? ?? true,
  );
}

Map<String, dynamic> toJson() {
  return {
    'uid': uid,
    'email': email,
    'displayName': displayName,
    'watchList': watchList,
    'criticalAlertsEnabled': criticalAlertsEnabled,
    'quietHoursEnabled': quietHoursEnabled,
    'allNotificationsEnabled': allNotificationsEnabled,
  };
}

```

Services

#services/api_services.dart

This is the main API integration file.

I added a complete Dio-based service in the empty ApiService class.

This service handles:

- NVD API calls
- HIBP API calls
- Dio configuration
- Request logging in debug mode
- Basic retry logic
- Error mapping
- Threat caching with Hive

1- Dio Confiugration

I configured Dio with 15 second connect timeout, 30 seconds receive timeout, debug request logging, one automatic retry for connection errors.

The debug request logging only runs in debug mode. This avoid exposing unnecessary network info in release builds.

```
class ApiService {
  late final Dio _dio;

  String errorMessage = '';

  ApiService() {
    _dio = Dio(
      BaseOptions(
        connectTimeout: const Duration(seconds: 15),
        receiveTimeout: const Duration(seconds: 30),
      ),
    );

    _dio.interceptors.add(
      InterceptorsWrapper(
        onRequest: (options, handler) {
          if (kDebugMode) {
            debugPrint('[API] ${options.method} ${options.url}');
          }
          handler.next(options);
        },
        onError: (error, handler) async {
          final shouldRetry = error.type == DioExceptionType.connectionError &&
            error.requestOptions.extra['retried'] != true;

```

```
          if (!shouldRetry) {
            handler.next(error);
            return;
          }

          try {
            final retryOptions = error.requestOptions;
            retryOptions.extra['retried'] = true;
            final response = await _dio.fetch<dynamic>(retryOptions);
            handler.resolve(response);
          } on DioException catch (retryError) {
            handler.next(retryError);
          } catch (_) {
            handler.next(error);
          }
        },
      ),
    );
  }
}
```

Sending the apiKey in the header

```
try {
  final response = await _dio.get<Map<String, dynamic>>({
    AppStrings.nvdBaseUrl,
    queryParameters: queryParameters,
    options: Options(
      headers: {
        'apiKey': AppStrings.nvdApiKey,
      },
    ),
  });
}
```

Adding internet access in the android manifest:

2- fetchThreats

I added the fetchThreats method. This method fetches CVE advisories from NVD

```
// -- NVD threat feed --
Future<List<ThreatAdvisory>> fetchThreats({
  int page = 0,
  String? keyword,
  String? severity,
}) async {
  errorMessage = '';

  final queryParameters = <String, dynamic>{
    'resultsPerPage': AppStrings.nvdResultsPerPage,
    'startIndex': page * AppStrings.nvdResultsPerPage,
    'apiKey': AppStrings.nvdApiKey,
  };

  final trimmedKeyword = keyword?.trim();
  if (trimmedKeyword != null && trimmedKeyword.isNotEmpty) {
    queryParameters['keywordSearch'] = trimmedKeyword;
  }

  final normalizedSeverity = severity?.trim().toUpperCase();
  if (normalizedSeverity != null && normalizedSeverity.isNotEmpty) {
    queryParameters['cvssV3Severity'] = normalizedSeverity;
  }

  try {
    final response = await _dio.get<Map<String, dynamic>>(
      AppStrings.nvdBaseUrl,
      queryParameters: queryParameters,
    );
  }
}
```

```
final vulnerabilities =
  response.data?['vulnerabilities'] as List<dynamic> ?? <dynamic>[];

final threats = vulnerabilities
  .map((item) => item is Map<String, dynamic> ? item['cve'] : null)
  .whereType<Map<String, dynamic>>()
  .map(ThreatAdvisory.fromJson)
  .toList();

await _cacheThreats(
  page: page,
  keyword: trimmedKeyword ?? '',
  severity: normalizedSeverity ?? '',
  threats: threats,
);

return threats;
} on DioException catch (error) {
  errorMessage = _mapNvdError(error);
  return <ThreatAdvisory>[];
} catch (_) {
  errorMessage = 'Could not load threat intelligence. Please try again.';
  return <ThreatAdvisory>[];
}
}
```

Pagination: $\text{startIndex} = \text{page} * \text{nvdResultsPerPage}$
 => page 0 start at 0, page 1 starts at 20, page 2 starts at 40

The method parses the vulnerabilities array and converts every item into a ThreatAdvisory model

3- NVD Error Handling

I added readable error messages for NVD failures

```
// -- Error mapping --
String _mapNvdError(DioException error) {
  switch (error.response?.statusCode) {
    case 403:
      return 'Invalid API key. Please check your NVD API key in settings.';
    case 429:
      return 'NVD rate limit reached. Please wait a moment.';
    case 503:
      return 'NVD service is unavailable. Please try again soon.';
    default:
      if (error.type == DioExceptionType.connectionError ||
          error.type == DioExceptionType.connectionTimeout ||
          error.type == DioExceptionType.receiveTimeout) {
        return 'Network error. Please check your connection.';
      }
      return 'Could not load threat intelligence. Please try again.';
  }
}
```

This prevents crashes and gives the user a clear message

4- checkPassword

I added the checkPassword method for Have I Been Pwned

How it works?

- The password is converted to a SHA-1 hash
- The hash is converted to uppercase
- The first 5 characters are sent to HIBP
- HIBP returns matching hash suffixes
- The app checks locally if the remaining suffix exists
- If found, it returns the breach count
- If not found, the password is considered not seen in known breaches

Privacy guarante:

The full password never leaves the device. The full hash also never leaves the device. Only the first 5 characters of the hash are sent.

```
// -- HIBP password range check --
Future<PasswordCheckResult> checkPassword(String password) async {
  errorMessage = '';

  try {
    final digest = sha1.convert(utf8.encode(password));
    final hash = digest.toString().toUpperCase();
    final prefix = hash.substring(0, 5);
    final suffix = hash.substring(5);

    final response = await _dio.get<String>({
      uri: '${AppStrings.hibpBaseUrl}/${prefix}',
      options: Options(responseType: ResponseType.plain),
    });
  }
```

```
    final lines = (response.data ?? '').split('\n');
    for (final line in lines) {
      final parts = line.trim().split(':');
      if (parts.length != 2) continue;

      if (parts.first.toUpperCase() == suffix) {
        final count = int.tryParse(parts.last) ?? 0;
        return PasswordCheckResult(
          isPwned: true,
          breachCount: count,
          password: password,
        );
      }
    }
  }
```

```
  return PasswordCheckResult(
    isPwned: false,
    breachCount: 0,
    password: password,
  );
}
```

```
},
} catch (_) {
  errorMessage = 'Could not connect to breach database. Please try again.';
  return PasswordCheckResult(
    isPwned: false,
    breachCount: 0,
    password: password,
  );
}
```


5- Hive Caching

I added basic caching for fetched threats

After a successful NVD response, the app stores the parsed threats in a Hive box

This prepares the project for the offline capability deliverable.

```
// -- Local cache --
Future<void> _cacheThreats({
  required int page,
  required String keyword,
  required String severity,
  required List<ThreatAdvisory> threats,
}) async {
  try {
    final box = await Hive.openBox<dynamic>(AppHiveBoxes.threatCache);
    final key = _cacheKey(page: page, keyword: keyword, severity: severity);
    await box.put(
      key,
      threats.map((threat) => threat.toCacheJson()).toList(),
    );
  } catch (_) {
    // Cache writes should never block the live API experience.
  }
}

String _cacheKey({
  required int page,
  required String keyword,
  required String severity,
}) {
  return 'page=$page|keyword=$keyword|severity=$severity';
}
```

Controllers

#controllers/threat_feed_controller

I replaced the empty ThreatFeedController with a full GetX controller

This controller manages the threat feed screen state.

1- onInit()

when the controller starts, it immediately calls fetchThreats().

This loads the first page of CVEs when the dashboard opens.

```
class ThreatFeedController extends GetxController {
  final ApiService _apiService = Get.find<ApiService>();

  RxList<ThreatAdvisory> threats = <ThreatAdvisory>[].obs;
  RxBool isLoading = false.obs;
  RxBool isLoadingMore = false.obs;
  RxString errorMessage = ''.obs;
  RxString selectedSeverity = ''.obs;
  RxString searchKeyword = ''.obs;
  RxInt currentPage = 0.obs;
  RxBool hasMorePages = true.obs;

  @override
  void onInit() {
    super.onInit();
    fetchThreats(refresh: true);
  }
}
```

2- fetchThreats()

This method loads the first page of threats

It is used for: initial loading, pull-to-refresh, search refresh, filter refresh

It clears old errors at the start, and sets isLoading to true while the API request is running.

```

Future<void> fetchThreats((bool refresh = false)) async {
    errorMessage.value = '';
    isLoading.value = true;

    if (refresh) {
        currentPage.value = 0;
        hasMorePages.value = true;
    }

    try {
        final results = await _apiService.fetchThreats(
            page: 0,
            keyword: searchKeyword.value,
            severity: selectedSeverity.value,
        );

        threats.assignAll(results);
        currentPage.value = 0;
        hasMorePages.value = results.length == AppStrings.nvdResultsPerPage;

        if (_apiService.errorMessage.isNotEmpty) {
            errorMessage.value = _apiService.errorMessage;
        }
    } catch (_) {
        errorMessage.value =
            'Could not load threat intelligence. Please try again.';
    } finally {
        isLoading.value = false;
    }
}

```

3- loadMoreThreats

This method is used for infinite scrolling.

When the user scrolls near the bottom, it loads the next page from NVD.

```

Future<void> loadMoreThreats() async {
    errorMessage.value = '';

    if (isLoading.value || isLoadingMore.value || !hasMorePages.value) {
        return;
    }

    isLoadingMore.value = true;

    try {
        final nextPage = currentPage.value + 1;
        final results = await _apiService.fetchThreats(
            page: nextPage,
            keyword: searchKeyword.value,
            severity: selectedSeverity.value,
        );

        if (_apiService.errorMessage.isNotEmpty) {
            errorMessage.value = _apiService.errorMessage;
            return;
        }

        threats.addAll(results);
        currentPage.value = nextPage;
        hasMorePages.value = results.Length == AppStrings.nvdResultsPerPage;
    } catch (_) {
        errorMessage.value = 'Could not load more threats. Please try again.';
    } finally {
        isLoadingMore.value = false;
    }
}

```

4- Filtering and Search

The filterBySeverity() method updates the selected severity and reloads the feed.

The search() method updates the search keyword and reloads the feed

The clearFilters() method resets the selectedSeverity and the searchKeyword, then it reloads the full threat feed.

```
// -- Filtering and search --
Future<void> filterBySeverity(String severity) async {
  errorMessage.value = '';
  selectedSeverity.value = severity.toUpperCase() == 'ALL' ? '' : severity;
  await fetchThreats(refresh: true);
}

Future<void> search(String keyword) async {
  errorMessage.value = '';
  searchKeyword.value = keyword.trim();
  await fetchThreats(refresh: true);
}

Future<void> clearFilters() async {
  errorMessage.value = '';
  selectedSeverity.value = '';
  searchKeyword.value = '';
  await fetchThreats(refresh: true);
}
```

#controllers/password_controller

I created the GetX controller instead of the empty PasswordController
This controller manages the Password Health Screen.

1- checkPassword()

This method:

- Clears previous errors
- Clears the old result
- Validates the password using Validators.password()
- Calls ApiService.checkPassword()
- Stores the result
- Shows an error message if HIBP fails.

```
// -- Breach check --
Future<void> checkPassword(String password) async {
  errorMessage.value = '';
  result.value = null;

  final validationError = Validators.password(password);
  if (validationError != null) {
    errorMessage.value = validationError;
    return;
  }

  isChecking.value = true;

  try {
    final checkResult = await _apiService.checkPassword(password);
    if (_apiService.errorMessage.isNotEmpty) {
      errorMessage.value = _apiService.errorMessage;
      return;
    }
    result.value = checkResult;
  } catch (_) {}
  errorMessage.value =
    'Could not connect to breach database. Please try again.';
  finally {
    isChecking.value = false;
  }
}
```

2- clearResult()

This method clears the result and error message.

It is used when the user taps "Check another password"

```
// -- Reset --
void clearResult() {
    errorMessage.value = '';
    result.value = null;
}
```

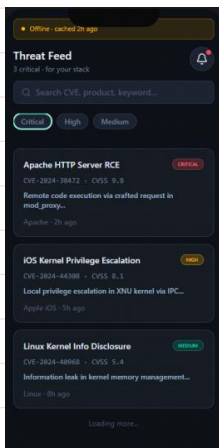
Views

#views/home/threat_feed_view

I created the Threat Feed screen.

This screen replaces the temporary home screen as the main dashboard tab.

The screen will look like this:



```
void initState() {
    super.initState();
    searchController = TextEditingController();
    scrollController = ScrollController();
    scrollController.addListener(_handleScroll);
}

@override
void dispose() {
    _searchDebounce?.cancel();
    searchController.dispose();
    scrollController.dispose();
    super.dispose();
}

void _handleScroll() {
    if (!scrollController.hasClients) return;

    final threshold = scrollController.position.maxScrollExtent - 220;
    if (scrollController.position.pixels >= threshold) {
        Get.find<ThreatFeedController>().loadMoreThreats();
    }
}

void _handleSearchChanged(String value) {
    _searchDebounce?.cancel();
    _searchDebounce = Timer(const Duration(milliseconds: 500), () {
        Get.find<ThreatFeedController>().search(value);
    });
}
```

```

Obx(
  () => _SearchBar(
    controller: searchController,
    enabled: !controller.isLoading.value &&
      !controller.isLoadingMore.value,
    onChanged: _handleSearchChanged,
    onSubmitted: controller.search,
  ),
),
const SizedBox(height: 14),
Obx(
  () => _SeverityFilters(
    selectedSeverity: controller.selectedSeverity.value,
    isDisabled: controller.isLoading.value ||
      controller.isLoadingMore.value,
    onSelected: controller.filterBySeverity,
  ),
),
const SizedBox(height: 14),
Obx(() {
  if (controller.errorMessage.value.isEmpty) {
    return const SizedBox.shrink();
  }
  return _ErrorBanner(
    message: controller.errorMessage.value,
    isRetrying: controller.isLoading.value,
    onRetry: () => controller.fetchThreats(refresh: true),
  );
});

```

1- Part of Search Bar

```

@override
Widget build(BuildContext context) {
  return TextField(
    controller: controller,
    enabled: enabled,
    onChanged: onChanged,
    onSubmitted: onSubmitted,
    style: const TextStyle(color: AppColors.white, fontSize: 15),
    decoration: InputDecoration(
      hintText: 'Search CVEs...',
      hintStyle: const TextStyle(color: AppColors.textHint, fontSize: 15),
      prefixIcon: const Icon(
        Icons.search,
        color: AppColors.textMuted,
        size: 20,
      ),
      filled: true,
      fillColor: AppColors.surface,
      contentPadding: const EdgeInsets.symmetric(
        horizontal: 16,
        vertical: 16,
      ),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10),
        borderSide: const BorderSide(color: AppColors.border),
      ),
    ),
  ),
);

```

2- Part of Severity Filters

```

Widget build(BuildContext context) {
  const filters = ['ALL', 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'];

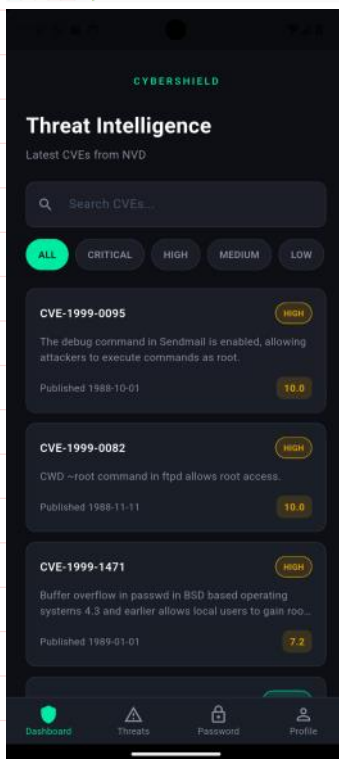
  return SizedBox(
    height: 30,
    child: ListView.separated(
      scrollDirection: Axis.horizontal,
      itemCount: filters.length,
      separatorBuilder: (_, __) => const SizedBox(width: 8),
      itemBuilder: (context, index) {
        final filter = filters[index];
        final isSelected = selectedSeverity.isEmpty
          ? filter == 'ALL'
          : filter == selectedSeverity;

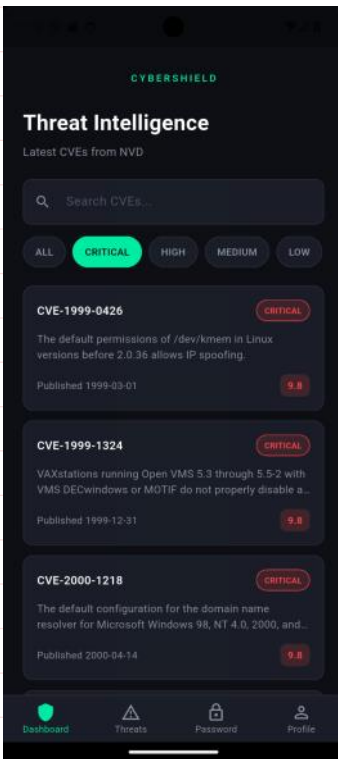
        return GestureDetector(
          onTap: isSelected ? null : () => onSelected(filter),
          child: AnimatedContainer(
            duration: const Duration(milliseconds: 160),
            padding: const EdgeInsets.symmetric(horizontal: 14),
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: isSelected ? AppColors.primary : AppColors.surface,
              borderRadius: BorderRadius.circular(999),
              border: Border.all(
                color: isSelected ? AppColors.primary : AppColors.border,
              ),
            ),
          ),
        );
      },
    ),
  );
}

```

3- Part of Threat Cards

```
Widget build(BuildContext context) {  
  return InkWell(  
    borderRadius: BorderRadius.circular(12),  
    onTap: onTap,  
    child: Container(  
      width: double.infinity,  
      padding: const EdgeInsets.all(16),  
      decoration: BoxDecoration(  
        color: AppColors.surface,  
        borderRadius: BorderRadius.circular(12),  
        border: Border.all(color: AppColors.border),  
      ),  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: [  
          Row(  
            children: [  
              Expanded(  
                child: Text(  
                  threat.id,  
                  style: const TextStyle(  
                    color: AppColors.white,  
                    fontSize: 14,  
                    fontWeight: FontWeight.w600,  
                  ),  
                ),  
              _SeverityChip(threat: threat),  
            ],  
          ),  
        ],  
      ),  
    ),  
  ),  
);
```





#views/home/threat_detail_view

I created a basic threat detail screen. This screen opens when the user taps a CVE card. It is currently a stub-style detail screen, but it displays real data from the selected ThreatAdvisory model

```
class _SeverityChip extends StatelessWidget {
  final ThreatAdvisory threat;

  const _SeverityChip({required this.threat});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
      decoration: BoxDecoration(
        color: threat.severityColor.withValues(alpha: 0.15),
        borderRadius: BorderRadius.circular(999),
        border: Border.all(color: threat.severityColor),
      ),
      child: Text(
        threat.severity,
        style: TextStyle(
          color: threat.severityColor,
          fontSize: 11,
          fontWeight: FontWeight.w700,
        ),
      ),
    );
  }
}
```

```

class _ScoreCircle extends StatelessWidget {
  final ThreatAdvisory threat;

  const _ScoreCircle({required this.threat});

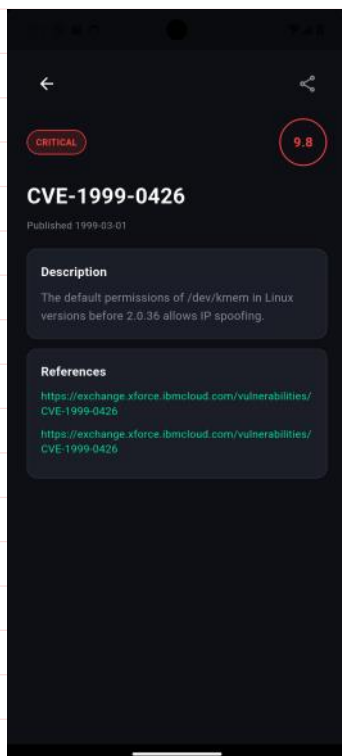
  @override
  Widget build(BuildContext context) {
    return Container(
      width: 58,
      height: 58,
      alignment: Alignment.center,
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        border: Border.all(color: threat.severityColor, width: 2),
      ),
      child: Text(
        threat.baseScore.toStringAsFixed(1),
        style: TextStyle(
          color: threat.severityColor,
          fontSize: 16,
          fontWeight: FontWeight.bold,
        ),
      ),
    );
  }
}

```

```

const SizedBox(height: 18),
Row(
  children: [
    _SeverityChip(threat: threat),
    const Spacer(),
    _ScoreCircle(threat: threat),
  ],
),
const SizedBox(height: 18),
Text(
  threat.id,
  style: const TextStyle(
    color: AppColors.white,
    fontSize: 26,
    fontWeight: FontWeight.bold,
  ),
),
const SizedBox(height: 8),
Text(
  'Published ${_formatDate(threat.publishedDate)}',
  style: const TextStyle(
    color: AppColors.textMuted,
    fontSize: 12,
  ),
),
const SizedBox(height: 20),
_DetailCard(
  title: 'Description',
  child: Text(

```



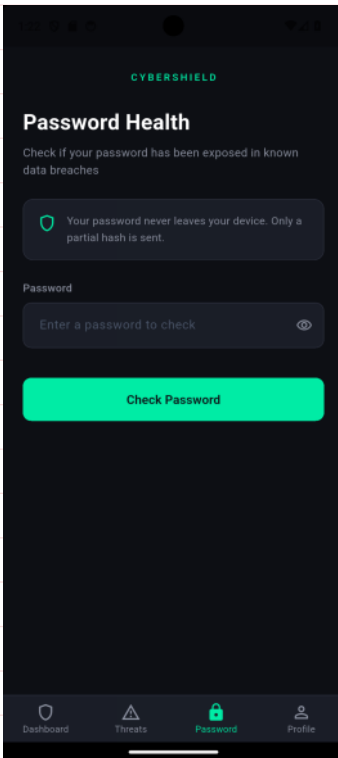
I created the Password Health Screen.

```
const FieldLabel(Label: 'Password'),  
const SizedBox(height: 8),  
Obx(  
  () => _PasswordField(  
    controller: passwordController,  
    enabled: !controller.isChecking.value,  
    obscureText: controller.obscurePassword.value,  
    onToggle: () {  
      controller.obscurePassword.value =  
        !controller.obscurePassword.value;  
    },  
    onChanged: (_) => controller.clearResult(),  
  ),  
,  
const SizedBox(height: 16),  
Obx(() {  
  if (controller.errorMessage.value.isEmpty) {  
    return const SizedBox.shrink();  
  }  
  return _ErrorBanner(message: controller.errorMessage.value);  
}),  
const SizedBox(height: 20),  
Obx(  
  () => _PrimaryButton(  
    label: 'Check Password',  
    isLoading: controller.isChecking.value,  
    onPressed: () => controller.checkPassword(  
      passwordController.text,
```

The info card explains: "Your password never leaves your device. Only a partial hash is sent".

Mobile Project Page 69

```
Widget build(BuildContext context) {
  return Container(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: AppColors.surface,
      borderRadius: BorderRadius.circular(12),
      border: Border.all(color: AppColors.border),
    ),
    child: const Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Icon(Icons.shield_outlined, color: AppColors.primary, size: 24),
        SizedBox(width: 12),
        Expanded(
          child: Text(
            'Your password never leaves your device. Only a partial hash is sent.',
            style: TextStyle(
              color: AppColors.textMuted,
              fontSize: 13,
              height: 1.5,
            ),
          ),
        ),
      ],
    ),
  );
}
```



2- Password Input

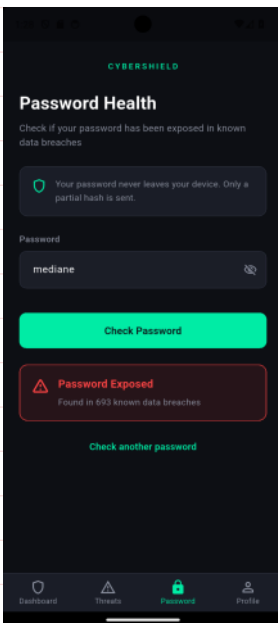
```
Obx(
  () => _PasswordField(
    controller: passwordController,
    enabled: !controller.isChecking.value,
    obscureText: controller.obscurePassword.value,
    onToggle: () {
      controller.obscurePassword.value =
        !controller.obscurePassword.value;
    },
    onChanged: (_) => controller.clearResult(),
  ),
),
const SizedBox(height: 16),
Obx(() {
  if (controller.errorMessage.value.isEmpty) {
    return const SizedBox.shrink();
  }
  return _ErrorBanner(message: controller.errorMessage.value);
}),
const SizedBox(height: 20),
Obx(
  () => _PrimaryButton(
    label: 'Check Password',
    isLoading: controller.isChecking.value,
    onPressed: () => controller.checkPassword(
      passwordController.text,
    ),
  ),
),
),
```

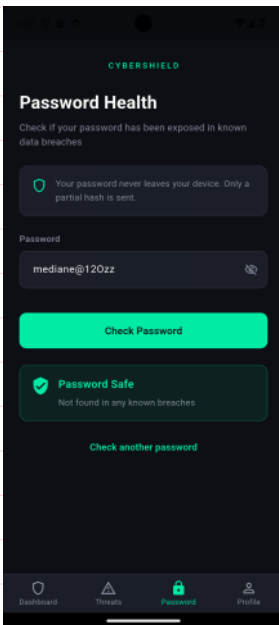
3- Result Card

- If the password is found in breaches: red danger styling, warning icon, "Password Exposed", "Found in X known data breaches"
- If the password is not found: teal success styling, shield icon, "Password Safe", "Not found in any known breaches"

```
@override
Widget build(BuildContext context) {
  final isPwmed = result.isPwmed;
  final color = isPwmed ? AppColors.danger : AppColors.primary;
  final icon = isPwmed ? Icons.warning_amber_rounded : Icons.verified_user;
  final title = isPwmed ? 'Password Exposed' : 'Password Safe';
  final subtitle = isPwmed
    ? 'Found in ${result.breachCount} known data breaches'
    : 'Not found in any known breaches';

  return Column(
    children: [
      Container(
        width: double.infinity,
        padding: const EdgeInsets.all(16),
        decoration: BoxDecoration(
          color: color.withValues(alpha: isPwmed ? 0.1 : 0.08),
          borderRadius: BorderRadius.circular(12),
          border: Border.all(
            color: color.withValues(alpha: isPwmed ? 1 : 0.3),
          ),
        ),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(icon, color: color, size: 28),
            const SizedBox(width: 12),
            Expanded(
```





#views/shell/main_shell_view

I created the main shell view. This is the authenticated app layout.

It uses IndexedStack so each tab keeps its state when switching tabs.

```
bottomNavigationBar: Container(
  decoration: const BoxDecoration(
    color: AppColors.surface,
    border: Border(
      top: BorderSide(color: AppColors.border, width: 1),
    ),
  ),
  child: BottomNavigationBar(
    currentIndex: currentIndex,
    onTap: (index) => setState(() => currentIndex = index),
    type: BottomNavigationBarType.fixed,
    backgroundColor: AppColors.surface,
    selectedItemColor: AppColors.primary,
    unselectedItemColor: AppColors.textMuted,
    selectedFontSize: 11,
    unselectedFontSize: 11,
    elevation: 0,
    items: const [
      BottomNavigationBarItem(
        icon: Icon(Icons.shield_outlined),
        activeIcon: Icon(Icons.shield),
        label: 'Dashboard',
      ),
      BottomNavigationBarItem(
        icon: Icon(Icons.warning_amber_outlined),
        activeIcon: Icon(Icons.warning_amber),
        label: 'Threats',
      ),
    ],
  ),
)
```

- **Dashboard Tab:** shows the ThreatFeedView, this is the main API integration screen for NVD
- **Threats Tab:** is currently a stub screen. It will be used later for saved advisories or more detailed threat workflows.
- **Password Tab:** shows the PasswordCheckerView. It is the API integration screen for HIBP
- **Profile Tab:** is currently a stub profile screen. It shows the logged-in email and includes a sign out button.

Bindings

#bindings/home_binding

I updated HomeBinding. It now registers ApiService and ThreatFeedController.

```
import 'package:get/get.dart';

import '../controllers/threat_feed_controller.dart';
import '../services/api_service.dart';

class HomeBinding extends Bindings {
  @override
  void dependencies() {
    if (!Get.isRegistered<ApiService>()) {
      Get.put(ApiService());
    }
    if (!Get.isRegistered<ThreatFeedController>()) {
      Get.put(ThreatFeedController());
    }
  }
}
```

#bindings/shell_binding

This binding registers all dependencies needed by the authenticated shell:

```
class ShellBinding extends Bindings {
  @override
  void dependencies() {
    if (!Get.isRegistered<AuthService>()) {
      Get.put(AuthService());
    }
    if (!Get.isRegistered<AuthController>()) {
      Get.put(AuthController());
    }
    if (!Get.isRegistered<ApiService>()) {
      Get.put(ApiService());
    }
    if (!Get.isRegistered<ThreatFeedController>()) {
      Get.put(ThreatFeedController());
    }
    if (!Get.isRegistered<PasswordController>()) {
      Get.put>PasswordController());
    }
  }
}
```

Routes

#routes/app_routes && routes/app_pages

I updated both app_routes and app_pages:

```
abstract class AppRoutes {
  static const INITIAL = '/';
  static const HOME = '/home';
  static const SHELL = '/shell';
  static const LOGIN = '/login';
  static const REGISTER = '/register';
  static const FORGOT_PASSWORD = '/forgot-password';
  static const ONBOARDING = '/onboarding';
  static const THREAT_DETAIL = '/threat-detail';
  static const PASSWORD_CHECKER = '/password-checker';
}
```

The old HOME route still exists for compatibility, but the login flow now goes to SHELL.

And I adjusted the navigation in the login view:

```
// login.dart
Obx(() {
  if (controller.currentUser.value != null) {
    Future.microtask(() => Get.offAllNamed(AppRoutes.SHELL));
  }
  return const SizedBox.shrink();
}),
```

```
// -- Authenticated app shell --
GetPage(
  name: AppRoutes.SHELL,
  page: () => const MainShellView(),
  binding: ShellBinding(),
),

// -- Feature routes --
GetPage(
  name: AppRoutes.HOME,
  page: () => const HomeView(),
  bindings: [
    AuthBinding(),
    HomeBinding(),
  ],
),
GetPage(
  name: AppRoutes.THREAT_DETAIL,
  page: () => const ThreatDetailView(),
  binding: ThreatBinding(),
),
GetPage(
  name: AppRoutes.PASSWORD_CHECKER,
  page: () => const PasswordCheckerView(),
  binding: PasswordBinding(),
),
```

Main

#main.dart

I updated the main to initialize Hive

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const CyberShieldApp());
}
```

Test

#test/widget_test

I changed the old default Flutter test.

The new test checks that ThreatAdvisory correctly parses the NVD response shape

```

test('ThreatAdvisory parses the NVD CVE response shape', () {
  final advisory = ThreatAdvisory.fromJson({
    'id': 'CVE-2024-12345',
    'published': '2024-03-15T10:00:00.000',
    'lastModified': '2024-03-16T08:00:00.000',
    'descriptions': [
      {'lang': 'en', 'value': 'A vulnerability in CyberShield dependencies.'},
    ],
    'metrics': {
      'cvssMetricV31': [
        {
          'cvssData': {
            'baseScore': 9.8,
            'baseSeverity': 'CRITICAL',
          },
        },
      ],
    },
  },
  'references': [
    {'url': 'https://example.com/advisory'},
  ],
});

expect(advisory.id, 'CVE-2024-12345');
expect(advisory.description, contains('CyberShield'));
expect(advisory.baseScore, 9.8);
expect(advisory.severity, 'CRITICAL');
expect(advisory.severityColor, AppColors.danger);
expect(advisory.references, ['https://example.com/advisory']);

```

Error Handling:

I added error Handling so API failures do not crash the app

```

// -- Error mapping --
String _mapNvdError(DioException error) {
  switch (error.response?.statusCode) {
    case 403:
      return 'Invalid API key. Please check your NVD API key in settings.';
    case 429:
      return 'NVD rate limit reached. Please wait a moment.';
    case 503:
      return 'NVD service is unavailable. Please try again soon.';
    default:
      if (error.type == DioExceptionType.connectionError ||
          error.type == DioExceptionType.connectionTimeout ||
          error.type == DioExceptionType.receiveTimeout) {
        return 'Network error. Please check your connection.';
      }
      return 'Could not load threat intelligence. Please try again.';
  }
}

```

Loading States

I added loading states for API actions.

Threat feed:

- Main loading spinner
- Pull-to-refresh
- Load-more spinner

```

Obx(() {
  if (controller.errorMessage.value.isEmpty) {
    return const SizedBox.shrink();
  }
  return _ErrorBanner(
    message: controller.errorMessage.value,
    isRetrying: controller.isLoading.value,
    onRetry: () => controller.fetchThreats(refresh: true),
  );
}),

```

```

child: Obx(() {
  if (controller.isLoading.value &&
    controller.threats.isEmpty) {
    return const Center(
      child: CircularProgressIndicator(
        color: AppColors.primary,
      ),
    );
  }
}

```

```

itemBuilder: (context, index) {
  if (index >= controller.threats.length) {
    return const Padding(
      padding: EdgeInsets.symmetric(vertical: 18),
      child: Center(
        child: SizedBox(
          width: 22,
          height: 22,
          child: CircularProgressIndicator(
            color: AppColors.primary,
            strokeWidth: 2,
          ),
        ),
      ),
    );
  }

  return RefreshIndicator(
    color: AppColors.primary,
    backgroundColor: AppColors.surface,
    onRefresh: () => controller.fetchThreats(refresh: true),
  );
}

```

Password checker:

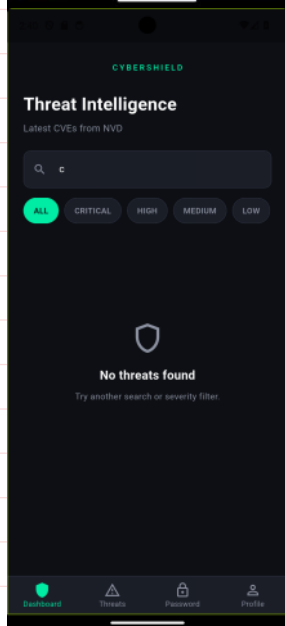
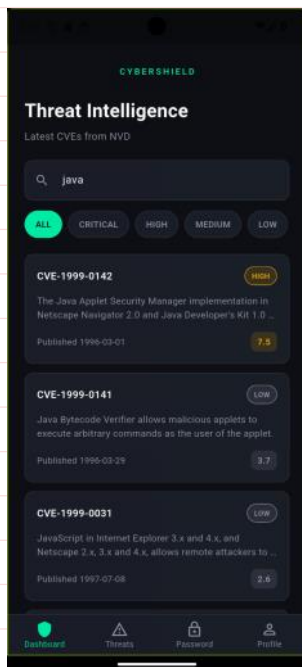
- Button loading spinner
- Input disabled while checking

```

Obx(
  () => _PasswordField(
    controller: passwordController,
    enabled: !controller.isChecking.value,
    obscureText: controller.obscurePassword.value,
    onToggle: () {
      controller.obscurePassword.value =
        !controller.obscurePassword.value;
    },
    onChanged: (_) => controller.clearResult(),
  ),
)

```

The search functionality:



Core Features

Core Features

In this step I will implement Deliverable 6: Core Features for the Cybershield Flutter mobile app.

The goal of this steps is to complete the main authenticated experience of the app after the API integration work from deliverable 5.

Deliverable 6 adds three major app features:

- Full Password Shield screen
- Offline Security Knowledge Base
- Full Profile screen

I also upgraded the bottom navigation shell so the app now exposes five tabs: Feed, Shield, Library, Alerts, Profile

The app already had:

- Authentication flow
- Login screen
- Register screen
- Forgot password screen
- Main shell view
- NVD threat feed
- HIBP password breach check
- Hive initialized

This deliverable builds on top of those completed features.

When I finish, the app should contain:

- 1- A stronger password analysis screen
- 2- Real-time password strength feedback
- 3- HIBP breach checking from the Shield tab
- 4- An offline cybersecurity library
- 5- Local article caching with Hive
- 6- Persistent bookmarks
- 7- Article search and category filtering
- 8- A full profile screen

- 9- Watchlist chips
- 10- Notification toggles
- 11- Last synced app info
- 12- Sign out confirmation dialog
- 13- Five-tab bottom navigation
- 14- Alerts empty state.

utils/password_analyzer

I expanded the shared PasswordAnalyzer utility.

Before this deliverable, the analyzer only returned:
strength level, bar colors, label, label color, border color

For Deliverable 6, the analyzer now returns a complete PasswordStrengthResult.

Added fields:

length, entropyBits, hasUppercase, hasNumbers, hasSymbols, hasDictionaryWord

```
class PasswordStrengthResult {  
    final PasswordStrength level;  
    final List<Color> bars;  
    final String label;  
    final Color labelColor;  
    final Color borderColor;  
    final int length;  
    final double entropyBits;  
    final bool hasUppercase;  
    final bool hasNumbers;  
    final bool hasSymbols;  
    final bool hasDictionaryWord;  
  
    const PasswordStrengthResult({  
        required this.level,  
        required this.bars,  
        required this.label,  
        required this.labelColor,  
        required this.borderColor,  
        required this.length,  
        required this.entropyBits,  
        required this.hasUppercase,  
        required this.hasNumbers,  
        required this.hasSymbols,  
        required this.hasDictionaryWord,  
    });  
}
```

```
bool get hasPassword => level != PasswordStrength.empty;  
  
bool get isVeryStrong => level == PasswordStrength.veryStrong;  
  
String get dictionaryLabel => hasDictionaryWord ? 'Detected' : 'Clean';  
  
String? get improvementTip {  
    if (isVeryStrong) {  
        return null;  
    }  
    if (length < 12) {  
        return 'Use at least 12 characters';  
    }  
    if (!hasUppercase) {  
        return 'Add uppercase letters to increase entropy';  
    }  
    if (!hasNumbers) {  
        return 'Add numbers to increase entropy';  
    }  
    if (!hasSymbols) {  
        return 'Add a special character (!@#%) to increase entropy';  
    }  
    if (hasDictionaryWord) {  
        return 'Avoid common words and predictable phrases';  
    }  
    return 'Use more variety to increase entropy';  
}
```

I added a hardcoded common wordlist:

```
static const Color _empty = Color(0xFF2A203A);
static const Color _red = Color(0xFFFF4444);
static const Color _amber = Color(0xFFE5A000);
static const Color _teal = Color(0xFF00E5A0);
static const Color _green = Color(0xFF00C853);

static const List<String> _commonWords = [
  'password',
  'letmein',
  'welcome',
  'admin',
  'login',
  'qwerty',
  'abc',
  'monkey',
  'dragon',
  'master',
  'hello',
  'shadow',
  'sunshine',
  'princess',
  'football',
  'baseball',
  'soccer',
  'trustno',
  'iloveyou',
  'superman',
];
```

If a word is detected, the analysis card shows: Dictionary => Detected

If no word is detected, it shows: Dictionary => Clean

Entropy Calculation

```
static PasswordStrengthResult evaluate(String value) {
  final length = value.length;
  final hasLowercase = value.contains(RegExp(r'[a-z]'));
  final hasUppercase = value.contains(RegExp(r'[A-Z]'));
  final hasNumbers = value.contains(RegExp(r'[0-9]'));
  final hasSymbols = value.contains(RegExp(r'[!@#$%^&*~`~\|;:,./\s]'));
  final normalized = value.toLowerCase();
  final hasDictionaryWord = _commonWords.any(
    (word) => word.length > 4 && normalized.contains(word),
  );
  final entropyBits = _calculateEntropyBits(
    length: length,
    hasLowercase: hasLowercase,
    hasUppercase: hasUppercase,
    hasNumbers: hasNumbers,
    hasSymbols: hasSymbols,
  );
  if (value.isEmpty) {
    return PasswordStrengthResult(
      level: PasswordStrength.empty,
      bars: List.generate(4, (_) => _empty),
      label: '',
      labelColor: Colors.transparent,
      borderColor: _empty,
      length: 0,
      entropyBits: 0,
      hasUppercase: false,
      hasNumbers: false,
      hasSymbols: false,
      hasDictionaryWord: false,
    );
  }
  int score = 0;
  if (length >= 12) score += 2;
  if (length >= 8 && length < 12) score++;
  if (hasLowercase && hasUppercase) score++;
  if (hasNumbers) score++;
  if (hasSymbols) score++;
  if (!hasDictionaryWord) score++;
  if (entropyBits >= 72) score++;
```

```

static double _calculateEntropyBits({
    required int length,
    required bool hasLowercase,
    required bool hasUppercase,
    required bool hasNumbers,
    required bool hasSymbols,
}) {
    var charsetSize = 0;
    if (hasLowercase) {
        charsetSize += 26;
    }
    if (hasUppercase) {
        charsetSize += 26;
    }
    if (hasNumbers) {
        charsetSize += 10;
    }
    if (hasSymbols) {
        charsetSize += 32;
    }
    if (charssetSize == 0) {
        return 0;
    }
    return length * (math.log(charssetSize) / math.ln2);
}

```

Strength Levels

The analyzer still returns the four strength levels:

```

if (score <= 2) {
    return PasswordStrengthResult(
        level: PasswordStrength.weak,
        bars: [_red, _empty, _empty, _empty],
        label: 'Weak',
        labelColor: _red,
        borderColor: _red,
        length: length,
        entropyBits: entropyBits,
        hasUppercase: hasUppercase,
        hasNumbers: hasNumbers,
        hasSymbols: hasSymbols,
        hasDictionaryWord: hasDictionaryWord,
    );
}

if (score <= 3) {
    return PasswordStrengthResult(
        level: PasswordStrength.fair,
        bars: [_amber, _amber, _empty, _empty],
        label: 'Fair',
        labelColor: _amber,
        borderColor: _amber,
        length: length,
        entropyBits: entropyBits,
        hasUppercase: hasUppercase,
        hasNumbers: hasNumbers,
        hasSymbols: hasSymbols,
        hasDictionaryWord: hasDictionaryWord,
    );
}

```

```

if (score <= 5) {
    return PasswordStrengthResult(
        level: PasswordStrength.strong,
        bars: [_teal, _teal, _teal, _empty],
        label: 'Strong',
        labelColor: _teal,
        borderColor: _teal,
        length: length,
        entropyBits: entropyBits,
        hasUppercase: hasUppercase,
        hasNumbers: hasNumbers,
        hasSymbols: hasSymbols,
        hasDictionaryWord: hasDictionaryWord,
    );
}

return PasswordStrengthResult(
    level: PasswordStrength.veryStrong,
    bars: List.generate(4, (_) => _green),
    label: 'Very Strong',
    labelColor: _green,
    borderColor: _green,
    length: length,
    entropyBits: entropyBits,
    hasUppercase: hasUppercase,
    hasNumbers: hasNumbers,
    hasSymbols: hasSymbols,
    hasDictionaryWord: hasDictionaryWord,
);

```

#controllers/password_controller

I replaced the previous PasswordController with a full Password Shield controller

The controller now separates:

- local strength analysis
- HIBP breach checking
- password visibility state
- breach result state

I added `analyzePassword()`. This method runs every time the user types in the password field.

This is important because if the user changes the password after checking a breach result, the old breach result should no longer be shown.

```
class PasswordController extends GetXController {
    final ApiService _apiService = Get.find<ApiService>();

    RxString password = ''.obs;
    Rx<PasswordStrengthResult> strengthResult = PasswordAnalyzer.evaluate('').obs;
    RxBool isBreachChecked = false.obs;
    Rxn<PasswordCheckResult> breachResult = Rxn<PasswordCheckResult>();
    RxString errorMessage = ''.obs;
    RxBool isChecking = false.obs;
    RxBool obscureText = true.obs;

    void analyzePassword(String value) {
        password.value = value;
        strengthResult.value = PasswordAnalyzer.evaluate(value);
        isBreachChecked.value = false;
        breachResult.value = null;
        errorMessage.value = '';
    }
}
```

I added `checkBreach()`. This method only runs when the user taps: Check for breaches. The local strenght analysis does not call the API. The API is onlt called by the button.

```
Future<void> checkBreach() async {
    final currentPassword = password.value;
    if (currentPassword.isEmpty || isChecking.value) {
        return;
    }
}
```

I added `clearAll()`. It resets: password, strengthResult and others. It is called when the user clear the password input.

```
void clearAll() {
    password.value = '';
    strengthResult.value = PasswordAnalyzer.evaluate('');
    isBreachChecked.value = false;
    breachResult.value = null;
    errorMessage.value = '';
    isChecking.value = false;
    obscureText.value = true;
}
```

#services/api_service

I updated the HIBP result creation in `ApiService`.

`ApiService.checkPassword` still works the same way.

But now the returned `PasswordCheckResult` does not include the original password. This keeps the API integration privacy-safe.

```

return PasswordCheckResult(
  isPwned: false,
  breachCount: 0,
);
} on DioException catch (_) {
  errorMessage = 'Could not connect to breach database. Please try again.';
  return PasswordCheckResult(
    isPwned: false,
    breachCount: 0,
  );
} catch (_) {
  errorMessage = 'Could not connect to breach database. Please try again.';
  return PasswordCheckResult(
    isPwned: false,
    breachCount: 0,
  );
}

```

#views/password/password_checker_view

I rebuilt the Password Checker screen into the full Password Shield screen.

```

return Scaffold(
  backgroundColor: AppColors.background,
  body: SafeArea(
    child: ListView(
      padding: const EdgeInsets.symmetric(horizontal: 24),
      children: [
        const SizedBox(height: 28),
        const Row(
          children: [
            Expanded(
              child: Text(
                'Password Shield',
                style: TextStyle(
                  color: AppColors.white,
                  fontSize: 26,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ),
            Text(
              'Nothing is stored',
              style: TextStyle(color: AppColors.textMuted, fontSize: 12),
            ),
          ],
        ),
        const SizedBox(height: 28),
        const _FieldLabel(label: 'Enter password to analyze'),
        const SizedBox(height: 8),
      ],
    ),
  ),
);

```

Password Input:

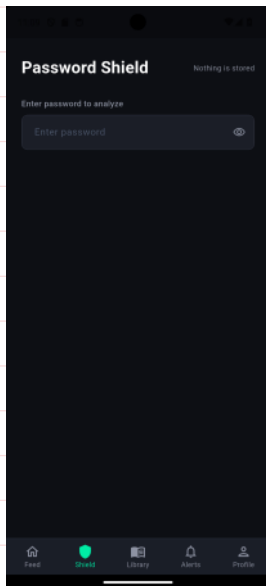
The password input now supports:

- hidden password by default
- visibility toggle
- real-time analysis
- dynamic border color

```

@override
Widget build(BuildContext context) {
  return TextField(
    controller: controller,
    obscureText: obscureText,
    onChanged: onChanged,
    style: const TextStyle(color: AppColors.white, fontSize: 15),
    decoration: InputDecoration(
      hintText: 'Enter password',
      hintStyle: const TextStyle(color: AppColors.textHint, fontSize: 15),
      filled: true,
      fillColor: AppColors.surface,
      suffixIcon: GestureDetector(
        onTap: onTap,
        child: Icon(
          obscureText
            ? Icons.visibility_outlined
            : Icons.visibility_off_outlined,
          color: AppColors.textMuted,
          size: 20,
        ),
      ),
    ),
    contentPadding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 16,
    ),
  );
}

```



Strength Row

The strength row appears only when the input is not empty

```
const Expanded(
  child: Text(
    'Strength',
    style: TextStyle(color: AppColors.textMuted, fontSize: 13),
  ),
),
Text(
  result.label,
  style: TextStyle(
    color: result.labelColor,
    fontSize: 13,
    fontWeight: FontWeight.w600,
  ),
),
],
),
const SizedBox(height: 10),
Row(
  children: List.generate(4, (index) {
    return Expanded(
      child: Container(
        height: 4,
        margin: EdgeInsets.only(right: index < 3 ? 4 : 0),
        decoration: BoxDecoration(
          color: result.bars[index],
          borderRadius: BorderRadius.circular(4),
          border: result.bars[index] == AppColors.border
            ? Border.all(color: AppColors.border)
            : null
        )
      )
    )
  })
)
```

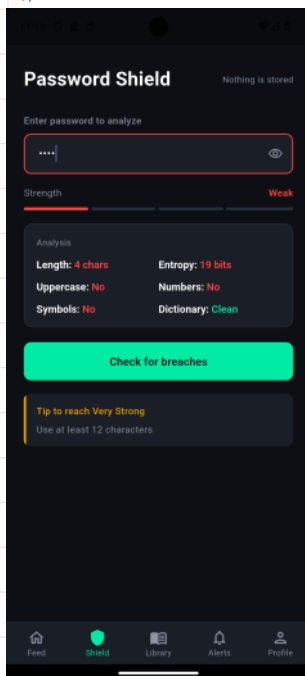
Analysis Card

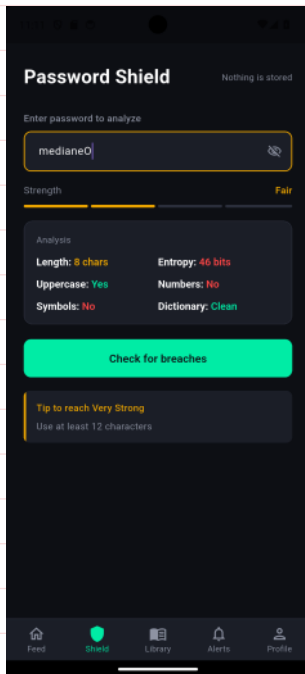
It shows: Length, Entropy, Uppercase, Numbers, Symbols, Dictionary


```

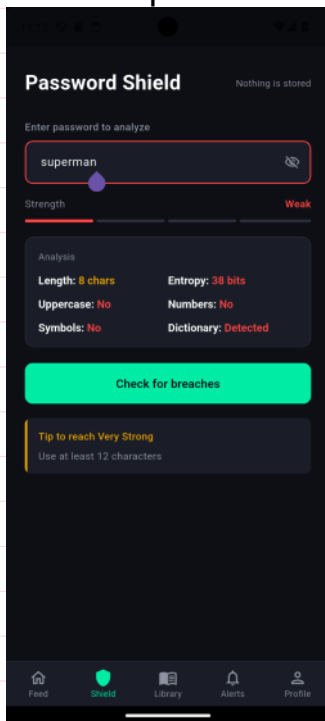
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    const Text(
      'Analysis',
      style: TextStyle(color: AppColors.textMuted, fontSize: 12),
    ),
    const SizedBox(height: 14),
    Row(
      children: [
        Expanded(
          child: _StatItem(
            label: 'Length',
            value: '${result.length} chars',
            valueColor: _lengthColor(result.length),
          ),
        ),
        Expanded(
          child: _StatItem(
            label: 'Entropy',
            value: '${result.entropyBits.round()} bits',
            valueColor: _entropyColor(result.entropyBits),
          ),
        ),
      ],
    ),
    const SizedBox(height: 14),
    Row(
      children: [
        Expanded(
          child: _StatItem(
            label: 'Uppercase',
            value: result.hasUppercase ? '✓' : '✗',
            valueColor: result.hasUppercase
              ? AppColors.primary
              : AppColors.danger,
          ),
        ),
        Expanded(
          child: _StatItem(
            label: 'Numbers',
            value: result.hasNumbers ? '✓' : '✗',
            valueColor:
              result.hasNumbers ? AppColors.primary : AppColors.danger,
          ),
        ),
      ],
    ),
  ],
),

```

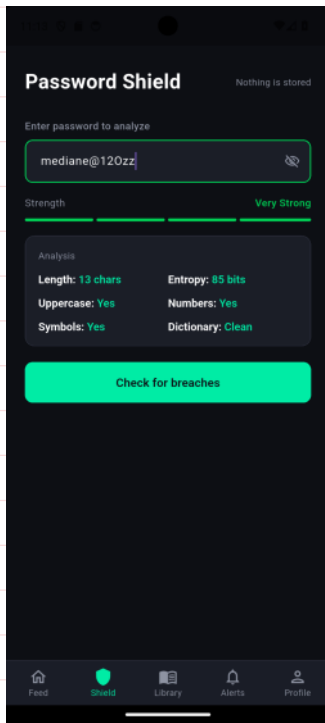




Word Superman is detected in the dictionary:



The improvement tip card appears until the password becomes Very Strong

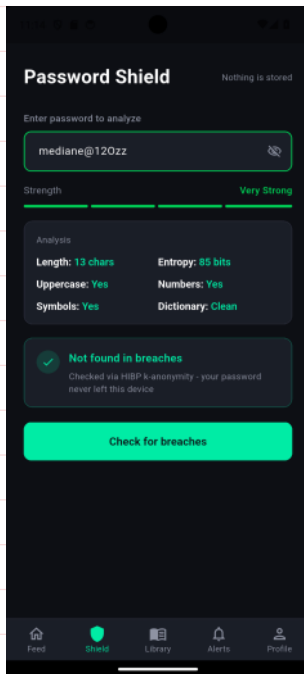


Breach Result Card

The breach result card appears after the HIBP check finishes.

```
final isPwned = result.isPwned;
final color = isPwned ? AppColors.danger : AppColors.primary;
final title = isPwned
  ? 'Found in ${_formatCount(result.breachCount)} breaches'
  : 'Not found in breaches';
final subtitle = isPwned
  ? 'This password is compromised. Change it immediately.'
  : 'Checked via HIBP k-anonymity · your password never left this device';

return Container(
  width: double.infinity,
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: AppColors.surface,
    borderRadius: BorderRadius.circular(12),
    border: Border.all(color: color.withValues(alpha: 0.3)),
  ),
  child: Row(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Container(
        width: 32,
        height: 32,
        decoration: BoxDecoration(
          color: color.withValues(alpha: 0.15),
          shape: BoxShape.circle,
        ),
        child: Icon(
```



#models/knowledge_article

I replaced the empty KnowledgeArticle stub with a full model

The category can be:

- Passwords
- Network
- Pishing
- Mobile
- Incident

I also added fromJson and toJson

```
class KnowledgeArticle {  
  final String id;  
  final String title;  
  final String summary;  
  final String category;  
  final int readTimeMinutes;  
  final String content;  
  final bool isCached;  
  
  const KnowledgeArticle({  
    required this.id,  
    required this.title,  
    required this.summary,  
    required this.category,  
    required this.readTimeMinutes,  
    required this.content,  
    this.isCached = true,  
  });  
}
```

```

factory KnowledgeArticle.fromJson(Map<String, dynamic> json) {
  return KnowledgeArticle(
    id: json['id']?.toString() ?? '',
    title: json['title']?.toString() ?? '',
    summary: json['summary']?.toString() ?? '',
    category: json['category']?.toString() ?? '',
    readTimeMinutes: int.tryParse(json['readTimeMinutes']?.toString()) ?? 1,
    content: json['content']?.toString() ?? '',
    isCached: json['isCached'] as bool? ?? true,
  );
}

Map<String, dynamic> toJson() {
  return {
    'id': id,
    'title': title,
    'summary': summary,
    'category': category,
    'readTimeMinutes': readTimeMinutes,
    'content': content,
    'isCached': isCached,
  };
}

```

#services/storage_service

I replaced the empty StorageService with a full Hive-backed storage service.

Hive boxes: knowledge_articles, bookmarks, sync_meta

```

class StorageService {
  static const knowledgeArticlesBox = 'knowledge_articles';
  static const bookmarksBox = 'bookmarks';
  static const syncMetaBox = 'sync_meta';

  Box<dynamic>? _articlesBox;
  Box<dynamic>? _bookmarksBox;
  Box<dynamic>? _syncMetaBox;

  Future<void> init() async {
    _articlesBox ??= await Hive.openBox<dynamic>(knowledgeArticlesBox);
    _bookmarksBox ??= await Hive.openBox<dynamic>(bookmarksBox);
    _syncMetaBox ??= await Hive.openBox<dynamic>(syncMetaBox);
  }

  Future<void> saveArticles(List<KnowledgeArticle> articles) async {
    await init();
    await _articlesBox!.clear();
    for (final article in articles) {
      await _articlesBox!.put(article.id, article.toJson());
    }
  }

  List<KnowledgeArticle> getCachedArticles() {
    final values = _articlesBox?.values ?? const Iterable<dynamic>.empty();
    return values
      .whereType<Map>()
      .map((item) =>
        KnowledgeArticle.fromJson(Map<String, dynamic>.from(item)))
      .where((article) => article.id.isNotEmpty)
      .toList();
  }

  Future<void> toggleBookmark(String articleId) async {
    await init();
    final isBookmarked = _bookmarksBox!.get(articleId) == true;
    if (isBookmarked) {
      await _bookmarksBox!.delete(articleId);
      return;
    }
    await _bookmarksBox!.put(articleId, true);
  }
}

```

In the main.dart, StorageService.init() opens the Hive boxes.

This makes sure the boxes are ready before controllers try to read them.

```

final storageService = StorageService();
await storageService.init();

```

Static Article Data

I embedded the offline article data directly in StorageService (26 articles).

```
{
  'id': 'pw-001',
  'title': 'How to create a strong master password',
  'category': 'Passwords',
  'readTimeMinutes': 5,
  'summary': 'Build an unbreakable foundation password using passphrases',
  'content': '## What makes a master password strong\n'
    '1. Use a passphrase of 4+ random words\n'
    '2. Insert numbers between words\n'
    '3. Add a special character\n\n'
    '## Quick checklist\n'
    '☐ At least 16 characters\n'
    '☐ No dictionary words used alone\n'
    '☐ Unique - not reused anywhere\n'
    '☐ Stored in a password manager',
  'isCached': true,
},
```

Screenshots of the project later on.

Bookmark Persistence

Bookmarks are stored in the bookmarks Hive box.

The article ID is used as the key.

If the article is not bookmarked => toggleBookmark saves it

If it is already bookmarked => toggleBookmark removes it.

This means bookmarks persist across app sessions.

```
Future<void> toggleBookmark(String articleId) async {
  await init();
  final isBookmarked = _bookmarksBox!.get(articleId) == true;
  if (isBookmarked) {
    await _bookmarksBox!.delete(articleId);
    return;
  }
  await _bookmarksBox!.put(articleId, true);
}

Set<String> getBookmarkedIds() {
  return (_bookmarksBox?.keys ?? const Iterable<dynamic>.empty())
    .map((key) => key.toString())
    .toSet();
}
```

Sync Metadata

The sync_meta box stores last_sync

This is used for offline banner cache age and article reader footer

```
DateTime? getLastSyncTime() {
  final raw = _syncMetaBox?.get('last_sync')?.toString();
  if (raw == null || raw.isEmpty) return null;
  return DateTime.tryParse(raw);
}

Future<void> setLastSyncTime(DateTime dt) async {
  await init();
  await _syncMetaBox!.put('last_sync', dt.toIso8601String());
}
```

#controllers/knowledg_controller

I replace the empty KnowledgeController with a full GetX controller.

```
class KnowledgeController extends GetxController {
  final StorageService _storageService = Get.find<StorageService>();

  RxList<KnowledgeArticle> articles = <KnowledgeArticle>[].obs;
  RxList<KnowledgeArticle> filteredArticles = <KnowledgeArticle>[].obs;
  RxString selectedCategory = 'All'.obs;
  RxString searchQuery = ''.obs;
  RxBool isLoading = false.obs;
  RxSet<String> bookmarkedIds = <String>{}.obs;
  RxBool showBookmarksOnly = false.obs;
  RxBool isOffline = false.obs;
  DateTime? lastSyncTime;

  StreamSubscription<List<ConnectivityResult>>? _connectivitySubscription;

  static const categories = [
    'Passwords',
    'Network',
    'Phishing',
    'Mobile',
    'Incident',
  ];
};
```

onInit()

When the controller starts, it:

- 1- Opens storage
- 2- Loads cached articles from Hive
- 3- Seeds static articles if Hive is empty
- 4- Saves the seeded articles to Hive
- 5- Loads bookmarked article IDs
- 6- Loads last sync time
- 7- Applies filters
- 8- Start connectivity monitoring

```
void onInit() {
  super.onInit();
  _load();
  _watchConnectivity();
}

@override
void onClose() {
  _connectivitySubscription?.cancel();
  super.onClose();
}

Future<void> _load() async {
  isLoading.value = true;
  await _storageService.init();

  var cachedArticles = _storageService.getCachedArticles();
  if (cachedArticles.isEmpty) {
    cachedArticles = _storageService.getStaticArticles();
    await _storageService.saveArticles(cachedArticles);
    await _storageService.setLastSyncTime(DateTime.now());
  }

  articles.assignAll(cachedArticles);
  bookmarkedIds.assignAll(_storageService.getBookmarkedIds());
  lastSyncTime = _storageService.getLastSyncTime();
  _applyFilters();
  isLoading.value = false;
}
```

Filtering

The controller supports: category filter, search query, bookmarks-only filter

The `_applyFilters` method combines all three

This means the list can show:

- all articles
- one category
- search results
- bookmarked articles only

```
void _applyFilters() {
    Iterable<KnowledgeArticle> results = articles;

    final category = selectedCategory.value;
    if (category != 'All') {
        results = results.where((article) => article.category == category);
    }

    final query = searchQuery.value.toLowerCase();
    if (query.isNotEmpty) {
        results = results.where((article) {
            return article.title.toLowerCase().contains(query) ||
                article.summary.toLowerCase().contains(query) ||
                article.category.toLowerCase().contains(query) ||
                article.content.toLowerCase().contains(query);
        });
    }

    if (showBookmarksOnly.value) {
        results = results.where((article) => bookmarkedIds.contains(article.id));
    }

    filteredArticles.assignAll(results);
}
```

Search

Search checks article:

- title
- summary
- category
- content

This lets the user search across the whole offline library

```
void search(String query) {
    searchQuery.value = query.trim();
    _applyFilters();
}

final query = searchQuery.value.toLowerCase();
if (query.isNotEmpty) {
    results = results.where((article) {
        return article.title.toLowerCase().contains(query) ||
            article.summary.toLowerCase().contains(query) ||
            article.category.toLowerCase().contains(query) ||
            article.content.toLowerCase().contains(query);
    });
}
```

Offline State

The controller uses `connectivity_plus` to detect offline state.

If the device is offline: `isOffline` becomes true, and the `LibraryView` shows the offline

banner.

```
Future<void> _watchConnectivity() async {  
  final connectivity = Connectivity();  
  final initial = await connectivity.checkConnectivity();  
  _updateOfflineState(initial);  
  _connectivitySubscription =  
    connectivity.onConnectivityChanged.listen(_updateOfflineState);  
}  
  
void _updateOfflineState(List<ConnectivityResult> results) {  
  isOffline.value = results.contains(ConnectivityResult.none);  
}
```

#views/knowledge_base/library_view

I created the main Library screen

It contains:

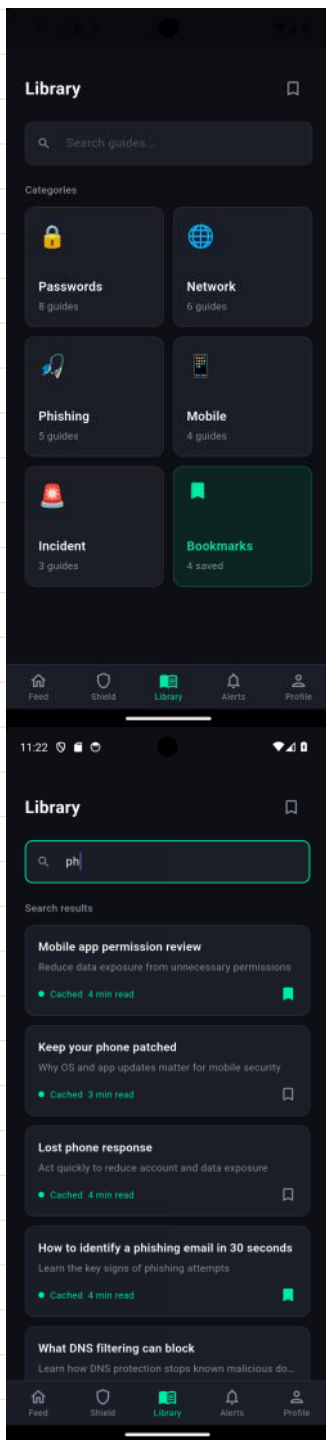
- offline banner
- header row
- bookmark toggle button
- search field
- category grid
- article list mode

Offline Banner:

The offline banner appears at the top of the screen when the device is offline:

```
return Scaffold(  
  backgroundColor: AppColors.background,  
  body: SafeArea(  
    child: ListView(  
      padding: EdgeInsets.zero,  
      children: [  
        Obx(() {  
          if (!controller.isOffline.value) return const SizedBox.shrink();  
          return _OfflineBanner(  
            text: 'Offline - ${controller.offlineCacheAge}';  
          ),  
        ]),  
    ),  
  ),  
);
```

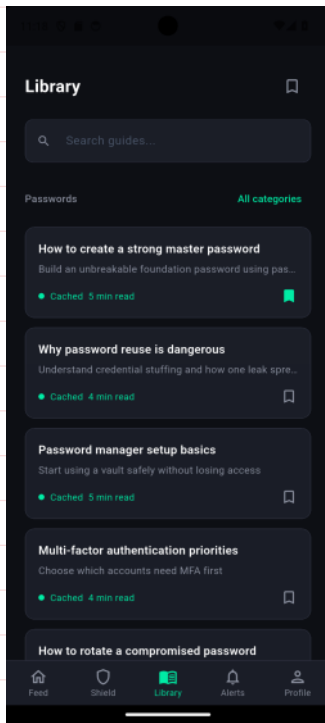
Category Grid



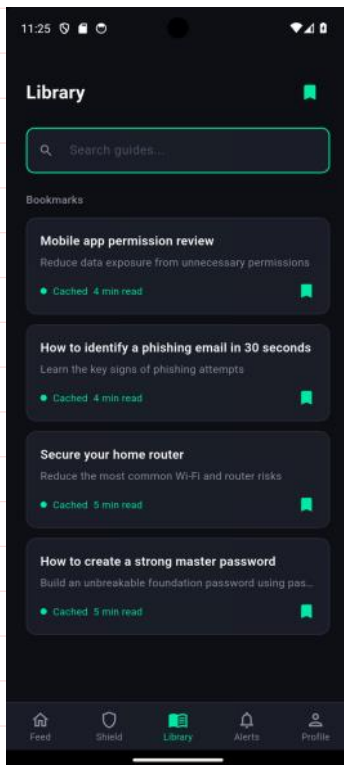
#views/knowledge_base/article_list_view

I created the Article List screen

This screen can show articles for a specific category

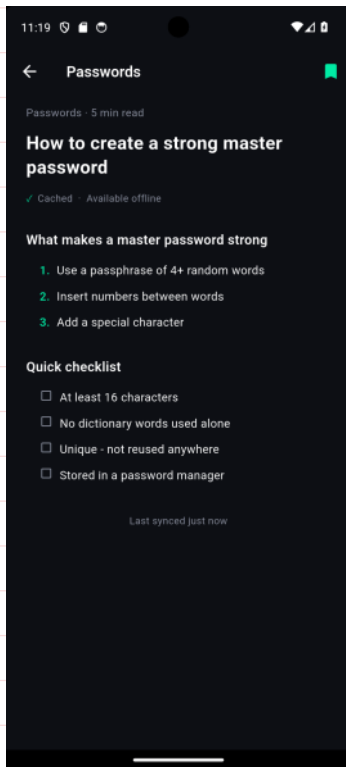


The bookmarks:



#views/knowledge_base/article_reader_view

I create the Article Reader screen.



Manual Content Parser

I did not add a markdown package.

The content is parsed manually using: `String.split('\n')`

Rendering rules:

- lines starting with `##` become section headers
- lines starting with `1.` or `2.` become numbered list items
- lines starting with square box become checklist items
- lines starting with check mark become checked checklist items
- normal lines become body text

```
class _ArticleContentRenderer {
  _ArticleContentRenderer._();

  static List<Widget> render(String content) {
    return content
      .split('\n')
      .map((line) => _renderLine(line))
      .whereType<Widget>()
      .toList();
  }

  static Widget? _renderLine(String line) {
    final trimmed = line.trim();
    if (trimmed.isEmpty) {
      return const SizedBox(height: 8);
    }

    if (trimmed.startsWith('## ')) {
      return Padding(
        padding: const EdgeInsets.only(top: 20, bottom: 8),
        child: Text(
          trimmed.substring(3),
          style: const TextStyle(
            color: AppColors.white,
            fontSize: 16,
            fontWeight: FontWeight.w600,
          ),
        ),
      );
    }
  }
}
```

```

final numberedMatch = RegExp(r'^(\d+)\s+(.*)$').firstMatch(trimmed);
if (numberedMatch != null) {
  return Padding(
    padding: const EdgeInsets.only(left: 16, top: 7, bottom: 3),
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          '${numberedMatch.group(1)}.',
          style: const TextStyle(
            color: AppColors.primary,
            fontSize: 14,
            fontWeight: FontWeight.w600,
          ),
        ),
        const SizedBox(width: 8),
        Expanded(
          child: Text(
            numberedMatch.group(2) ?? '',
            style: const TextStyle(
              color: AppColors.white,
              fontSize: 14,
              height: 1.5,
            ),
          ),
        ),
      ],
    ),
  );
}

```

Article Footer

At the bottom of the reader, I added: Last synced Xh ago
This uses the sync time stored in the controller.

#controllers/profile_controller

I created ProfileController

This controller manages the Profile screen state

```

class ProfileController extends GetxController {
  final AuthController authController = Get.find<AuthController>();
  final AuthService authService = Get.find<AuthService>();

  Rx<UserProfile> userProfile = const UserProfile(
    uid: '',
    email: '',
    displayName: 'CyberShield User',
    watchList: <String>[],
  ).obs;

  RxList<String> watchList = <String>[].obs;
  RxBool criticalAlertsEnabled = true.obs;
  RxBool quietHoursEnabled = true.obs;
  RxBool allNotificationsEnabled = true.obs;
  Rxn<DateTime> lastSyncTime = Rxn<DateTime>();
}

```

User Profile Initialization

On init, the controller reads the current Firebase user from:
AuthController.currentUser , AuthService.currentUser

It uses:

- user uuid
- user email
- user display name if available

If displayName is empty, it create a readable name from the email.

```
void _populateFromCurrentUser() {  
  final user = authController.currentUser.value ?? authService.currentUser;  
  final email = user?.email ?? '';  
  final displayName = (user?.displayName?.trim().isEmpty ?? false)  
    ? user?.displayName!.trim()  
    : _nameFromEmail(email);  
  final initialWatchList = <String>[  
    'Flutter',  
    'Firebase',  
    'Android',  
    'NVD',  
  ];  
  
  userProfile.value = UserProfile(  
    uid: user?.uid ?? '',  
    email: email,  
    displayName: displayName,  
    watchList: initialWatchList,  
    criticalAlertsEnabled: criticalAlertsEnabled.value,  
    quietHoursEnabled: quietHoursEnabled.value,  
    allNotificationsEnabled: allNotificationsEnabled.value,  
  );  
  watchList.assignAll(initialWatchList);  
  lastSyncTime.value = DateTime.now();  
}
```

Watchlist Methods

Methods added:

- 1- addTechnology
- 2- removeTechnology

1- addTechnology:

- trims the input
- ignores empty names
- prevents duplicates
- updates the UserProfile object

2- removeTechnology:

- removes the chip
- updates the UserProfile object

```
void addTechnology(String name) {  
  final value = name.trim();  
  if (value.isEmpty) return;  
  final exists = watchList.any(  
    (item) => item.toLowerCase() == value.toLowerCase(),  
  );  
  if (exists) return;  
  watchList.add(value);  
  _syncProfile();  
}  
  
void removeTechnology(String name) {  
  watchList.removeWhere(  
    (item) => item.toLowerCase() == name.toLowerCase(),  
  );  
  _syncProfile();  
}
```

Notification Methods

Methods added:

- setCriticalAlerts
- setQuietHours
- setAllNotification

These methods update the GetX observable values and sync the UserProfile.

```
void setCriticalAlerts(bool value) {
    criticalAlertsEnabled.value = value;
    _syncProfile();
}

void setQuietHours(bool value) {
    quietHoursEnabled.value = value;
    _syncProfile();
}

void setAllNotifications(bool value) {
    allNotificationsEnabled.value = value;
    _syncProfile();
}
```

Sync Method

I added syncNow().

For this deliverable, it is a placeholder

It sets: lastSyncTime = DateTime.now()

Later deliverables can connect this to real sync behavior.

```
Future<void> syncNow() async {
    lastSyncTime.value = DateTime.now();
}
```

Display Initials

It creates initials from:

- displayName if available
- email local part if displayName is missing

```
String get displayInitials {
    final displayName = userProfile.value.displayName.trim();
    final email = userProfile.value.email.trim();
    final source =
        displayName.isNotEmpty ? displayName : email.split('@').first;
    final parts = source
        .split(RegExp(r'\s+'))
        .where((part) => part.trim().isNotEmpty)
        .toList();

    if (parts.isEmpty) return 'CS';
    if (parts.length == 1) {
        final cleaned = parts.first.replaceAll(RegExp(r'^A-Za-z0-9'), '');
        if (cleaned.isEmpty) return 'CS';
        return cleaned.substring(0, 1).toUpperCase();
    }

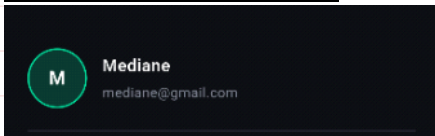
    return '${parts.first[0]}${parts.last[0]}'.toUpperCase();
}
```

```
String _nameFromEmail(String email) {
  if (email.isEmpty) return 'CyberShield User';
  final localPart = email.split('@').first;
  final words = localPart
    .split(RegExp(r'[-.]'))
    .where((word) => word.isNotEmpty)
    .map((word) => '${word[0].toUpperCase()}${word.substring(1)}');
  return words.isEmpty ? 'CyberShield User' : words.join(' ');
}
```

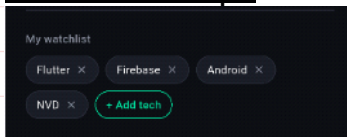
#views/profile/profile_view

I replaced the profile stub with the full Profile screen.

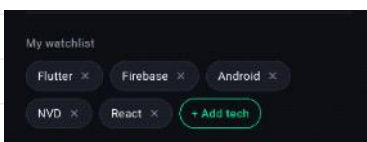
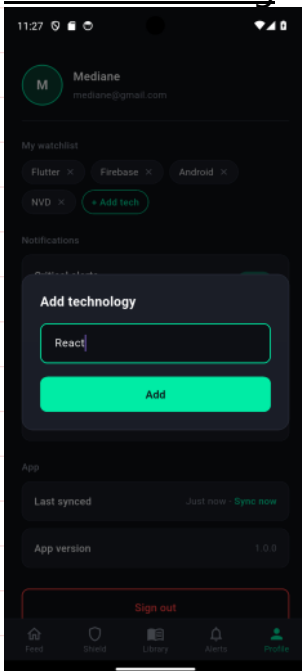
Avatar and User Info



Watchlist Chips



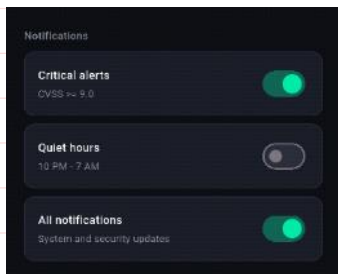
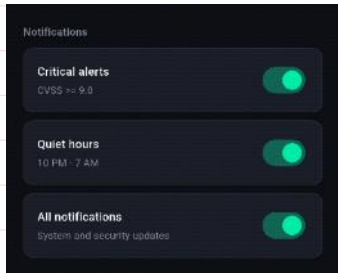
Add Tech Dialog



Notification Toggle Cards

I added three notification cards:

- Critical alertssssss
- Quiet hours
- All notifications



App Info Cards

I added two app info cards:

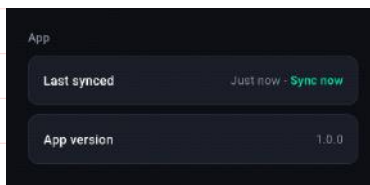
- Last synced
- App version

Last synced row:

- shows time ago
- includes tappable Sync now text

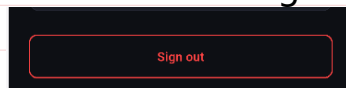
App version row:

- shows 1.0.0

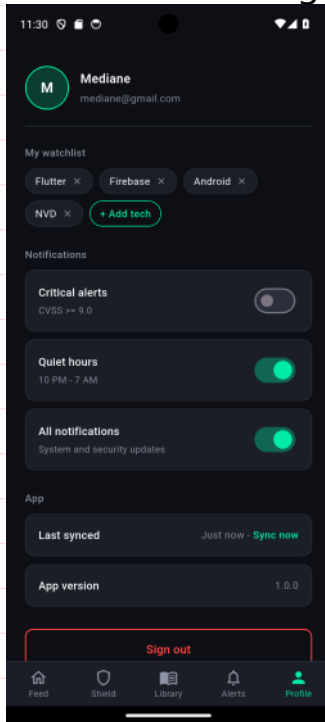


Sign Out Dialog

It confirms the sign out.



The Full Profile Page:



#views/alerts/alerts_view

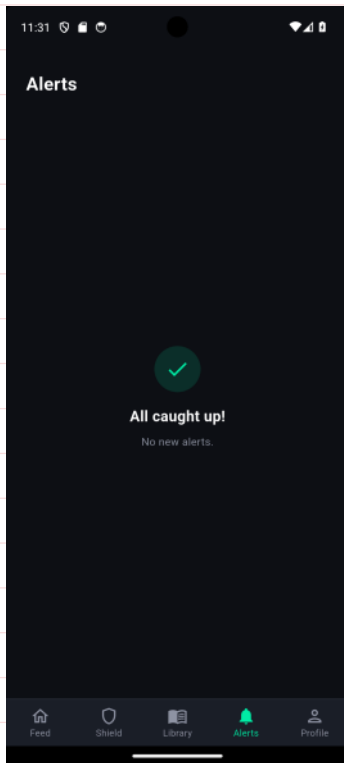
I created the Alerts screen

This is currently an empty state screen

It shows: alerts title, All caught up!, No new alerts.

The unread alert count is currently hardcoded to 0 in the shell.

The real unread count will be added in a later deliverable.



#views/shell/main_shell_view

I upgraded MainShellView from the old tab setup to the final five-tab shell

Tabs in order:

- 1- Feed
- 2- Shield
- 3- Library
- 4- Alerts
- 5- Profile

The shell still uses IndexedStack. This preserves tab state when switching between tabs.

#routes/app_routes

I added the new route constants.

```

abstract class AppRoutes {
    static const INITIAL = '/';
    static const HOME = '/home';
    static const SHELL = '/shell';
    static const LOGIN = '/login';
    static const REGISTER = '/register';
    static const FORGOT_PASSWORD = '/forgot-password';
    static const ONBOARDING = '/onboarding';
    static const THREAT_DETAIL = '/threat-detail';
    static const PASSWORD_CHECKER = '/password-checker';
    static const LIBRARY = '/library';
    static const LIBRARY_CATEGORY = '/library/category';
    static const LIBRARY_ARTICLE = '/library/article';
    static const ALERTS = '/alerts';
    static const PROFILE = '/profile';
}

```

#routes/app_pages

I updated app_pages

```

// -- Feature routes --
GetPage(
  name: AppRoutes.HOME,
  page: () => const HomeView(),
  bindings: [
    AuthBinding(),
    HomeBinding(),
  ],
),
GetPage(
  name: AppRoutes.THREAT_DETAIL,
  page: () => const ThreatDetailView(),
  binding: ThreatBinding(),
),
GetPage(
  name: AppRoutes.PASSWORD_CHECKER,
  page: () => const PasswordCheckerView(),
  binding: PasswordBinding(),
),
GetPage(
  name: AppRoutes.LIBRARY,
  page: () => const LibraryView(),
  binding: KnowledgeBinding(),
),
GetPage(
  name: AppRoutes.LIBRARY_CATEGORY,
  page: () => const ArticleListView(),
  binding: KnowledgeBinding(),
),
GetPage(
  name: AppRoutes.LIBRARY_ARTICLE,
  page: () => const ArticleReaderView(),
  binding: KnowledgeBinding(),
),
GetPage(
  name: AppRoutes.ALERTS,
  page: () => const AlertsView(),
),
GetPage(
  name: AppRoutes.PROFILE,
  page: () => const ProfileView(),
  binding: ProfileBinding(),
),

```

#bindings/knowledge_binding

I created KnowledgeBinding.

It registers: StorageService, KnowledgeController

```

class KnowledgeBinding extends Bindings {
  @override
  void dependencies() {
    if (!Get.isRegistered<StorageService>()) {
      Get.lazyPut<StorageService>(() => StorageService(), fenix: true);
    }
    if (!Get.isRegistered<KnowledgeController>()) {
      Get.lazyPut<KnowledgeController>(() => KnowledgeController(),
        fenix: true);
    }
  }
}

```

#bindings/profile_binding

I replaced the empty ProfileBinding

It now registers: AuthService, AuthController, ProfileController

It checks whether AuthService and AuthController are already registered before creating them.

This prevents duplicate auth instances

```

class ProfileBinding extends Bindings {
  @override
  void dependencies() {
    if (!Get.isRegistered<AuthService>()) {
      Get.put(AuthService());
    }
    if (!Get.isRegistered<AuthController>()) {
      Get.put(AuthController());
    }
    if (!Get.isRegistered<ProfileController>()) {
      Get.put(ProfileController());
    }
  }
}

```

#bindings/shell_bindings

```

class ShellBinding extends Bindings {
  @override
  void dependencies() {
    if (!Get.isRegistered<AuthService>()) {
      Get.put(AuthService());
    }
    if (!Get.isRegistered<AuthController>()) {
      Get.put(AuthController());
    }
    if (!Get.isRegistered<ApiService>()) {
      Get.put(ApiService());
    }
    if (!Get.isRegistered<StorageService>()) {
      Get.put(StorageService());
    }
    if (!Get.isRegistered<ThreatFeedController>()) {
      Get.put(ThreatFeedController());
    }
    if (!Get.isRegistered<PasswordController>()) {
      Get.put>PasswordController();
    }
    if (!Get.isRegistered<KnowledgeController>()) {
      Get.put(KnowledgeController());
    }
    if (!Get.isRegistered<ProfileController>()) {
      Get.put(ProfileController());
    }
  }
}

```

#main.dart

I updated the main.dart to initialize StorageService

The app now initializes in this order:

- 1- WidgetsFlutterBinding.ensureInitialized()
- 2- Hive.initFlutter()
- 3- StorageService()
- 4- StorageService.init()
- 5- Get.put(storageService)
- 6- Firebase.initializeApp()
- 7- runApp()

This is important because the KnowledgeController reads from Hive.

The Hive boxes must already be open before the controller tries to access articles, bookmarks, or sync metadata.

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  final storageService = StorageService();
  await storageService.init();
  Get.put<StorageService>(storageService);
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const CyberShieldApp());
}

class CyberShieldApp extends StatelessWidget {
  const CyberShieldApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'CyberShield',
      debugShowCheckedModeBanner: false,
      initialRoute: AppRoutes.LOGIN,
      getPages: AppPages.routes,
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
    );
  }
}
```

The deliverable 6 (Core Features) is successfully implemented.

Offline Capability

In this step I will implement Deliverable 7: Offline Capability for the CyberShield Flutter mobile app.

The goal of this step is to make the most important app features keep working when the device loses internet access.

Deliverable 7 focuses on:

- Threat Feed offline caching
- Stale-while-revalidate behavior
- Visible offline and cached-data indicators
- Password breach check graceful offline message
- Profile preference persistence
- Startup initialization for all Hive boxes

The app already had:

- Password Shield screen
- Offline Security Knowledge Base
- Persistent article bookmarks
- Profile screen
- StorageService initialized before runApp()
- connectivity_plus already installed

#services/storage_service

I expanded StorageService so it owns all Hive storage used by offline features.

Before, StorageService opened: knowledge_articles, bookmarks, sync_meta

Now it also opens: threat_cache, user_profile

I added also box constant: threatCacheBox, profileBox

The StorageService.init() opens five boxes.

The method uses null-check assignment: ??=

This keeps initialization safe if init() is called more than once.

```

class StorageService {
    static const knowledgeArticlesBox = 'knowledge_articles';
    static const bookmarksBox = 'bookmarks';
    static const syncMetaBox = 'sync_meta';
    static const threatCacheBox = 'threat_cache';
    static const profileBox = 'user_profile';

    Box<dynamic>? _articlesBox;
    Box<dynamic>? _bookmarksBox;
    Box<dynamic>? _syncMetaBox;
    Box<dynamic>? _threatCacheBox;
    Box<dynamic>? _profileBox;

    Future<void> init() async {
        _articlesBox ??= await Hive.openBox<dynamic>(knowledgeArticlesBox);
        _bookmarksBox ??= await Hive.openBox<dynamic>(bookmarksBox);
        _syncMetaBox ??= await Hive.openBox<dynamic>(syncMetaBox);
        _threatCacheBox ??= await Hive.openBox<dynamic>(threatCacheBox);
        _profileBox ??= await Hive.openBox<dynamic>(profileBox);
    }
}

```

Threat Cache Methods

I added threat-cache methods:

- saveThreatPage()
- getCacheThreatPage()
- clearThreatCache()

- saveThreatPage() stores a serialized list of threat maps under a cache key.
- getCacheThreatPage() reads a cached threat page and converts raw Hive maps back into: Map<String, dynamic>
- clearThreatCache() clears all cached threat pages.

No Hive TypeAdapters were added, no build_runner code was added.

The cache uses the same map-based storage style as the knowledge base.

```

Future<void> saveThreatPage({
    required String cacheKey,
    required List<Map<String, dynamic>> serialized,
}) async {
    await init();
    await _threatCacheBox!.put(cacheKey, serialized);
}

List<Map<String, dynamic>> getCacheThreatPage(String cacheKey) {
    final exact = _readThreatPage(cacheKey);
    if (exact.isNotEmpty) return exact;

    final matchingKey = _newestMatchingThreatKey(cacheKey);
    if (matchingKey == null) return <Map<String, dynamic>>[];
    return _readThreatPage(matchingKey);
}

Future<void> clearThreatCache() async {
    await init();
    await _threatCacheBox!.clear();
}

```

Rolling Cache Key Fallback

The NVD cache key includes:

- page
- keyword
- severity
- pubStart
- pubEnd

Because pubStart and pubEnd are based on the current recent window, the exact timestamp can change between app launches.

To make cached startup more reliable, getCachedThreatPage() first checks for an exact cache key.

If the exact key is not found, it looks for the newest cached key with the same: page, keyword, severity

This keeps the existing cache key format unchanged while still allowing the app to reuse the most recent matching cached data.

```
String? _newestMatchingThreatKey(String cacheKey) {  
    final prefix = _threatFilterPrefix(cacheKey);  
    if (prefix == null) return null;  
  
    final keys = _threatCacheBox?.keys ?? const Iterable<dynamic>.empty();  
    final matches = keys  
        .map((key) => key.toString())  
        .where((key) => key.startsWith(prefix))  
        .toList()  
        ..sort();  
  
    if (matches.isEmpty) return null;  
    return matches.last;  
}  
  
String? _threatFilterPrefix(String cacheKey) {  
    final markerIndex = cacheKey.indexOf('|pubStart=');  
    if (markerIndex <= 0) return null;  
    return '${cacheKey.substring(0, markerIndex)}|';  
}
```

Profile Persistence

I added profile persistence methods:

- saveProfile() stores the user profile JSON map in the user_profile Hive box
- loadProfile() reads the stored map and returns: Map<String, dynamic>?

This is used by ProfileController to restore watchlist chips and notification toggles after app restart.

```
Future<void> saveProfile(Map<String, dynamic> profileJson) async {  
    await init();  
    await _profileBox!.put('profile', profileJson);  
}  
  
Map<String, dynamic>? loadProfile() {  
    final raw = _profileBox?.get('profile');  
    if (raw is! Map) return null;  
    return Map<String, dynamic>.from(raw);  
}
```

#services/api_service

I updated ApiService so it no longer opens the threat cache Hive box directly

Before:

- ApiService._cacheThreats() lazily called Hive.openBox()

Now:

- StorageService opens the threat_cache box at startup

- ApiService uses Get.find()
- ApiService calls StorageService.saveThreatPage()

This avoids duplicate Hive.openBox calls

```
try {
    final key = _cacheKey(
        page: page,
        keyword: keyword,
        severity: severity,
        publishStartDate: publishStartDate,
        publishEndDate: publishEndDate,
    );
    await Get.find<StorageService>().saveThreatPage(
        cacheKey: key,
        serialized: threats.map((threat) => threat.toCacheJson()).toList(),
    );
} catch (_) {
    // Cache writes should never block the live API experience.
}
```

getCachedThreats()

I added: getCachedThreats(String cacheKey)

This method

- 1- Reads cached threat maps from StorageService
- 2- Converts each map back into a ThreatAdvisory
- 3- Uses ThreatAdvisory.fromCache()

I reused the existing ThreatAdvisory.fromCache() factory. I did not create a new deserialization factory.

```
List<ThreatAdvisory> getCachedThreats(String cacheKey) {
    final cached = Get.find<StorageService>().getCachedThreatPage(cacheKey);
    return cached.map(ThreatAdvisory.fromCache).toList();
}
```

buildThreatCacheKey()

I added a public helper: buildThreatCacheKey().

This lets ThreatFeedController build the same page 0 cache key that ApiService uses internally.

It uses the existing private _cacheKey() helper.

The _cacheKey() helper itself was kept unchanged.

The key format is still: page=...|keyword=...|severity=...|pubStart=...|pubEnd=...

```
String buildThreatCacheKey({
    required int page,
    String? keyword,
    String? severity,
}) {
    final publishWindow = _recentPublishWindow();
    final trimmedKeyword = keyword?.trim() ?? '';
    final normalizedSeverity = severity?.trim().toUpperCase() ?? '';

    return _cacheKey(
        page: page,
        keyword: trimmedKeyword,
        severity: normalizedSeverity,
        publishStartDate: _formatNvdDate(publishWindow.start),
        publishEndDate: _formatNvdDate(publishWindow.end),
    );
}
```

#controllers/threat_feed_controller

I updated ThreatFeedController to support offline state and cached threat results.

I added reactive state: isOffline, isShowingCachedData

I added connectivity subscription: StreamSubscription<List>

```
RxBool isOffline = false.obs;  
RxBool isShowingCachedData = false.obs;  
  
StreamSubscription<List<ConnectivityResult>>? _connectivitySubscription;
```

Connectivity Monitoring

The controller now checks connectivity on startup.

It uses: Connectivity().checkConnectivity()

Then it listens to: connectivity.onConnectivityChanged

Offline is detected with: results.contains(ConnectivityResult.none)

This matches connectivity_plus version 6.x, where the result is a list.

```
Future<void> _watchConnectivity() async {  
  final connectivity = Connectivity();  
  final initial = await connectivity.checkConnectivity();  
  _updateOfflineState(initial);  
  _connectivitySubscription =  
    connectivity.onConnectivityChanged.listen(_updateOfflineState);  
}  
  
void _updateOfflineState(List<ConnectivityResult> results) {  
  isOffline.value = results.contains(ConnectivityResult.none);  
}
```

onClose()

I added onClose(). It cancels the connectivity subscription.

This prevents subscription leaks when the controller is disposed.

```
@override  
void onClose() {  
  _connectivitySubscription?.cancel();  
  super.onClose();  
}
```

Stale-While-Revalidate Flow

fetchThreats() now follows a stale-while-revalidate pattern

Flow:

- 1- Build the page 0 cache key for the current filters
- 2- Read cached threats from ApiService.getCachedThreats()
- 3- If offline and cache exists, show cached threats

- 4- If offline and cache is empty, show the offline empty state
- 5- If online, fetch fresh threats from NVD
- 6- If fresh fetch succeeds, show fresh data and clear cached-data state
- 7- If fresh fetch fails and cache exists, show cached threats
- 8- If fresh fetch fails and cache is empty, show the normal error message

```
void _showCachedThreats(List<ThreatAdvisory> cachedThreats) {  
    threats.assignAll(_newestFirst(cachedThreats));  
    currentPage.value = 0;  
    hasMorePages.value = false;  
    errorMessage.value = '';  
    isShowingCachedData.value = true;  
}
```

Cached Data State

When cached threats are shown:

- Threats is populated from cache
- errorMessage is cleared
- hasMorePages is set to false
- isShowingCachedData is set to true

When fresh data loads successfully:

- isShowingCachedData is set to false

```
void _showCachedThreats(List<ThreatAdvisory> cachedThreats) {  
    threats.assignAll(_newestFirst(cachedThreats));  
    currentPage.value = 0;  
    hasMorePages.value = false;  
    errorMessage.value = '';  
    isShowingCachedData.value = true;  
}
```

Load Cached Filtering

The controller can filter cached base threat data locally.

This helps severity filter and keyword search still work against cached results when the device is offline.

Local filtering checks: severity, CVE ID, description, severity text

```
List<ThreatAdvisory> _applyLocalFilters(List<ThreatAdvisory> cachedThreats) {  
    Iterable<ThreatAdvisory> results = cachedThreats;  
  
    final severity = selectedSeverity.value.trim().toUpperCase();  
    if (severity.isNotEmpty) {  
        results = results.where(  
            (threat) => threat.severity.toUpperCase() == severity,  
        );  
    }  
  
    final keyword = searchKeyword.value.trim().toLowerCase();  
    if (keyword.isNotEmpty) {  
        results = results.where((threat) {  
            return threat.id.toLowerCase().contains(keyword) ||  
                threat.description.toLowerCase().contains(keyword) ||  
                threat.severity.toLowerCase().contains(keyword);  
        });  
    }  
  
    return results.toList();  
}
```

Load More Protection

Infinite scrolling is disabled while:

- offline
- showing cached data

This avoids trying to fetch additional pages from the network when the feed is currently using stale cached results.

```
Future<void> loadMoreThreats() async {
  errorMessage.value = '';

  if (isLoading.value || isLoadingMore.value || !hasMorePages.value) {
    return;
  }

  if (isOffline.value || isShowingCachedData.value) {
    return;
  }

  isLoadingMore.value = true;

  try {
    final nextPage = currentPage.value + 1;
    final results = await _apiService.fetchThreats(
      page: nextPage,
      keyword: searchKeyword.value,
      severity: selectedSeverity.value,
    );

    if (_apiService.errorMessage.isNotEmpty) {
      errorMessage.value = _apiService.errorMessage;
      return;
    }

    threats.addAll(results);
    threats.assignAll(_newestFirst(threats));
    currentPage.value = nextPage;
    hasMorePages.value = _apiService.hasMoreThreatPages;
  } catch (_) {
    errorMessage.value = 'Could not load more threats. Please try again.';
  } finally {
    isLoadingMore.value = false;
  }
}
```

#views/home/threat_feed_view

I updated the Threat Feed screen to show visible offline and cached-data states.

I kept existing widgets the same.

I added new widgets instead of removing the existing ones.

Cached Data Banner

I added: `_CachedDataBanner`

The banner appears above the severity filter chips when:

- `isOffline` is true
- or `isShowingCachedData` is true

Offline message:

Offline - showing cached results

Network failure message: Network error - showing cached results

```
class _CachedDataBanner extends StatelessWidget {  
  final String text;  
  
  const _CachedDataBanner({required this.text});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      width: double.infinity,  
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),  
      decoration: BoxDecoration(  
        color: AppColors.warning.withValues(alpha: 0.1),  
        borderRadius: BorderRadius.circular(10),  
        border: Border.all(color: AppColors.warning),  
      ),  
      child: Row(  
        children: [  
          Container(  
            width: 8,  
            height: 8,  
            decoration: const BoxDecoration(  
              color: AppColors.warning,  
              shape: BoxShape.circle,  
            ),  
          ),  
          const SizedBox(width: 8),  
          Expanded(  
            child: Text(  

```

```
Obx(() {  
  if (!controller.isOffline.value &&  
      !controller.isShowingCachedData.value) {  
    return const SizedBox.shrink();  
  }  
  
  final message = controller.isOffline.value  
    ? 'Offline · showing cached results'  
    : 'Network error · showing cached results';  
  return Padding(  
    padding: const EdgeInsets.only(bottom: 14),  
    child: _CachedDataBanner(text: message),  
  );  
}),
```

Offline Empty State

I added: `_OfflineEmptyState`

This appears when:

- threats is empty
- device is offline

It replaces the generic empty state only for offline empty-cache cases

```
class _OfflineEmptyState extends StatelessWidget {  
  const _OfflineEmptyState();  
  
  @override  
  Widget build(BuildContext context) {  
    return ListView(  
      physics: const AlwaysScrollableScrollPhysics(),  
      children: const [  
        SizedBox(height: 120),  
        Icon(  
          Icons.wifi_off_outlined,  
          color: AppColors.textMuted,  
          size: 52,  
        ),  
        SizedBox(height: 16),  
        Center(  
          child: Text(  
            "You're offline",  
            style: TextStyle(  
              color: AppColors.white,  
              fontSize: 18,  
              fontWeight: FontWeight.w700,  
            ),  
          ),  
        ),  
      ],  
    ),  
  ),  
);
```

I updated ProfileController so profile preferences persist to Hive

The controller still populates the profile from the current Firebase user first

This preserves the correct sign-in user info while restoring local app preferences.

```
class ProfileController extends GetxController {  
  final AuthController authController = Get.find<AuthController>();  
}
```

This method:

1- Calls `StorageService.loadProfile()`

- 2- If saved data exists, converts it with `UserProfile.fromJson()`
- 3- Restores watchlist chips
- 4- Restore critical alert toggle
- 5- Restore quiet hours toggle
- 6- Restore all notifications toggle
- 7- Updates the current `UserProfile` value

```

void _loadPersistedProfile() {
    final persisted = _storageService.loadProfile();
    if (persisted == null) return;

    final savedProfile = UserProfile.fromJson(persisted);
    final current = userProfile.value;
    criticalAlertsEnabled.value = savedProfile.criticalAlertsEnabled;
    quietHoursEnabled.value = savedProfile.quietHoursEnabled;
    allNotificationsEnabled.value = savedProfile.allNotificationsEnabled;
    watchlist.assignAll(savedProfile.watchlist);
    userProfile.value = UserProfile(
        uid: current.uid.isNotEmpty ? current.uid : savedProfile.uid,
        email: current.email.isNotEmpty ? current.email : savedProfile.email,
        displayName: current.displayName.isNotEmpty
            ? current.displayName
            : savedProfile.displayName,
        watchlist: savedProfile.watchlist,
        criticalAlertsEnabled: savedProfile.criticalAlertsEnabled,
        quietHoursEnabled: savedProfile.quietHoursEnabled,
        allNotificationsEnabled: savedProfile.allNotificationsEnabled,
    );
}

```

saveProfile

I added: `_saveProfile()`.

This method save:

`userProfile.value.toJson()` to `StorageService.saveProfile()`

```

void _saveProfile() {
    _storageService.saveProfile(userProfile.value.toJson());
}

```

Persisted Watchlist

I updated: `addTechnology()` and `removeTechnology()`

After each watchlist change:

- 1- `_syncProfile()` updates the `UserProfile` object
- 2- `_saveProfile()` write the profile to Hive

Result: Watchlist chips survive app restart

```

void addTechnology(String name) {
    final value = name.trim();
    if (value.isEmpty) return;
    final exists = watchlist.any(
        (item) => item.toLowerCase() == value.toLowerCase(),
    );
    if (exists) return;
    watchlist.add(value);
    _syncProfile();
    _saveProfile();
}

void removeTechnology(String name) {
    watchlist.removeWhere(
        (item) => item.toLowerCase() == name.toLowerCase(),
    );
    _syncProfile();
    _saveProfile();
}

```

Persisted Notification Toggles

I updated:

- `setCriticalAlerts()`
- `setQuietHours()`

- setAllNotifications()

After each toggle change:

- 1- _syncProfile() updates the UserProfile object
- 2- _saveProfile() write the profile to Hive

Result: Notification preferences survive app restart

```
void setCriticalAlerts(bool value) {
    criticalAlertsEnabled.value = value;
    _syncProfile();
    _saveProfile();
}

void setQuietHours(bool value) {
    quietHoursEnabled.value = value;
    _syncProfile();
    _saveProfile();
}

void setAllNotifications(bool value) {
    allNotificationsEnabled.value = value;
    _syncProfile();
    _saveProfile();
}
```

#controllers/password_controller

I updated PasswordController to detect offline breach checks.

Added import: connectivity_plus

Added offline message:

No internet connection. Breach check requires network access. Your local strength analysis is still accurate.

```
class PasswordController extends GetxController {
    final ApiService _apiService = Get.find<ApiService>();
    static const _offlineBreachMessage =
        'No internet connection. Breach check requires network access. Your local strength analysis is still accurate.';
}
```

Offline Check Before API Call

Before calling ApiService.checkPassword(), checkBreach() now checks:

Connectivity().checkConnectivity()

If the result contains ConnectivityResult.none:

- errorMessage is set to the offline breach message
- isChecking is set to false
- the method returns early

This prevents an unnecessary network call while offline.

```

Future<void> checkBreach() async {
    final currentPassword = password.value;
    if (currentPassword.isEmpty || isChecking.value) {
        return;
    }

    isChecking.value = true;
    errorMessage.value = '';
    breachResult.value = null;
    isBreachChecked.value = false;

    try {
        final connectivityResult = await Connectivity().checkConnectivity();
        if (connectivityResult.contains(ConnectivityResult.none)) {
            errorMessage.value = _offlineBreachMessage;
            isChecking.value = false;
            return;
        }

        final checkResult = await _apiService.checkPassword(currentPassword);
        if (_apiService.errorMessage.isNotEmpty) {
            errorMessage.value = _isNetworkError(_apiService.errorMessage)
                ? _offlineBreachMessage
                : _apiService.errorMessage;
            return;
        }
        breachResult.value = checkResult;
        isBreachChecked.value = true;
    } catch (_) {
        final connectivityResult = await Connectivity().checkConnectivity();
        errorMessage.value = connectivityResult.contains(ConnectivityResult.none)
            ? _offlineBreachMessage
            : 'Could not connect to breach database. Please try again.';
    } finally {
        isChecking.value = false;
    }
}

```

#views/password/password_checker_view

No structural layout changes were required for this file.

The existing error banner already read: controller.errorMessage

Because the controller now sets a better offline message, the screen shows the correct message automatically.

Local password strength analysis is unaffected

The user can still type a password and see:

- strength level
- entropy
- length
- character checks
- dictionary detection
- improvement tip

Only the online HIBP breach check requires network access.

!!!!!!! hot sura

#utils/constants

I updated AppHiveBoxes

I added: profileCache = user_profile

```
class AppHiveBoxes {  
  AppHiveBoxes._();  
  
  static const threatCache = 'threat_cache';  
  static const profileCache = 'user_profile';  
}
```

#main.dart

No structural changes were needed in main

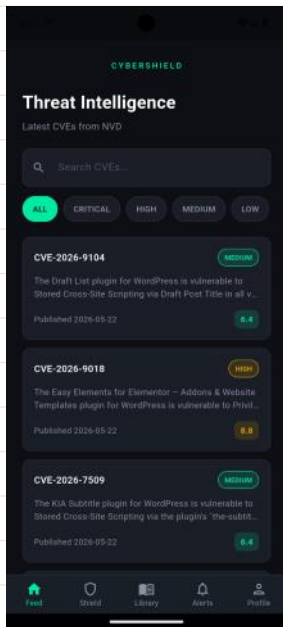
The initialization order stayed the same.

Because StorageService.init() now opens the new boxes, main.dart still opens all offline storage before runApp().

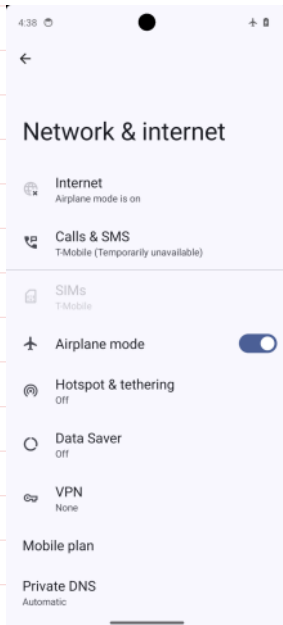
```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Hive.initFlutter();  
  final storageService = StorageService();  
  await storageService.init();  
  Get.put<StorageService>(storageService);  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  runApp(const CyberShieldApp());  
}  
  
class CyberShieldApp extends StatelessWidget {  
  const CyberShieldApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'CyberShield',  
      debugShowCheckedModeBanner: false,  
      initialRoute: AppRoutes.LOGIN,  
      getPages: AppPages.routes,  
      theme: ThemeData(  
        primarySwatch: Colors.blue,  
      ),  
    );  
  }  
}
```

Screenshots:

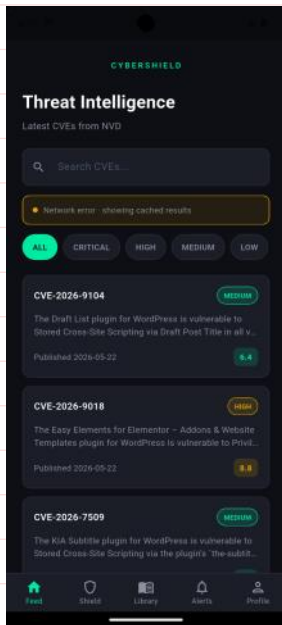
Feed page



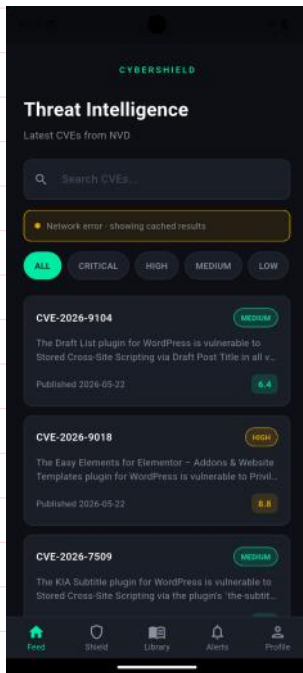
Turn internet off.

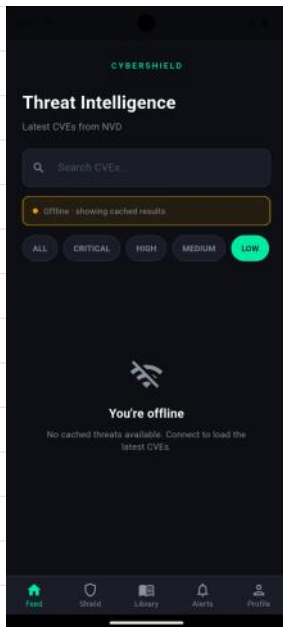


Offline Feed Page:

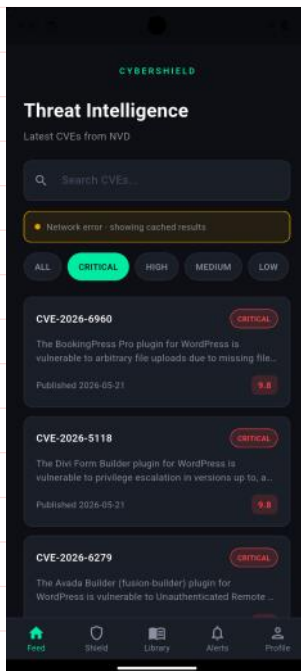


Offline Feed Refresh:

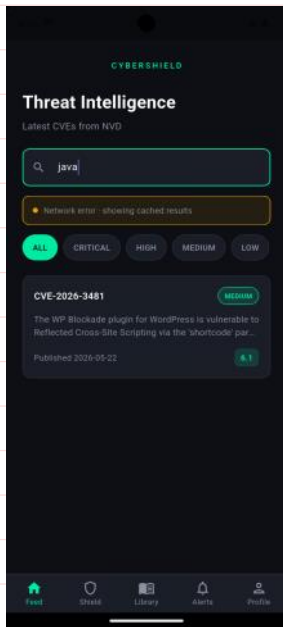




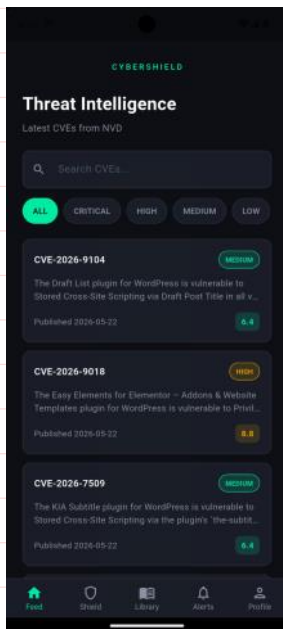
Offline Feed Filter by severity:



Offline Feed Search:

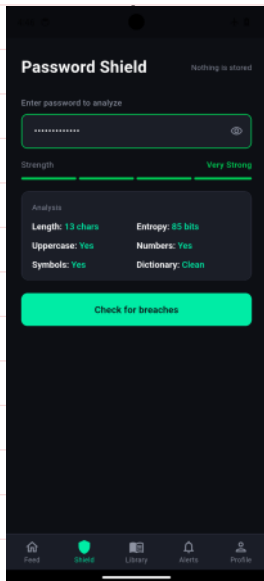


Turn internet back on:

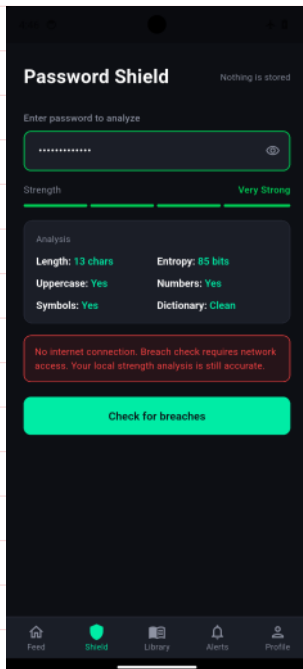


Password Shield

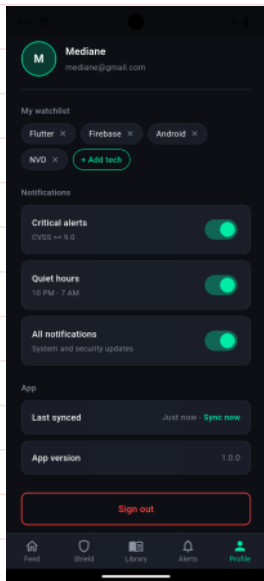
Turn internet off.
Enter a password:



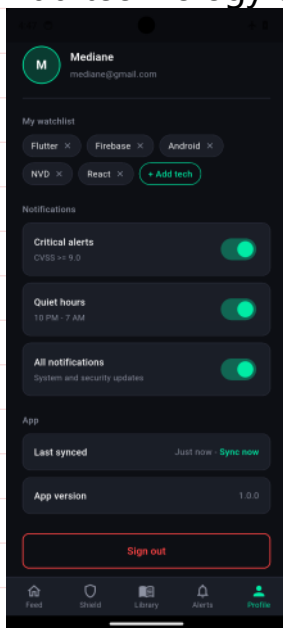
Check for breaches:



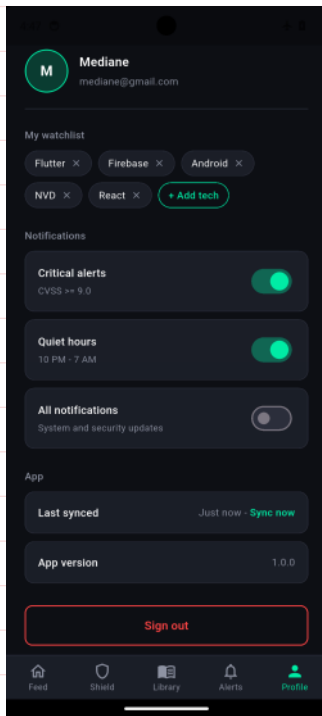
Profile



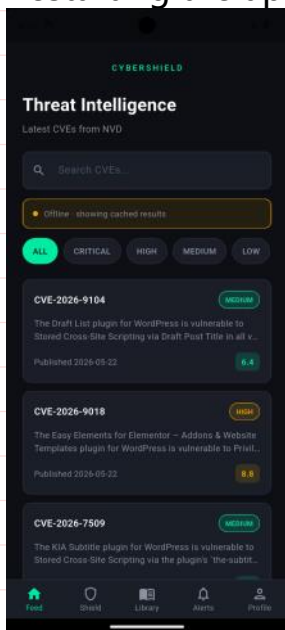
Add technology chip:



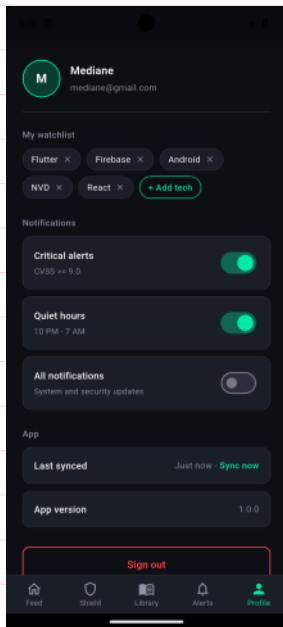
Toggle notification settings:



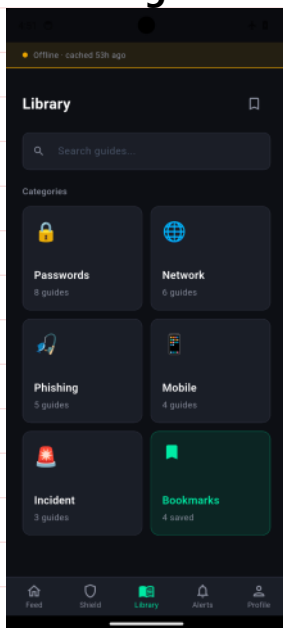
Restarting the app



Chip and toggles are restored:



Knowledge Base:



Push Notifications

In this step I will implement Deliverable 8: Add Push Notifications for the CyberShield Flutter mobile app.

The goal of this step is to connect the app to Firebase Cloud Messaging and show real security alerts inside the app.

Deliverable 8 focuses on:

- Firebase Cloud Messaging setup
- Android notification permission
- Foreground notification handling
- Background notification tap handling
- Terminated app notification tap handling
- In-app alert history
- Persistent notification history with Hive
- Alert read and unread state
- Alert severity styling
- Alert filtering
- Watchlist topic subscriptions
- Alerts tab unread badge

Push Notification Overview

Deliverable 8 adds real push alert behavior to CyberShield.

The app at the end should receive Firebase notifications and store them locally.

Main improvements:

- ♦ The app subscribes to the main cybershield_alerts topic
- ♦ Incoming alerts are saved to Hive
- ♦ Alerts appear in the Alerts tab
- ♦ Alerts keep their severity, CVE ID, title, body, and time
- ♦ The bottom navigation shows a red unread dot
- ♦ Alerts can be filtered by severity
- ♦ Alerts can be marked all read
- ♦ Alerts can be cleared
- ♦ Tapping an alert with a CVE ID opens the threat detail page
- ♦ Tapping a notification opens the Alerts tab

#pubspec.yaml

The project includes: firebase_messaging. It is the package used to receive Firebase Cloud Messaging notifications in Flutter.

```
hive_flutter: ^1.1.0
firebase_core: ^3.6.0
firebase_auth: ^5.3.0
firebase_messaging: ^15.0.0
flutter_secure_storage: ^9.0.0
```

#android/app/src/main/AndroidManifest.xml

I added Android notification permission: android.permission.POST_NOTIFICATION

The manifest also already includes: android.permission.INTERNET

Without this permission, notifications may not appear even if Firebase receives the message correctly.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

#models/app_notification

I created the AppNotification model

This model represents one received push notification saved in local history

```
class AppNotification {
  final String id;
  final String title;
  final String body;
  final String? severity;
  final String? cveId;
  final DateTime timestamp;
  final bool isRead;

  const AppNotification({
    required this.id,
    required this.title,
    required this.body,
    this.severity,
    this.cveId,
    required this.timestamp,
    this.isRead = false,
  });

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'title': title,
      'body': body,
      'severity': severity,
      'cveId': cveId,
      'timestamp': timestamp.toIso8601String(),
      'isRead': isRead,
    };
  }

  AppNotification copyWith({bool? isRead}) {
    return AppNotification(
      id: id,
      title: title,
      body: body,
      severity: severity,
      cveId: cveId,
      timestamp: timestamp,
      isRead: isRead ?? this.isRead,
    );
  }
}
```

Flexible Payload Parsing:

I made AppNotification support multiple Firebase data key names.

```
factory AppNotification.fromJson(Map<String, dynamic> json) {  
  final cveId = _firstJsonValue(json, const [  
    'cveId',  
    'cveID',  
    'cve_id',  
    'cve',  
  ]);  
  
  return AppNotification(  
    id: json['id']?.toString() ?? '',  
    title: _firstJsonValue(json, const [  
      'title',  
      'notificationTitle',  
      'notification_title',  
      'alertTitle',  
      'alert_title',  
    ]) ??  
    'CyberShield Alert',  
    body: _firstJsonValue(json, const [  
      'body',  
      'message',  
      'description',  
      'summary',  
      'notificationBody',  
      'notification_body',  
      'alertBody',  
      'alert_body',  
    ]) ??  
    ''  
  );  
}
```

This is useful because Firebase Console custom data can use slightly different key names

#services/storage_service

I expanded StorageService with a new Hive box for notification history.

New box: notification_history.

The box is opened inside: StorageService.init

```
class StorageService {  
  static const knowledgeArticlesBox = 'knowledge_articles';  
  static const bookmarksBox = 'bookmarks';  
  static const syncMetaBox = 'sync_meta';  
  static const threatCacheBox = 'threat_cache';  
  static const profileBox = 'user_profile';  
  static const notificationHistoryBox = 'notification_history';  
  
  Box<dynamic>? _articlesBox;  
  Box<dynamic>? _bookmarksBox;  
  Box<dynamic>? _syncMetaBox;  
  Box<dynamic>? _threatCacheBox;  
  Box<dynamic>? _profileBox;  
  Box<dynamic>? _notificationHistoryBox;  
  
  Future<void> init() async {  
    _articlesBox ??= await Hive.openBox<dynamic>(knowledgeArticlesBox);  
    _bookmarksBox ??= await Hive.openBox<dynamic>(bookmarksBox);  
    _syncMetaBox ??= await Hive.openBox<dynamic>(syncMetaBox);  
    _threatCacheBox ??= await Hive.openBox<dynamic>(threatCacheBox);  
    _profileBox ??= await Hive.openBox<dynamic>(profileBox);  
    _notificationHistoryBox ??=  
      await Hive.openBox<dynamic>(notificationHistoryBox);  
  }  
}
```

saveNotification()

I added: saveNotification()

This saves a notification JSON map into Hive

The notification ID is used as the Hive key

If the ID is missing, the method falls back to the current timestamp.

```
Future<void> saveNotification(Map<String, dynamic> notificationJson) async {
  await init();
  final key = notificationJson['id']?.toString() ??
    DateTime.now().millisecondsSinceEpoch.toString();
  await _notificationHistoryBox!.put(key, notificationJson);
}
```

getNotificationHistory()

This reads notification maps from Hive and sorts them by timestamp
Newest alerts appear first in the Alerts tab.

```
List<Map<String, dynamic>> getNotificationHistory() {
  final values =
    _notificationHistoryBox?.values ?? const Iterable<dynamic>.empty();
  return values
    .whereType<Map>()
    .map((item) => Map<String, dynamic>.from(item))
    .toList()
    ..sort((a, b) {
      final aTime = a['timestamp']?.toString() ?? '';
      final bTime = b['timestamp']?.toString() ?? '';
      return bTime.compareTo(aTime);
    });
}
```

markNotificationRead()

This updates one notification and sets: isRead = true

```
Future<void> markNotificationRead(String id) async {
  await init();
  final raw = _notificationHistoryBox?.get(id);
  if (raw is! Map) return;
  final updated = Map<String, dynamic>.from(raw);
  updated['isRead'] = true;
  await _notificationHistoryBox!.put(id, updated);
}
```

markAllNotificationsRead()

This loops through all notification records and marks each one as read.

```
Future<void> markAllNotificationsRead() async {
  await init();
  final keys = _notificationHistoryBox?.keys.toList() ?? [];
  for (final key in keys) {
    final raw = _notificationHistoryBox?.get(key);
    if (raw is! Map) continue;
    final updated = Map<String, dynamic>.from(raw);
    updated['isRead'] = true;
    await _notificationHistoryBox!.put(key, updated);
  }
}
```

clearNotificationHistory()

This clears the whole notification history Hive box.

```
Future<void> clearNotificationHistory() async {
  await init();
  await _notificationHistoryBox!.clear();
}
```

getUnreadNotificationCount()

This counts all notification records where: isRead != true
The shell uses this count to show the unread dot on the Alerts tab.

```

int getUnreadNotificationCount() {
    final values =
        _notificationHistoryBox?.values ?? const Iterable<dynamic>.empty();
    return values
        .whereType<Map>()
        .where((item) => item['isRead'] != true)
        .length;
}

```

#services/notification_service

This service owns the Firebase Messaging integration

Responsibilities:

- ◆ request notification permission
- ◆ set foreground notification presentation options
- ◆ log the FCM token in debug mode
- ◆ subscribe to base topics
- ◆ subscribe and unsubscribe watchlist topics
- ◆ listen for foreground messages
- ◆ listen for opened background notifications
- ◆ handle initial notification when the app was terminated
- ◆ convert RemoteMessage into AppNotification
- ◆ save notification history to Hive
- ◆ refresh NotificationController
- ◆ show foreground snackbar
- ◆ route notification taps to the Alerts tab

init()

```

Future<void> init() async {
    await _requestPermission();
    await _messaging.setForegroundNotificationPresentationOptions(
        alert: true,
        badge: true,
        sound: true,
    );
    await _logToken();
    await _subscribeToBaseTopics();
    _listenForeground();
    _listenBackground();
    await _handleInitialMessage();
}

```

This sets up the full notification lifecycle.

Android Permission Handling

The service checks Android notification settings with: `FirebaseMessaging.instance.getNotificationSettings()`


```

Future<void> _requestPermission() async {
  if (defaultTargetPlatform == TargetPlatform.android) {
    final settings = await _messaging.getNotificationSettings();
    if (kDebugMode) {
      debugPrint(
        '[FCM] Android permission status: '
        '${settings.authorizationStatus.name}',
      );
    }
    return;
  }

  final settings = await _messaging.requestPermission(
    alert: true,
    badge: true,
    sound: true,
  );
  if (kDebugMode) {
    debugPrint(
      '[FCM] Permission status: ${settings.authorizationStatus.name}';
    );
  }
}

```

FCM Token Logging

The service gets the FCM token with: `_messaging.getToken()`

In debug mode, it prints: [FCM] Token: ...

It also listens to token refresh events.

This token can be used for direct device testing from Firebase Console.

```

Future<void> _logToken() async {
  final token = await _messaging.getToken();
  if (kDebugMode) {
    debugPrint('[FCM] Token: $token');
  }

  _messaging.onTokenRefresh.listen((token) {
    if (kDebugMode) {
      debugPrint('[FCM] Refreshed token: $token');
    }
  });
}

```

```

I/flutter ( 7604): [FCM] Token: eS_ZejdOSV0-l_dFJsMETn:APA91bGt6CSpg8LMTogu5e99b
HX4hRB5p1w001MKUcy3QTfARFY04vDdGthvdQZk3dmVkkx8FZm1s7ykg2_dPXR9ag0A0ZTITqCnxzEg

```

Base Topic Subscription

The app subscribes to: `cybershield_alerts`

This topic is used for general CyberShield alerts.

```

Future<void> _subscribeToBaseTopics() async {
  await _messaging.subscribeToTopic('cybershield_alerts');
}

```

Watchlist Topic Subscription

I added methods:

`subscribeToWatchlistTopics()`

`unsubscribeFromWatchlistTopics()`

Each watchlist technology becomes a topic.

Example:

Flutter becomes: `tech_flutter`

Firebase becomes: `tech_firebase`

Android becomes: `tech_android`

Spaces are converted to underscores.

```
Future<void> subscribeToWatchlistTopics(List<String> watchlist) async {  
  for (final tech in watchlist) {  
    final topic = _topicFromTech(tech);  
    await _messaging.subscribeToTopic(topic);  
  }  
}  
  
Future<void> unsubscribeFromWatchlistTopics(List<String> watchlist) async {  
  for (final tech in watchlist) {  
    final topic = _topicFromTech(tech);  
    await _messaging.unsubscribeFromTopic(topic);  
  }  
}
```

Foreground Messages

Foreground messages are handled with: `FirebaseMessaging.onMessage.listen()`

When a foreground message arrives:

- 1- The message is converted to `AppNotification`
- 2- The alert is save to Hive
- 3- The in-memory alerts list is refreshed
- 4- A `GetX` snackbar is shown

```
void _listenForeground() {  
  FirebaseMessaging.onMessage.listen((RemoteMessage message) {  
    if (kDebugMode) {  
      debugPrint('[FCM] Foreground message received: ${message.messageId}');  
    }  
    _handleMessage(message);  
    _showForegroundSnackbar(message);  
  });  
}
```

```
void _showForegroundSnackbar(RemoteMessage message) {  
  final data = message.data;  
  final title = message.notification?.title ??  
    data['title']?.toString() ??  
    'CyberShield Alert';  
  final body = message.notification?.body ?? data['body']?.toString() ?? '';  
  final severityColor = _snackbarSeverityColor(data['severity']?.toString());  
  
  Get.snackbar(  
    title,  
    body,  
    backgroundColor: AppColors.surface,  
    colorText: AppColors.white,  
    borderColor: severityColor,  
    borderWidth: 1,  
    borderRadius: 12,  
    margin: const EdgeInsets.all(16),  
    duration: const Duration(seconds: 5),  
    snackPosition: SnackPosition.TOP,  
    icon: Icon(  
      Icons.notifications_active_outlined,  
      color: severityColor,  
      size: 22,  
    ),  
    shouldIconPulse: false,  
  );  
}
```

Tapping View:

- 1- Closes the snackbar
- 2- Saves the notification as read
- 3- Opens the Alerts tab

Background Notification Open

Background notification taps are handled with:
FirebaseMessaging.onMessageOpenedApp.listen()

When the user taps a notification while the app is already in memory:

- 1- The message is saved to Hive.
- 2- The alert is marked read.
- 3- NotificationController refreshes.
- 4- The shell switches to the Alerts tab.

```
void _listenBackground() {  
  FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {  
    if (kDebugMode) {  
      debugPrint('[FCM] Notification opened: ${message.messageId}');  
    }  
    unawaited(_handleOpenedMessage(message));  
  });  
}  
  
Future<void> _handleOpenedMessage(RemoteMessage message) async {  
  final notification = _messageToNotification(message).copyWith(isRead: true);  
  await _saveAndRefresh(notification);  
  await _storageService.markNotificationRead(notification.id);  
  
  if (Get.isRegistered<NotificationController>()) {  
    Get.find<NotificationController>().refreshFromStorage();  
  }  
  openAlertsWhenReady();  
}
```

Terminated App Notification Open

Terminated app notification taps are handled with: _messaging.getInitialMessage()
This checks whether the app was launched by tapping a notification.

If a message exists:

- 1- The notification is saved.
- 2- It is marked read.
- 3- The app stores a pending open-alerts request.
- 4- When the shell becomes ready, it switches to the Alerts tab.

```
Future<void> _handleInitialMessage() async {  
  final message = await _messaging.getInitialMessage();  
  if (message != null) {  
    if (kDebugMode) {  
      debugPrint('[FCM] Initial notification opened: ${message.messageId}');  
    }  
    unawaited(_handleOpenedMessage(message));  
  }  
}
```

```

void openAlertsWhenReady() {
    _shouldOpenAlertsWhenShellReady = true;

    WidgetsBinding.instance.addPostFrameCallback((_) {
        if (!_shouldOpenAlertsWhenShellReady) return;
        if (Get.IsRegistered<MainShellViewState>()) {
            _shouldOpenAlertsWhenShellReady = false;
            Get.find<MainShellViewState>().switchToTab(3);
            return;
        }

        final currentRoute = Get.currentRoute;
        final onAuthRoute = currentRoute == AppRoutes.LOGIN ||
            currentRoute == AppRoutes.REGISTER ||
            currentRoute == AppRoutes.FORGOT_PASSWORD ||
            currentRoute.isEmpty;
        if (!onAuthRoute) {
            Get.offAllNamed(AppRoutes.SHELL);
        }
    });
}

bool consumePendingAlertsOpen() {
    if (!_shouldOpenAlertsWhenShellReady) return false;
    _shouldOpenAlertsWhenShellReady = false;
    return true;
}

```

Notification Tap Routing Fix

During testing, the notification arrived correctly, but tapping it only dismissed the Android notification.

Problem:

- ◆ The message was saved.
- ◆ But the app did not navigate to the Alerts tab.
- ◆ In cold start, notification handling ran before the GetX navigator and shell were ready.
- ◆ The alert only appeared after manually refreshing the Alerts page.

Fix:

- ◆ `_handleOpenedMessage` now saves the alert and marks it read.
- ◆ `openAlertsWhenReady` stores a pending request.
- ◆ `MainShellViewState` consumes that request after the shell is built.
- ◆ `switchToTab(3)` opens the Alerts tab.
- ◆ `NotificationController` refreshes from Hive.

Result:

- ◆ Tapping a notification now opens the Alerts tab.
- ◆ The opened notification appears as read instead of staying in the unread grey placeholder state.

```
Future<void> _handleOpenedMessage(RemoteMessage message) async {
  final notification = _messageToNotification(message).copyWith(isRead: true);
  await _saveAndRefresh(notification);
  await _storageService.markNotificationRead(notification.id);

  if (Get.isRegistered<NotificationController>()) {
    Get.find<NotificationController>().refreshFromStorage();
  }

  openAlertsWhenReady();
}
```

```
void openAlertsWhenReady() {
  _shouldOpenAlertsWhenShellReady = true;

  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (!_shouldOpenAlertsWhenShellReady) return;
    if (Get.isRegistered<MainShellViewState>()) {
      _shouldOpenAlertsWhenShellReady = false;
      Get.find<MainShellViewState>().switchToTab(3);
      return;
    }

    final currentRoute = Get.currentRoute;
    final onAuthRoute = currentRoute == AppRoutes.LOGIN ||
      currentRoute == AppRoutes.REGISTER ||
      currentRoute == AppRoutes.FORGOT_PASSWORD ||
      currentRoute.isEmpty;
    if (!onAuthRoute) {
      Get.offAllNamed(AppRoutes.SHELL);
    }
  });
}
```

#controllers/notification_controller

I created NotificationController.

This controller manages the Alerts tab state.

```
class NotificationController extends GetxController {
  RxList<AppNotification> notifications = <AppNotification>[].obs;
  RxInt unreadCount = 0.obs;
  RxBool isLoading = false.obs;
  RxString selectedSeverityFilter = ''.obs;

  @override
  void onInit() {
    super.onInit();
    refreshFromStorage();
  }
}
```

refreshFromStorage()

This method reads notification history from StorageService.

It converts each map into AppNotification.

It updates:

- ◆ notifications
- ◆ unreadCount

```
void refreshFromStorage() {
  final storageService = Get.find<StorageService>();
  final rawList = storageService.getNotificationHistory();
  final parsed = rawList
    .map((json) => AppNotification.fromJson(json))
    .where((notification) => notification.id.isNotEmpty)
    .toList();
  notifications.assignAll(parsed);
  unreadCount.value = storageService.getUnreadNotificationCount();
}
```

addNotificationInMemory()

This method inserts a newly received notification into the current list.

If the notification already exists, it updates the existing item instead of duplicating it.

This is important because the same FCM message can be processed when received and again when opened.

```
void addNotificationInMemory(AppNotification notification) {  
    final existingIndex =  
        notifications.indexWhere((item) => item.id == notification.id);  
    if (existingIndex != -1) {  
        notifications[existingIndex] = notification;  
        _updateUnreadCount();  
        return;  
    }  
}
```

openNotification()

This method is called when the user taps an alert card.

It:

- 1- Marks the alert as read.
- 2- Checks whether the alert has a CVE ID.
- 3- Builds a ThreatAdvisory object.
- 4- Opens the threat detail route.
- 5- The detail screen can then show the CVE ID, severity, score, description, and dates based on the alert.

The detail screen can then show the CVE ID, severity, score, description, and dates based on the alert.

```
void openNotification(AppNotification notification) {  
    unawaited(markAsRead(notification.id));  
  
    final cveId = notification.cveId;  
    if (cveId == null || cveId.isEmpty) return;  
  
    final advisory = ThreatAdvisory(  
        id: cveId,  
        description: notification.body.isNotEmpty  
            ? notification.body  
            : 'No description available for this advisory.',  
        baseScore: _scoreFromSeverity(notification.severity),  
        severity: _normalizeSeverity(notification.severity),  
        publishedDate: notification.timestamp,  
        lastModifiedDate: notification.timestamp,  
        referenceUrls: const [],  
    );  
    Get.toNamed(AppRoutes.THREAT_DETAIL, arguments: advisory);  
}
```

Severity Filtering

The controller supports severity filters: All CRITICAL HIGH MEDIUM LOW

filterBySeverity() stores the selected filter.

filteredNotifications returns only matching alerts.

```

void filterBySeverity(String severity) {
    selectedSeverityFilter.value =
        severity.toUpperCase() == 'ALL' ? '' : severity.toUpperCase();
}

List<AppNotification> get filteredNotifications {
    if (selectedSeverityFilter.value.isEmpty) return notifications;
    return notifications
        .where((notification) =>
            (notification.severity?.toUpperCase() ?? '') ==
            selectedSeverityFilter.value)
        .toList();
}

```

Read and Clear Methods

The controller includes:

- ◆ markAsRead()
- ◆ markAllRead()
- ◆ clearAll()

markAsRead reads one alert.

markAllRead marks all alerts as read.

clearAll removes all local alert history.

```

Future<void> markAsRead(String id) async {
    final storageService = Get.find<StorageService>();
    await storageService.markNotificationRead(id);
    refreshFromStorage();
}

Future<void> markAllRead() async {
    final updated = notifications
        .map((notification) => notification.copyWith(isRead: true))
        .toList();
    notifications.assignAll(updated);
    unreadCount.value = 0;

    final storageService = Get.find<StorageService>();
    await storageService.markAllNotificationsRead();
}

Future<void> clearAll() async {
    final storageService = Get.find<StorageService>();
    await storageService.clearNotificationHistory();
    refreshFromStorage();
}

int get currentUnreadCount => unreadCount.value;

```

#views/alerts/alerts_view

I replaced the placeholder Alerts empty state with a real alerts list.

The Alerts screen now includes:

- ◆ Alerts title
- ◆ Mark all read action
- ◆ severity filter bar
- ◆ pull-to-refresh list
- ◆ alert cards
- ◆ empty state

- ◆ Clear all button

Lifecycle Refresh

AlertsView implements WidgetsBindingObserver

When the app resumes it calls: controller.refreshFromStorage()

This helps the Alerts screen pick up notifications that arrived while the app was backgrounded.

```
@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state == AppLifecycleState.resumed) {
    controller.refreshFromStorage();
  }
}
```

Severity Filter Bar

```
class _SeverityFilterBar extends StatelessWidget {
  final NotificationController controller;

  const _SeverityFilterBar({required this.controller});

  @override
  Widget build(BuildContext context) {
    const filters = ['ALL', 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'];

    return SizedBox(
      height: 36,
      child: Obx(() {
        final selectedFilter = controller.selectedSeverityFilter.value;
        return ListView.separated(
          scrollDirection: Axis.horizontal,
          itemCount: filters.length,
          separatorBuilder: (_, __) => const SizedBox(width: 8),
          itemBuilder: (context, index) {
            final filter = filters[index];
            final selected = selectedFilter.isEmpty
              ? filter == 'ALL'
              : selectedFilter == filter;
            return GestureDetector(
              onTap: () => controller.filterBySeverity(filter),
              child: AnimatedContainer(
                duration: const Duration(milliseconds: 160),
                padding: const EdgeInsets.symmetric(horizontal: 14),
                alignment: Alignment.center,
                decoration: BoxDecoration(
                  color: selected ? AppColors.primary : AppColors.surface,
                  borderRadius: BorderRadius.circular(999),
                  border: Border.all(
                    color: selected ? AppColors.primary : AppColors.border,
                    width: 1,
                  ),
                ),
                child: Text(
                  filter,
                  style: TextStyle(
                    color:
                      selected ? AppColors.background : AppColors.textMuted,
                    fontSize: 12,
                    fontWeight: FontWeight.w600,
                  ),
                ),
              ),
            );
          },
        );
      })
    );
  }
}
```


#views/shell/main_shell_view

I updated MainShellView so it can react to notification taps and unread counts.

Changes:

- ♦ The state class is now MainShellViewState.
- ♦ The state is registered in GetX.
- ♦ NotificationService can switch to the Alerts tab.
- ♦ The shell checks for pending notification tap requests after build.
- ♦ Alerts tab refreshes when selected.
- ♦ Alerts icon shows a red unread dot when unreadCount is greater than 0.

```
class MainShellView extends StatefulWidget {  
  const MainShellView({super.key});  
  
  @override  
  State<MainShellView> createState() => MainShellViewState();  
}  
  
class MainShellViewState extends State<MainShellView> {  
  int currentIndex = 0;
```

```
@override  
void initState() {  
  super.initState();  
  Get.put<MainShellViewState>(this);  
  WidgetsBinding.instance.addPostFrameCallback((_) {  
    if (!Get.isRegistered<NotificationService>()) return;  
    if (Get.find<NotificationService>().consumePendingAlertsOpen()) {  
      switchToTab(3);  
    }  
  });  
}  
  
@override  
void dispose() {  
  if (Get.isRegistered<MainShellViewState>()) {  
    Get.delete<MainShellViewState>();  
  }  
  super.dispose();  
}  
  
void switchToTab(int index) {  
  setState(() => currentIndex = index);  
  if (index == 3 && Get.isRegistered<NotificationController>()) {  
    Get.find<NotificationController>().refreshFromStorage();  
  }  
}
```

```
BottomNavigationBarItem(  
  icon: Obx(() {  
    final count = Get.isRegistered<NotificationController>()  
      ? Get.find<NotificationController>().unreadCount.value  
      : 0;  
    return _AlertsNavIcon(  
      unreadAlertCount: count,  
      isActive: false,  
    );  
  }),  
  activeIcon: Obx(() {  
    final count = Get.isRegistered<NotificationController>()  
      ? Get.find<NotificationController>().unreadCount.value  
      : 0;  
    return _AlertsNavIcon(  
      unreadAlertCount: count,  
      isActive: true,  
    );  
  }),  
  label: 'Alerts',  
)
```

When the app is launched from a notification, NotificationService may receive the message before MainShellView exists.

To solve this:

- 1- NotificationService stores a pending open-alerts flag.
- 2- MainShellViewState checks the flag after the first frame.
- 3- If the flag exists, it switches to tab index 3.

Tab index 3 is: Alerts

#controllers/profile_controller

I updated ProfileController to connect watchlist chips to notification topics.

Injected: NotificationService

On startup:

- ♦ The controller subscribes to the current watchlist topics.

When a technology is added:

- ♦ The app saves the profile.
- ♦ The app subscribes to that topic.

When a technology is removed:

- ♦ The app saves the profile.
- ♦ The app unsubscribes from that topic.

```
class ProfileController extends GetxController {  
  final AuthController authController = Get.find<AuthController>();  
  final AuthService authService = Get.find<AuthService>();  
  final StorageService _storageService = Get.find<StorageService>();  
  final NotificationService _notificationService =  
    Get.find<NotificationService>();
```

```
@override  
void onInit() {  
  super.onInit();  
  _populateFromCurrentUser();  
  _loadPersistedProfile();  
  unawaited(  
    _notificationService.subscribeToWatchlistTopics(watchlist.toList());  
  );  
}
```

```
void addTechnology(String name) {  
  final value = name.trim();  
  if (value.isEmpty) return;  
  final exists = watchlist.any(  
    (item) => item.toLowerCase() == value.toLowerCase(),  
  );  
  if (exists) return;  
  watchlist.add(value);  
  _syncProfile();  
  _saveProfile();  
  unawaited(_notificationService.subscribeToWatchlistTopics([value]));  
}
```

```

void removeTechnology(String name) {
  watchList.removeWhere(
    (item) => item.toLowerCase() == name.toLowerCase(),
  );
  _syncProfile();
  _saveProfile();
  _notificationService.unsubscribeFromWatchlistTopics([name]);
}

```

#bindings/shell_binding

I updated ShellBinding to register notification dependencies for the authenticated shell. It now registers:

- ◆ NotificationService
- NotificationController

The binding checks whether each dependency is already registered before creating it. This prevents duplicate service and controller instances.

```

if (!Get.isRegistered<NotificationService>()) {
  Get.put(NotificationService());
}

if (!Get.isRegistered<NotificationController>()) {
  Get.put(NotificationController());
}

```

#main.dart

I updated the main.dart to initialize notification support,

StorageService must be ready before NotificationService tries to save received alerts

```

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  final storageService = StorageService();
  await storageService.init();
  Get.put<StorageService>(storageService);
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
  final notificationService = NotificationService();
  Get.put<NotificationService>(notificationService);
  await notificationService.init();
  runApp(const CyberShieldApp());
}

```

Background Handler:

I added: _firebaseMessaginBackgroundHandler()

This lets Firebase call the handler when a message arrives while the app in the background

```

@pragma('vm:entry-point')
Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  if (kDebugMode) {
    debugPrint('[FCM] Background message received: ${message.messageId}');
  }
}

```

Issue Found During Testing

Issue: The notification arrived correctly on the phone.
The severity, title, body, and visual details were correct.

But when I tapped the notification:

- ♦ the notification disappeared
- ♦ the app did not open the Alerts tab
- ♦ the alert did not appear immediately in the Alerts page
- ♦ manual refresh was needed
- ♦ after refresh, the alert appeared as an unread grey card
- ♦ opening the alert was needed to make it look like the read state

Cause:

- ♦ Background notification open events only saved the message.
- ♦ They did not navigate to the Alerts tab.
- ♦ Cold-start notification open handling ran before the shell was ready.
- ♦ The UI did not refresh immediately from Hive after the tap.

Fix:

- ♦ Added `_handleOpenedMessage()`.
- ♦ Saved opened notifications as read.
- ♦ Refreshed `NotificationController` after saving.
- ♦ Added `openAlertsWhenReady()`.
- ♦ Added `consumePendingAlertsOpen()`.
- ♦ Registered `MainShellViewState` in `GetX`.
- ♦ Switched to Alerts tab when the notification is tapped.

Result:

- ♦ Tapping the notification now opens the Alerts tab.
- ♦ The alert appears without manual refresh.
- ♦ The opened alert appears in read style.

Security Notes:

Notification security:

- ♦ FCM token is printed only in debug mode.
- ♦ Notification history is stored locally in Hive.

- ♦ Alert data is limited to title, body, severity, CVE ID, timestamp, and read state.
- ♦ No password or sensitive auth data is stored in notification history.
- ♦ Notification topic names are generated from user watchlist technology names.

Privacy notes:

- ♦ Push notifications can be visible on the device lock screen depending on system settings.
- ♦ Alert bodies should not contain private user data.
- ♦ They should contain general security advisory information only.

Firestore console Testing:

I tested notifications by publishing alerts from Firestore Console.

Important Firestore Console fields:

- ♦ Notification title
- ♦ Notification text/body
- ♦ Topic or device token target
- ♦ Custom data fields

Example Firestore Payload

Example custom data for a critical CVE alert:

title = Critical Alert
 body = VE-2026-9999 CVSS 9.8 - patch now
 severity = CRITICAL
 cveld = CVE-2026-9999

Example custom data for a medium CVE alert:

title = Critical Alert
 body = VE-2026-7777 CVSS 7.2 - patch now
 severity = MEDIUM
 cve_id = CVE-2026-7777

Expected behavior:

- ♦ Notification arrives on the phone.
- ♦ The system notification shows the correct title and body.
- ♦ The Alerts tab stores the alert.
- ♦ The severity chip uses the correct color.
- ♦ The alert is unread until opened.

- ♦ Tapping the notification opens the Alerts tab and marks it read.

Screenshots :

1 Notification

Notification title ⓘ
Critical Alert

Notification text
VE-2026-8888 CVSS 8.5 - patch now

Notification image (optional) ⓘ
Example: <https://yourapp.com/image.png>

Notification name (optional) ⓘ
Critical Alert

Device preview

This preview provides a general idea of how your message will appear on a mobile device. Actual message rendering will vary depending on the device. Test with a real device for actual results.

[Send test message](#)

Initial state Expanded view

Android

Test on device

You can test this campaign by entering or selecting the [FCM registration tokens](#) of your development device below.

Add an FCM registration token

☒ e5_Zejd0SV0-1_dFJ3sMETn:APA91bGt6CSpg8LMTogu5e99bUx4hR85g1wQ0
1MKUcv3QIFARFY04vDdGIbyd0ZK3dmVKcxRE2mJ57ykq2_dPXr9ao0AQZIIg
pCnxzEgex71Ex_ZffGveLU

Cancel

Test

2 Target

User segment **Topic**

Message topic ⓘ

cybershield_alerts

Next

3 Scheduling

Send to eligible users

Now

Next

6 Additional options (optional)

All fields optional

Android Notification Channel ⓘ

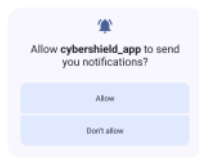
Custom data ⓘ

severity	HIGH
----------	------

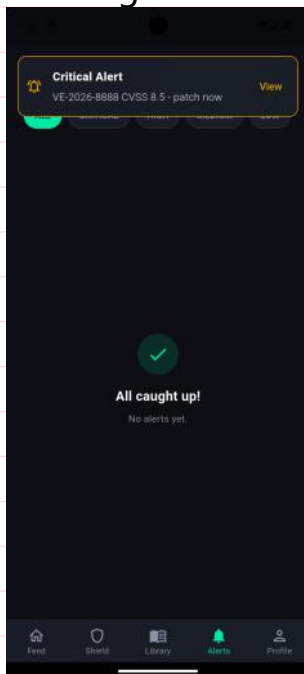
cveId	CVE-2026-8888
-------	---------------

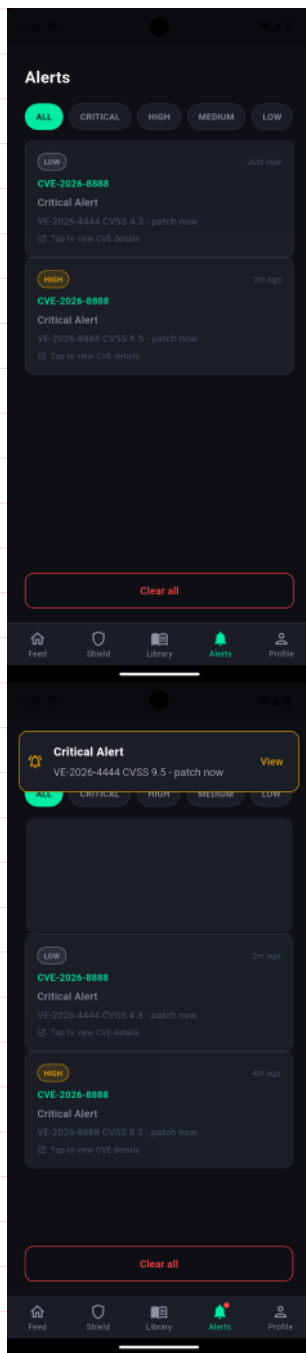
Key	Value
-----	-------

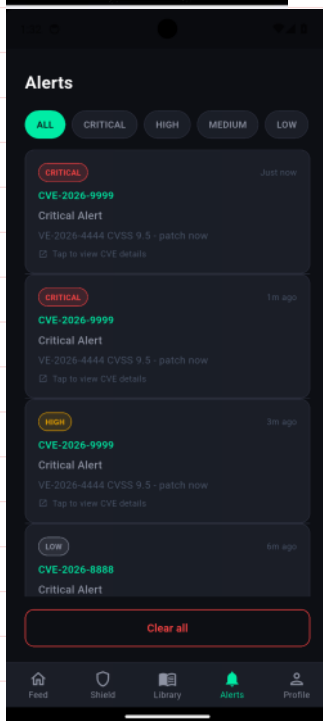
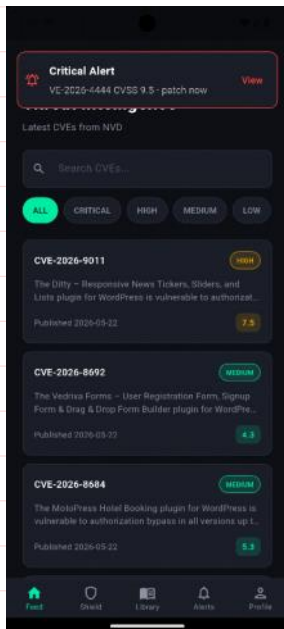
When I run the app:



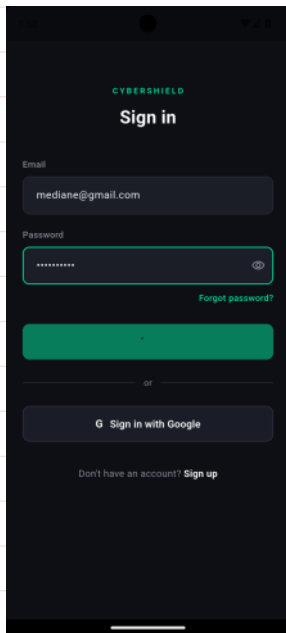
Sending a notification:



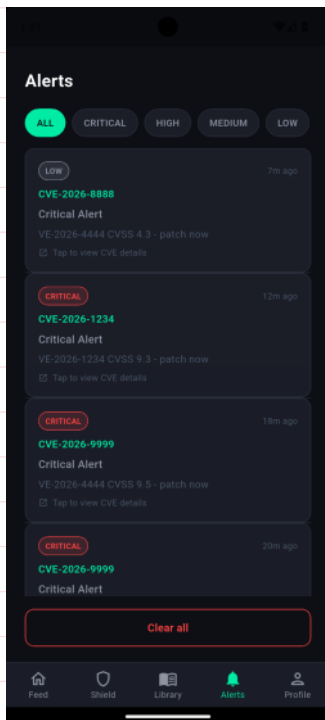




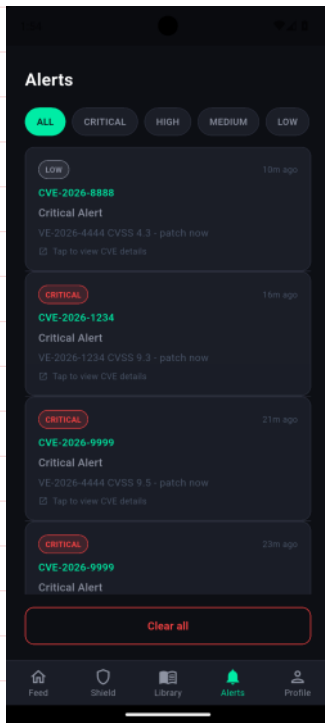
After sign out and log in:



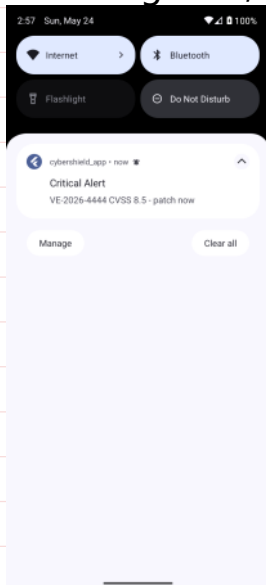
The alerts are saved and stored:



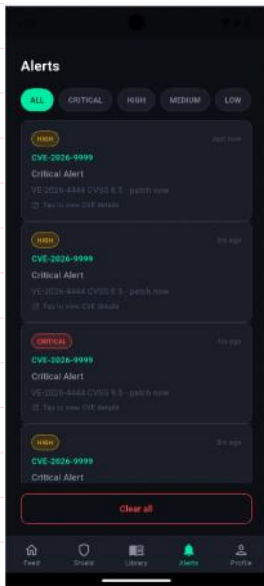
I stopped the app, and re-run it.



Receiving alert, when the app is in the background:



When I press on it, it takes me to the alert page:



Sending alert when the app is not running:

Review message

Notification content

 VE-2026-4444 CVSS 8.5 - patch now

Target

 Subscribers of cybershield_alerts topic

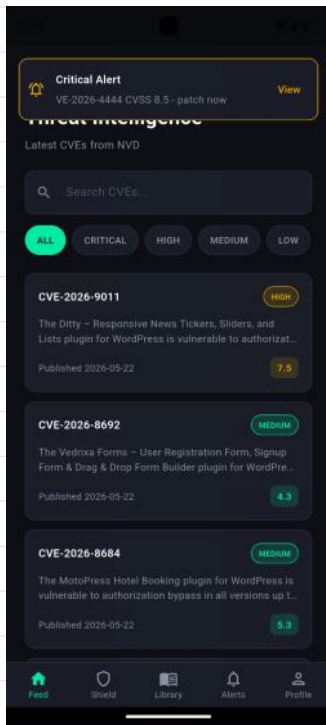
Scheduling

 Send now

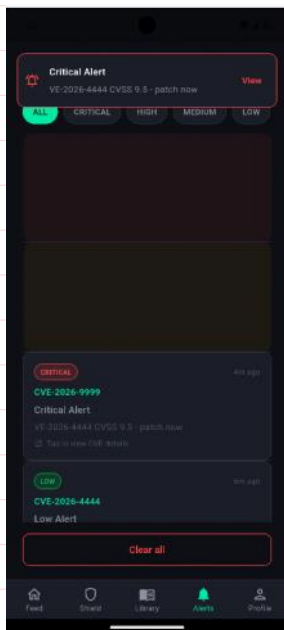
Cancel **Publish**



When I run the app:



The colored blocks, are the unread alerts, the yellow one is the high alert, the red one is the critical alert.



Testing & Optimization

In this step I implemented Deliverable 9: Testing and Optimization for the CyberShield Flutter mobile app.

The goal of this step is to finish important behavior gaps, improve app quality, and add a real automated test suite.

Deliverable 9 focuses on:

- ♦ Watchlist-based threat feed behavior.
- ♦ Notification preference enforcement.
- ♦ Critical alert filtering.
- ♦ NVD threat cache improvements.
- ♦ Password analysis cleanup and optimization.
- ♦ Storage and profile robustness.
- ♦ Unit tests for models, controllers, services, and utilities.
- ♦ Analyzer and test verification.

#controllers/threat_feed_controller

I updated ThreatFeedController with a watchlist-aware behavior.

```
class ThreatFeedController extends GetxController {
  final ApiService _apiService;
  final bool _autoBootstrap;
  final ProfileController? Function()? _profileControllerProvider;
  final List<ThreatAdvisory> _allThreats = <ThreatAdvisory>[];
  int _fetchGeneration = 0;
  bool _lastRemoteHasMorePages = false;
  ProfileController? _cachedProfileController;
```

```
Future<List<ThreatAdvisory>> _fetchRemoteThreatPage({
  required int page,
  required bool useRemoteSeverity,
}) async {
  final severity = useRemoteSeverity ? selectedSeverity.value : null;
  final watchlistKeywords = _watchlistKeywords();

  if (!isWatchlistModeActive.value || watchlistKeywords.isEmpty) {
    final results = await _apiService.fetchThreats(
      page: page,
      severity: severity,
    );
    _lastRemoteHasMorePages = _apiService.hasMoreThreatPages;
    return results;
  }

  final combined = <ThreatAdvisory>[];
  var hasMore = false;
  for (final keyword in watchlistKeywords) {
    final results = await _apiService.fetchThreats(
      page: page,
      keyword: keyword,
      severity: severity,
    );
    if (_apiService.errorMessage.isEmpty) {
      combined.addAll(results);
    }
  }
}
```

```

        _lastRemoteHasMorePages = hasMore;
        if (combined.isEmpty) {
            _apiService.errorMessage = '';
        }
        return _dedupeNewestFirst(combined);
    }

    List<String> _watchlistKeywords() {
        final watchlist = _profileController?.watchlist ?? <String>[];
        return watchlist
            .map((technology) => technology.trim())
            .where((technology) => technology.isNotEmpty)
            .toSet()
            .take(6)
            .toList();
    }

```

Watchlist Mode

Watchlist mode now controls whether the threat feed should focus on the technologies saved in Profile.

Before:

- ◆ The watchlist existed in the Profile screen.
- ◆ It was saved to Hive.
- ◆ It subscribed to FCM topics.
- ◆ But the threat feed still showed a generic latest NVD feed.

Now:

- ◆ Watchlist mode is active by default.
- ◆ The feed reads the current watchlist from ProfileController.
- ◆ The feed requests NVD data using keywordSearch for watchlist terms.
- ◆ Results are deduplicated and sorted newest first.
- ◆ The Watchlist chip can turn this behavior on or off.

If the watchlist contains: android linux SQL

The controller can fetch NVD results using those keywords.

Important:

Generic keywords such as "software", "injection", or "SQL" can match many unrelated CVEs because NVD keyword search is text-based.

More specific technology names produce better results, for example:

- ◆ Flutter
- ◆ Firebase
- ◆ Apache
- ◆ OpenSSL
- ◆ WordPress
- ◆ Linux Kernel

```

if (!isWatchlistModeActive.value || watchlistKeywords.isEmpty) {
    final results = await _apiService.fetchThreats(
        page: page,
        severity: severity,
    );
    _lastRemoteHasMorePages = _apiService.hasMoreThreatPages;
    return results;
}

final combined = <ThreatAdvisory>[];
var hasMore = false;
for (final keyword in watchlistKeywords) {
    final results = await _apiService.fetchThreats(
        page: page,
        keyword: keyword,
        severity: severity,
    );
    if (_apiService.errorMessage.isEmpty) {
        combined.addAll(results);
    }
    hasMore = hasMore || _apiService.hasMoreThreatPages;
}

```

Watchlist Chip

The feed now has an extra chip: WATCHLIST

When active:

- ♦ The chip uses a teal border
- ♦ The app fetches/filter threats using the profile watchlist

When inactive:

- ♦ The app shows the generic NVD feed

Local Watchlist Filtering

Even after fetching from NVD by keyword, the controller still applies a local filter.

The local filter checks:

- ♦ CVE ID
- ♦ CVE description

It matches against the watchlist keywords case-insensitively.

This prevents unrelated results from remaining in the list when possible.

```

List<ThreatAdvisory> _filteredThreats(List<ThreatAdvisory> source) {
    Iterable<ThreatAdvisory> results = source;

    final severity = selectedSeverity.value.trim().toUpperCase();
    if (severity.isNotEmpty) {
        results = results.where(
            (threat) => threat.severity.toUpperCase() == severity,
        );
    }

    final keyword = searchKeyword.value.trim().toLowerCase();
    if (keyword.isNotEmpty) {
        results = results.where((threat) {
            return threat.id.toLowerCase().contains(keyword) ||
                threat.description.toLowerCase().contains(keyword) ||
                threat.severity.toLowerCase().contains(keyword);
        });
    }
}

```



```

if (isWatchlistModeActive.value) {
    final watchlist = _watchlistKeywords();
    if (watchlist.isNotEmpty) {
        final lowerWatchlist = watchlist
            .map((technology) => technology.trim().toLowerCase())
            .where((technology) => technology.isNotEmpty)
            .toList();
        if (lowerWatchlist.isNotEmpty) {
            results = results.where((threat) {
                final haystack = '${threat.id} ${threat.description}'.toLowerCase();
                return lowerWatchlist.any(haystack.contains);
            });
        }
    }
}

return _newestFirst(results);
}

```

Critical Alerts And Threat Feed

The Profile critical alerts setting no longer filters the threat feed

Why:

- ◆ The Critical alerts switch is a notification preference
- ◆ The threat feed should continue to show CVE data that matches the watchlist, severity filter, and search query
- ◆ Turning Critical alerts OFF should not remove Critical CVEs from the feed

```

List<ThreatAdvisory> _filteredThreats(List<ThreatAdvisory> source) {
    Iterable<ThreatAdvisory> results = source;

    final severity = selectedSeverity.value.trim().toUpperCase();
    if (severity.isNotEmpty) {
        results = results.where(
            (threat) => threat.severity.toUpperCase() == severity,
        );
    }

    final keyword = searchKeyword.value.trim().toLowerCase();
    if (keyword.isNotEmpty) {
        results = results.where((threat) {
            return threat.id.toLowerCase().contains(keyword) ||
                threat.description.toLowerCase().contains(keyword) ||
                threat.severity.toLowerCase().contains(keyword);
        });
    }

    if (isWatchlistModeActive.value) {
        final watchlist = _watchlistKeywords();
        if (watchlist.isNotEmpty) {
            final lowerWatchlist = watchlist
                .map((technology) => technology.trim().toLowerCase())
                .where((technology) => technology.isNotEmpty)
                .toList();
            if (lowerWatchlist.isNotEmpty) {
                results = results.where((threat) {
                    final haystack = '${threat.id} ${threat.description}'.toLowerCase();
                    return lowerWatchlist.any(haystack.contains);
                });
            }
        }
    }

    return _newestFirst(results);
}

```

Generation Guard

I added a generation guard to loadMoreThreats().

Purpose:

- ◆ If the user changes filters while pagination is running, stale results are discarded
- ◆ This prevents old pages from being appended to the wrong filter state

```
Future<void> loadMoreThreats() async {
  final generation = _fetchGeneration;
  errorMessage.value = '';

  if (isLoading.value || isLoadingMore.value || !hasMorePages.value) {
    return;
  }
}
```

```
try {
  final nextPage = currentPage.value + 1;
  final results = await _fetchRemoteThreatPage(
    page: nextPage,
    useRemoteSeverity: selectedSeverity.value.isNotEmpty,
  );

  if (generation != _fetchGeneration) return;

  if (_apiService.errorMessage.isNotEmpty) {
    errorMessage.value = _apiService.errorMessage;
    return;
  }

  _allThreats
    ..addAll(results)
    ..replaceRange(0, _allThreats.length, _dedupeNewestFirst(_allThreats));
  _applyFiltersToLoadedThreats();
  currentPage.value = nextPage;
  hasMorePages.value = _lastRemoteHasMorePages;
} catch (_) {
  if (generation == _fetchGeneration) {
    errorMessage.value = 'Could not load more threats. Please try again.';
  }
} finally {
}
```

#views/home/threat_feed_view

I updated the Threat Feed UI

Changes:

- ◆ Added WATCHLIST chip beside severity chips
- ◆ The chip calls controller.toggleWatchlistMode()
- ◆ The chip reflects controller.isWatchlistModeActive
- ◆ The chip uses the existing CyberShield colors

```

return SizedBox(
  height: 38,
  child: ListView.separated(
    scrollDirection: Axis.horizontal,
    itemCount: filters.length + 1,
    separatorBuilder: (_, __) => const SizedBox(width: 8),
    itemBuilder: (context, index) {
      if (index == filters.length) {
        return _FilterChip(
          label: 'WATCHLIST',
          isSelected: isWatchlistModeActive,
          isDisabled: isDisabled,
          useOutlinedActiveStyle: true,
          onTap: onWatchlistToggle,
        );
      }

      final filter = filters[index];
      final isSelected = selectedSeverity.isEmpty
        ? filter == 'ALL'
        : filter == selectedSeverity;

      return _FilterChip(
        label: filter,
        isSelected: isSelected,

```

```

class _FilterChip extends StatelessWidget {
  final String label;
  final bool isSelected;
  final bool isDisabled;
  final bool useOutlinedActiveStyle;
  final VoidCallback onTap;

  const _FilterChip({
    required this.label,
    required this.isSelected,
    required this.isDisabled,
    required this.onTap,
    this.useOutlinedActiveStyle = false,
  });

  @override
  Widget build(BuildContext context) {
    final activeFill =
      useOutlinedActiveStyle ? AppColors.surface : AppColors.primary;
    final activeText =
      useOutlinedActiveStyle ? AppColors.primary : AppColors.background;

    return GestureDetector(
      onTap: isDisabled ? null : onTap,
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 160),
        padding: const EdgeInsets.symmetric(horizontal: 14),

```

```

        alignment: Alignment.center,
        decoration: BoxDecoration(
          color: isSelected ? activeFill : AppColors.surface,
          borderRadius: BorderRadius.circular(999),
          border: Border.all(
            color: isSelected ? AppColors.primary : AppColors.border,
          ),
        ),
        child: Text(
          label,
          style: TextStyle(
            color: isSelected
              ? activeText
              : AppColors.textMuted.withValues(alpha: isDisabled ? 0.5 : 1),
            fontSize: 12,
            fontWeight: FontWeight.w600,
          ),
        ),
      ),
    );
  }
}

```

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! hot suraa

#bindings/shell_binding

I updated ShellBinding registration order

Before:

- ♦ ThreatFeedController could be registered before ProfileController

Now:

- ♦ ProfileController is registered before ThreatFeedController

Why:

- ♦ The threat feed needs access to the user's watchlist on startup
- ♦ Registering ProfileController first lets the initial feed load use the saved watchlist correctly

```
if (!Get.isRegistered<NotificationService>()) {  
    Get.put(NotificationService());  
}  
  
if (!Get.isRegistered<NotificationController>()) {  
    Get.put(NotificationController());  
}  
  
if (!Get.isRegistered<ProfileController>()) {  
    Get.put(ProfileController());  
}
```

#services/notification_service

I updated NotificationService so notification preferences actually affect incoming alerts.

```
void _listenForeground() {  
    FirebaseMessaging.onMessage.listen((RemoteMessage message) {  
        if (kDebugMode) {  
            debugPrint('[FCM] Foreground message received: ${message.messageId}');  
        }  
        _handleMessage(message);  
        final notification = _messageToNotification(message);  
        if (_shouldDisplay(notification)) {  
            _showForegroundSnackbar(message);  
        }  
    });  
}
```

```
bool _shouldDisplay(AppNotification notification) {  
    if (!Get.isRegistered<ProfileController>()) return true;  
  
    final profile = Get.find<ProfileController>();  
    return shouldDisplayForPreferences(  
        notification,  
        allNotificationsEnabled: profile.allNotificationsEnabled.value,  
        criticalAlertsEnabled: profile.criticalAlertsEnabled.value,  
        quietHoursEnabled: profile.quietHoursEnabled.value,  
    );  
}  
  
@VisibleForTesting  
static bool shouldDisplayForPreferences(  
    AppNotification notification, {  
        required bool allNotificationsEnabled,  
        required bool criticalAlertsEnabled,  
        required bool quietHoursEnabled,  
        DateTime? now,  
    }) {  
    if (!allNotificationsEnabled) return false;  
  
    if (criticalAlertsEnabled && !_isCriticalAlert(notification)) {  
        return false;  
    }  
}
```

```

if (quietHoursEnabled) {
    final hour = (now ?? DateTime.now()).hour;
    if (hour >= 22 || hour < 7) return false;
}

return true;
}

static bool _isCriticalAlert(AppNotification notification) {
    final severity = (notification.severity ?? '').toUpperCase();
    return severity == 'CRITICAL' ||
        severity == 'HIGH' ||
        (notification.baseScore ?? 0) >= 9.0;
}

```

All Notifications Toggle

When All notifications is OFF:

- ◆ No alert is displayed
- ◆ No alert is saved to local history
- ◆ No foreground snackbar is shown

This was manually tested and worked correctly
(Screenshots at the end)

Critical Alerts Toggle

After manual testing, I corrected the critical alerts behavior
Current behavior:

If Critical alerts is ON:

- ◆ Only Critical and High severity alerts are allowed
- ◆ Alerts with baseScore >= 9.0 are also treated as critical
- ◆ Low, Medium, and Unknown alerts are suppressed

If Critical alerts is OFF:

- ◆ All severities may pass if All notifications is ON

This makes the switch behave as a critical-only mode for Firebase alerts. It does not change the CVE pages or threat feed visibility.

Quiet Hours Toggle

When Quiet hours is ON:

- ◆ Alerts are suppressed between 22:00 and 07:00 local device time

The logic uses: `DateTime.now().hour`

#views/profile/profile_view

I updated the Profile screen text and toggle behavior.

Changes:

- ♦ Added subtitle under My watchlist.
- ♦ Updated notification card subtitle
- ♦ Dimmed Critical alerts and Quiet hours when All notifications is OFF
- ♦ Disabled Critical alerts and Quiet hours switches when All notifications is OFF

Watchlist subtitle: Technologies below filter your threat feed

Critical alerts subtitle: On: Critical & High only. Off: all alerts

Quiet hours subtitle: Suppress alerts 10 PM - 7 AM

All notifications subtitle: Master switch for all alerts

#service/api_service

I updated ApiService cache key behavior.

Before:

- ♦ Cache keys did not clearly encode all feed modes
- ♦ Generic feed data could collide with severity-filtered or keyword-filtered data

Now:

- ♦ Cache key includes page
- ♦ Cache key includes severity
- ♦ Cache key includes keyword when present

```
String _cacheKey({
  required int page,
  required String keyword,
  required String severity,
  required String publishStartDate,
  required String publishEndDate,
}) {
  final severitySegment = severity.trim().isEmpty ? 'ALL' : severity.trim();
  final keywordSegment = keyword.trim().isEmpty
    ? ''
    : '_k${keyword.trim().toUpperCase().replaceAll(RegExp(r'^A-Z0-9+'), '_')}';
  return 'threats_p${page}_s$severitySegment$keywordSegment';
}
```

#services/storage_service

I made StorageService.init() idempotent

I added:

- ♦ `_initialized` boolean
- ♦ early return when boxes are already open

Why?

Because several services/controllers can call `init()`. The app should not try to reopen Hive boxes unnecessarily

```
Future<void> init() async {  
  if (_initialized) return;  
  
  _articlesBox ??= await Hive.openBox<dynamic>(knowledgeArticlesBox);  
  _bookmarksBox ??= await Hive.openBox<dynamic>(bookmarksBox);  
  _syncMetaBox ??= await Hive.openBox<dynamic>(syncMetaBox);  
  _threatCacheBox ??= await Hive.openBox<dynamic>(threatCacheBox);  
  _profileBox ??= await Hive.openBox<dynamic>(profileBox);  
  _notificationHistoryBox ??=  
    await Hive.openBox<dynamic>(notificationHistoryBox);  
  _initialized = true;  
}
```

#controllers/profile_controller

I added corrupted profile cache protection.

Before:

- ♦ If Hive contained corrupted or old profile data, `UserProfile.fromJson()` could crash the profile screen.

Now:

- ♦ `_loadPersistedProfile()` wraps parsing in try/catch.
- ♦ If parsing fails, the app keeps the current default/current-user profile instead of crashing.

```
void _loadPersistedProfile() {  
  final persisted = _storageService.loadProfile();  
  if (persisted == null) return;  
  
  late final UserProfile savedProfile;  
  try {  
    savedProfile = UserProfile.fromJson(persisted);  
  } catch (_) {  
    return;  
  }  
}
```

#controllers/password_controller

I improved PasswordController robustness and performance.

Changes:

- ♦ `isChecked` is set before async connectivity work.
- ♦ repeated breach-check taps are ignored reliably.
- ♦ password analysis is memoized.

- ♦ repeated calls with the same password do not recalculate entropy.
- ♦ connectivity check can be injected for deterministic tests.

```

PasswordController{
    ApiService? apiService,
    Future<List<ConnectivityResult>> Function()? checkConnectivity,
}) : _apiService = apiService ?? Get.find<ApiService>(),
    _checkConnectivity =
        checkConnectivity ?? (() => Connectivity().checkConnectivity());

void analyzePassword(String value) {
    if (value == _lastAnalyzedPassword) return;

    _lastAnalyzedPassword = value;
    password.value = value;
    strengthResult.value = PasswordAnalyzer.evaluate(value);
    isBreachChecked.value = false;
    breachResult.value = null;
    errorMessage.value = '';
}

```

```

Future<void> checkBreach() async {
    final currentPassword = password.value;
    if (currentPassword.isEmpty || isChecking.value) {
        return;
    }

    isChecking.value = true;
    errorMessage.value = '';
    breachResult.value = null;
    isBreachChecked.value = false;

    try {
        final connectivityResult = await _checkConnectivity();
        if (connectivityResult.contains(ConnectivityResult.none)) {
            errorMessage.value = _offlineBreachMessage;
            return;
        }

        final checkResult = await _apiService.checkPassword(currentPassword);
        if (_apiService.errorMessage.isNotEmpty) {
            errorMessage.value = _isNetworkError(_apiService.errorMessage)
                ? _offlineBreachMessage
                : _apiService.errorMessage;
            return;
        }
        breachResult.value = checkResult;
        isBreachChecked.value = true;
    } catch (_) {
        final connectivityResult = await _checkConnectivity();
        errorMessage.value = connectivityResult.contains(ConnectivityResult.none)
            ? _offlineBreachMessage
            : 'Could not connect to breach database. Please try again.';
    } finally {
        isChecking.value = false;
    }
}

```

#views/auth/register_view

I cleaned up Register password strength handling.

Before:

- ♦ Register had duplicated password strength logic.

Now:

- ♦ Register uses PasswordAnalyzer.evaluate().

- ◆ Register uses PasswordStrengthResult directly.
- ◆ Register and Password Shield now agree on strength labels, colors, bars, and border behavior.

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import '../controllers/auth_controller.dart';
import '../routes/app_routes.dart';
import '../utils/constants.dart';
import '../utils/validators.dart';
import '../utils/password_analyzer.dart';
```

```
const _AuthLabel(label: 'Password'),
const SizedBox(height: 8),
Obx(() => _AuthTextField(
  controller: passwordController,
  hintText: '.....',
  obscureText: obscurePassword.value,
  onChanged: (val) {
    strength.value = PasswordAnalyzer.evaluate(
      val); // ← PasswordAnalyzer
    confirmMatches.value =
      val == confirmController.text && val.isNotEmpty;
  },
  // ← single field from result, no more ternary chain
  borderColor:
    strength.value.level != PasswordStrength.empty
      ? strength.value.borderColor
      : null,
  suffixIcon: GestureDetector(
    onTap: () =>
      obscurePassword.value = !obscurePassword.value,
    child: Icon(
      obscurePassword.value
        ? Icons.visibility_outlined
        : Icons.visibility_off_outlined,
      color: AppColors.textMuted,
      size: 20,
    ),
  ),
```

#utils/password_analyzer

I adjusted PasswordAnalyzer scoring for quality

Changes:

- ◆ Very short passwords are capped below Strong.
- ◆ Passwords with no uppercase letters cannot become Very Strong.

This keeps the strength label aligned with the improvement tips.

#pubspec.yaml

I added test dependencies.

I added: mockito , build_runner
flutter_test already exist.

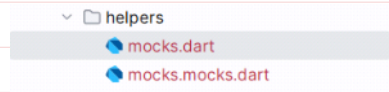
```
dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0
  mockito: ^5.4.0
  build_runner: ^2.4.0
```

#Mockito Mocks

I added a mock source file: test/helpers/mocks.dart

Then I ran build_runner to generate: test/helpers/mocks.mocks.dart

These mocks are used for controller and service tests



#Test Helper File

I created: test/helpers/test_helpers.dart

It contains:

- ♦ setupGetX()
- ♦ makeThreat()

setupGetX:

- ♦ enables Get.testMode
- ♦ resets GetX dependency injection after each test

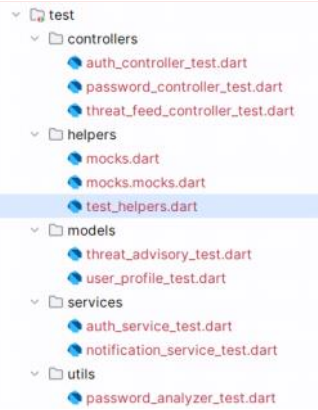
makeThreat:

- ♦ creates a minimal ThreatAdvisory fixture for tests

```
void setupGetX() {  
  setUp(() => Get.testMode = true);  
  tearDown(() => Get.reset());  
}  
  
ThreatAdvisory makeThreat({  
  String id = 'CVE-2024-0001',  
  String severity = 'CRITICAL',  
  double baseScore = 9.8,  
  String description = 'Test advisory',  
  DateTime? publishedDate,  
}) {  
  return ThreatAdvisory(  
    id: id,  
    severity: severity,  
    baseScore: baseScore,  
    description: description,  
    publishedDate: publishedDate ?? DateTime(2024, 3, 15),  
    lastModifiedDate: DateTime(2024, 3, 16),  
    referenceUrls: const [],  
  );  
}
```

Unit Test Added

I added full tests under the test folder:



#utils/password_analyzer_test

This file tests PasswordAnalyzer.evaluate().

Covered behavior:

- ♦ empty password
- ♦ weak password
- ♦ dictionary word detection
- ♦ numeric-only password
- ♦ short complex password
- ♦ strong password
- ♦ very strong password
- ♦ entropy greater than zero
- ♦ entropy increases with charset size
- ♦ four strength bars
- ♦ no dictionary word for random-looking password
- ♦ embedded dictionary word detection
- ♦ improvement tips
- ♦ hasPassword getter
- ♦ isVeryStrong getter

```

group('PasswordAnalyzer.evaluate', () {
  test('empty string returns empty result', () {
    final result = PasswordAnalyzer.evaluate('');

    expect(result.level, PasswordStrength.empty);
    expect(result.label, '');
    expect(result.entropyBits, 0);
  });

  test('single char is weak', () {
    expect(PasswordAnalyzer.evaluate('a').level, PasswordStrength.weak);
  });

  test('password detects dictionary word and is not strong', () {
    final result = PasswordAnalyzer.evaluate('password');

    expect(result.hasDictionaryWord, isTrue);
    expect(
      result.level,
      anyOf(PasswordStrength.weak, PasswordStrength.fair),
    );
  });

  test('123456 detects numbers only and is weak', () {
    final result = PasswordAnalyzer.evaluate('123456');

    expect(result.hasNumbers, isTrue);
    expect(result.hasUppercase, isFalse);
  });

  test('Hello! is fair because it is short', () {
    expect(PasswordAnalyzer.evaluate('Hello!').level, PasswordStrength.fair);
  });

  test('Hello123! is strong', () {
    expect(
      PasswordAnalyzer.evaluate('Hello123!').level,
      PasswordStrength.strong,
    );
  });

  test('complex long password is very strong', () {
    expect(
      PasswordAnalyzer.evaluate('C0mpl3x!Pass#99').level,
      PasswordStrength.veryStrong,
    );
  });

  test('entropyBits is positive for non-empty passwords', () {
    expect(PasswordAnalyzer.evaluate('a').entropyBits, greaterThan(0));
  });

  test('entropyBits increases as charset size grows', () {
    final lower = PasswordAnalyzer.evaluate('aaaaaaaa');
    final mixed = PasswordAnalyzer.evaluate('Aaaaaaaaa');

    expect(mixed.entropyBits, greaterThan(lower.entropyBits));
  });
});

```

```

test('bars length is always four', () {
  for (final value in ['', 'a', 'password', 'Hello123!', 'Xk9!mQ7@pL2#']) {
    expect(PasswordAnalyzer.evaluate(value).bars.length, 4);
  }
});

test('random-looking password has no dictionary word', () {
  expect(
    PasswordAnalyzer.evaluate('Xk9!mQ7@pL2#').hasDictionaryWord,
    isFalse,
  );
});

test('embedded common password is detected', () {
  expect(
    PasswordAnalyzer.evaluate('mypassword123').hasDictionaryWord,
    isTrue,
  );
});

test('improvementTip is null for very strong passwords', () {
  expect(PasswordAnalyzer.evaluate('C0mpl3x!Pass#99').improvementTip, null);
});

test('improvementTip suggests length when length is under 12', () {
  expect(PasswordAnalyzer.evaluate('Hello123!').improvementTip,
    'Use at least 12 characters');
});

```

```

test('improvementTip suggests uppercase when missing', () {
  expect(
    PasswordAnalyzer.evaluate('lowercase123!').improvementTip,
    'Add uppercase letters to increase entropy',
  );
});

test('hasPassword is false only for empty', () {
  expect(PasswordAnalyzer.evaluate('').hasPassword, isFalse);
  expect(PasswordAnalyzer.evaluate('a').hasPassword, isTrue);
});

test('isVeryStrong is true only for very strong', () {
  expect(PasswordAnalyzer.evaluate('C0mpl3x!Pass#99').isVeryStrong, isTrue);
  expect(PasswordAnalyzer.evaluate('Hello123!').isVeryStrong, isFalse);
});
}

```

#test/controllers/auth_controller_test

This file tests AuthController using a mocked AuthService

Covered behavior:

- ◆ current user loaded on init
- ◆ login loading state
- ◆ login success
- ◆ mapped FirebaseAuthException messages
- ◆ register errors
- ◆ Google sign-in cancellation
- ◆ reset password success and failure
- ◆ logout behavior
- ◆ errorMessage clearing at method start

I will put some of the tests, not all.

```
group('AuthController', () {  
  test('onInit sets currentUser from authService.currentUser', () {  
    when(authService.currentUser).thenReturn(user);  
  
    final controller = buildController();  
  
    expect(controller.currentUser.value, user);  
  });  
  
  test('login sets isLoading true then false', () async {  
    final completer = Completer<UserCredential>();  
    when(authService.signInWithEmail(any, any))  
      .thenAnswer((_) => completer.future);  
    final controller = buildController();  
  
    final future = controller.login('user@example.com', 'password');  
  
    expect(controller.isLoading.value, isTrue);  
    completer.complete(credential);  
    await future;  
    expect(controller.isLoading.value, isFalse);  
  });  
  
  test('login on success sets currentUser', () async {  
    when(authService.signInWithEmail(any, any))  
      .thenAnswer((_) async => credential);  
    final controller = buildController();
```

```
    test('errorMessage is cleared at the start of key methods', () async {  
      final loginCompleter = Completer<UserCredential>();  
      when(authService.signInWithEmail(any, any))  
        .thenAnswer((_) => loginCompleter.future);  
      final controller = buildController().errorMessage.value = 'stale';  
      final loginFuture = controller.login('user@example.com', 'password');  
      expect(controller.errorMessage.value, '');  
      loginCompleter.complete(credential);  
      await loginFuture;  
  
      final registerCompleter = Completer<UserCredential>();  
      when(authService.signUpWithEmail(any, any))  
        .thenAnswer((_) => registerCompleter.future);  
      controller.errorMessage.value = 'stale';  
      final registerFuture =  
        controller.register('user@example.com', 'password');  
      expect(controller.errorMessage.value, '');  
      registerCompleter.complete(credential);  
      await registerFuture;  
  
      final resetCompleter = Completer<void>();  
      when(authService.sendPasswordResetEmail(any))  
        .thenAnswer((_) => resetCompleter.future);  
      controller.errorMessage.value = 'stale';  
      final resetFuture = controller.resetPassword('user@example.com');  
      expect(controller.errorMessage.value, '');  
      resetCompleter.complete();  
    });  
  });  
});
```

```

test('register maps email-already-in-use error', () async {
  when(authService.signInWithEmail(any, any)).thenThrow(
    FirebaseAuthException(code: 'email-already-in-use'),
  );
  final controller = buildController();

  await controller.register('used@example.com', 'password');

  expect(
    controller.errorMessage.value,
    'An account with this email already exists.',
  );
});

test('register maps weak-password error', () async {
  when(authService.signInWithEmail(any, any)).thenThrow(
    FirebaseAuthException(code: 'weak-password'),
  );
  final controller = buildController();

  await controller.register('user@example.com', '123');

  expect(
    controller.errorMessage.value,
    'Password is too weak. Use at least 6 characters.',
  );
});

```

#test/controllers/password_controller_test

This file tests PasswordController using a mocked ApiService

Covered behavior:

- ◆ analyzePassword updates state
- ◆ breach result resets after password changes
- ◆ clearAll resets all state
- ◆ empty password does not check breach
- ◆ repeated check is ignored
- ◆ success stores breach result
- ◆ ApiService error message is shown
- ◆ offline connectivity message is shown
- ◆ isChecking resets in finally

Part of the tests:

```

group('PasswordController', () {
  test('analyzePassword empty returns empty and resets breach state', () {
    final controller = buildController()
    ..breachResult.value = const PasswordCheckResult(
      isPwned: true,
      breachCount: 1,
    )
    ..isBreachChecked.value = true;

    controller.analyzePassword('x');
    controller.analyzePassword('');

    expect(controller.strengthResult.value.level, PasswordStrength.empty);
    expect(controller.isBreachChecked.value, isFalse);
  });

  test('analyzePassword non-empty updates strength', () {
    final controller = buildController();

    controller.analyzePassword('Hello123!');

    expect(
      controller.strengthResult.value.level, isNot(PasswordStrength.empty));
  });

  test('analyzePassword sets password value', () {
    final controller = buildController();

```

```

    test('analyzePassword sets password value', () {
      final controller = buildController();

      controller.analyzePassword('Hello123!');

      expect(controller.password.value, 'Hello123!');
    });

    test('analyzePassword resets breach result and checked flag', () {
      final controller = buildController()
      ..breachResult.value = const PasswordCheckResult(
        isPwned: true,
        breachCount: 10,
      )
      ..isBreachChecked.value = true;

      controller.analyzePassword('new-password');

      expect(controller.breachResult.value, isNull);
      expect(controller.isBreachChecked.value, isFalse);
    });

    test('clearAll resets all observables to initial state', () {
      final controller = buildController()
      ..analyzePassword('Hello123!')
      ..breachResult.value = const PasswordCheckResult(
        isPwned: true,
        breachCount: 10,
      )
    }

```



```

test('checkBreach uses ApiService error message', () async {
  when(apiService.checkPassword(any)).thenAnswer(
    (_) async => const PasswordCheckResult(isPwned: false, breachCount: 0),
  );
  when(apiService.errorMessage).thenReturn('HIBP unavailable');
  final controller = buildController()..analyzePassword('Hello123!');

  await controller.checkBreach();

  expect(controller.errorMessage.value, 'HIBP unavailable');
});

test('checkBreach when connection is none sets offline message', () async {
  final controller = buildController(
    checkConnectivity: () async => const [ConnectivityResult.none],
  )..analyzePassword('Hello123!');

  await controller.checkBreach();

  expect(controller.errorMessage.value, contains('No internet connection'));
  verifyNever(apiService.checkPassword(any));
});

test('checkBreach always resets isChecking in finally', () async {
  final completer = Completer<PasswordCheckResult>();
  when(apiService.checkPassword(any)).thenAnswer((_) => completer.future);
  final controller = buildController()..analyzePassword('Hello123!');

```

#test/controllers/threat_feed_controller_test

This file tests ThreatFeedController filtering behavior

Covered behavior:

- ◆ fetchThreats populates list
- ◆ severity filters
- ◆ search filters
- ◆ clearFilters
- ◆ newest-first sorting
- ◆ watchlist mode default
- ◆ Watchlist toggle
- ◆ watchlist keyword filtering
- ◆ critical alert preference does not hide critical feed items
- ◆ empty watchlist behavior
- ◆ watchlist mode defaults to true.
- ◆ controller supports injected ProfileController provider for deterministic tests.

```

setUp() {
  apiService = MockApiService();
  fixtureThreats = [
    makeThreat(
      id: 'CVE-2024-0001',
      severity: 'CRITICAL',
      baseScore: 9.8,
      description: 'Apache RCE vulnerability',
      publishedDate: DateTime(2024, 3, 15),
    ),
    makeThreat(
      id: 'CVE-2024-0002',
      severity: 'HIGH',
      baseScore: 7.5,
      description: 'Flutter heap overflow',
      publishedDate: DateTime(2024, 3, 16),
    ),
    makeThreat(
      id: 'CVE-2024-0003',
      severity: 'MEDIUM',
      baseScore: 5.0,
      description: 'Android auth bypass',
      publishedDate: DateTime(2024, 3, 17),
    ),
    makeThreat(
      id: 'CVE-2024-0004',
      severity: 'LOW',
      baseScore: 2.0,

```

```

test('filterBySeverity CRITICAL keeps only CRITICAL advisories', () async {
  final controller = buildController();
  await controller.fetchThreats(refresh: true);

  await controller.filterBySeverity('CRITICAL');

  expect(controller.threats.map((threat) => threat.id), [
    'CVE-2024-0005',
    'CVE-2024-0001',
  ]);
});

test('filterBySeverity ALL clears severity filter and shows all', () async {
  final controller = buildController();
  await controller.fetchThreats(refresh: true);

  await controller.filterBySeverity('ALL');

  expect(controller.selectedSeverity.value, '');
  expect(controller.threats.length, 5);
});

test('filterBySeverity HIGH keeps only HIGH advisories', () async {
  final controller = buildController();
  await controller.fetchThreats(refresh: true);

  await controller.filterBySeverity('HIGH');

  expect(controller.threats.single.id, 'CVE-2024-0002');
});

```

```

test('search apache returns only CVE-2024-0001', () async {
  final controller = buildController();
  await controller.fetchThreats(refresh: true);

  await controller.search('apache');

  expect(controller.threats.single.id, 'CVE-2024-0001');
});

test('search flutter returns only CVE-2024-0002', () async {
  final controller = buildController();
  await controller.fetchThreats(refresh: true);

  await controller.search('flutter');

  expect(controller.threats.single.id, 'CVE-2024-0002');
});

test('search empty clears keyword filter and shows all', () async {
  final controller = buildController();
  await controller.fetchThreats(refresh: true);
  await controller.search('apache');

  await controller.search('');

  expect(controller.searchKeyword.value, '');
  expect(controller.threats.length, 5);
});

```

```

test('isWatchlistModeActive defaults to true', () {
  final controller = buildController();

  expect(controller.isWatchlistModeActive.value, isTrue);
});

test('toggleWatchlistMode flips isWatchlistModeActive', () async {
  final controller = buildController();

  await controller.toggleWatchlistMode();

  expect(controller.isWatchlistModeActive.value, isFalse);
});

test('watchlist mode filters by watchlist keywords', () async {
  final profile = MockProfileController();
  when(profile.watchlist).thenReturn(['Apache', 'Flutter'].obs);
  when(profile.criticalAlertsEnabled).thenReturn(true.obs);
  final controller = buildController(profileController: profile);

  await controller.fetchThreats(refresh: true);

  expect(controller.threats.map((threat) => threat.id), [
    'CVE-2024-0002',
    'CVE-2024-0001',
  ]);
});

```

#test/models/threat_advisory_test

This file tests ThreatAdvisory parsing

Covered behavior:

- ♦ well-formed NVD JSON parsing
- ♦ missing description fallback
- ♦ CVSS v3.1 extraction
- ♦ CVSS v3.0 extraction

- ♦ CVSS v2 extraction
- ♦ unknown severity normalization
- ♦ date parsing
- ♦ epoch fallback for invalid dates
- ♦ reference URL extraction
- ♦ cache round trip
- ♦ severity color mapping

```
void main() {
  final sampleNvdJson = {
    'id': 'CVE-2024-12345',
    'published': '2024-03-15T10:00:00.000',
    'lastModified': '2024-03-16T08:00:00.000',
    'descriptions': [
      {'lang': 'en', 'value': 'A critical buffer overflow in ExampleLib 1.2.'},
    ],
    'metrics': {
      'cvssMetricV31': [
        {
          'cvssData': {
            'baseScore': 9.8,
            'baseSeverity': 'CRITICAL',
          },
        },
      ],
    },
    'references': [
      {'url': 'https://nvd.nist.gov/vuln/detail/CVE-2024-12345'},
    ],
  };
}
```

```
group('ThreatAdvisory.fromJson', () {
  test('parses a well-formed NVD CVE JSON blob', () {
    final advisory = ThreatAdvisory.fromJson(sampleNvdJson);

    expect(advisory.id, 'CVE-2024-12345');
    expect(advisory.description, contains('ExampleLib'));
    expect(advisory.baseScore, 9.8);
    expect(advisory.severity, 'CRITICAL');
  });

  test('falls back when descriptions list is empty', () {
    final advisory = ThreatAdvisory.fromJson({
      ...sampleNvdJson,
      'descriptions': <Map<String, dynamic>>[],
    });

    expect(advisory.description, contains('No description'));
  });

  test('extracts severity from cvssMetricV31 first', () {
    expect(ThreatAdvisory.fromJson(sampleNvdJson).severity, 'CRITICAL');
  });
});
```

```

test('extracts severity from cvssMetricV2 when V3 is absent', () {
  final advisory = ThreatAdvisory.fromJson({
    ...sampleNvdJson,
    'metrics': {
      'cvssMetricV2': [
        {
          'cvssData': {'baseScore': 5.0},
          'baseSeverity': 'MEDIUM',
        },
      ],
    },
  });

  expect(advisory.severity, 'MEDIUM');
  expect(advisory.baseScore, 5.0);
});

test('normalizes unknown severity string to UNKNOWN', () {
  final advisory = ThreatAdvisory.fromJson({
    ...sampleNvdJson,
    'metrics': {
      'cvssMetricV31': [
        {
          'cvssData': {
            'baseScore': 1.0,
            'baseSeverity': 'ODD',
          },
        },
      ],
    },
  });

```

#test/models/user_profile_test

This file tests UserProfile serialization

Covered behavior:

- ♦ toJson/fromJson round trip
- ♦ default toggles are true
- ♦ empty watchlist items are filtered
- ♦ null watchlist becomes empty list
- ♦ empty JSON map does not crash

```

void main() {
  group('UserProfile serialization', () {
    test('toJson and fromJson round-trip preserves all fields', () {
      const profile = UserProfile(
        uid: 'u1',
        email: 'user@example.com',
        displayName: 'Cyber User',
        watchList: ['Flutter', 'Firebase'],
        criticalAlertsEnabled: false,
        quietHoursEnabled: false,
        allNotificationsEnabled: false,
      );

      final roundTrip = UserProfile.fromJson(profile.toJson());

      expect(roundTrip.uid, profile.uid);
      expect(roundTrip.email, profile.email);
      expect(roundTrip.displayName, profile.displayName);
      expect(roundTrip.watchList, profile.watchList);
      expect(roundTrip.criticalAlertsEnabled, isFalse);
      expect(roundTrip.quietHoursEnabled, isFalse);
      expect(roundTrip.allNotificationsEnabled, isFalse);
    });
  });
}

```

```

test('fromJson defaults criticalAlertsEnabled to true when missing', () {
  expect(UserProfile.fromJson(const {})).criticalAlertsEnabled, isTrue);
});

test('fromJson defaults quietHoursEnabled to true when missing', () {
  expect(UserProfile.fromJson(const {})).quietHoursEnabled, isTrue);
});

test('fromJson defaults allNotificationsEnabled to true when missing', () {
  expect(UserProfile.fromJson(const {})).allNotificationsEnabled, isTrue);
});

test('fromJson filters empty strings from watchlist', () {
  final profile = UserProfile.fromJson({
    'watchlist': ['Flutter', '', 'Firebase'],
  });

  expect(profile.watchlist, ['Flutter', 'Firebase']);
});

test('fromJson handles null watchlist field as empty list', () {
  expect(
    UserProfile.fromJson(const {'watchlist': null}).watchlist, isEmpty);
});

test('fromJson handles entirely empty JSON map without crashing', () {
  final profile = UserProfile.fromJson(const {});

```

#test/services/auth_service_test

This file tests AuthService using mocked FirebaseAuth, GoogleSignIn, and FlutterSecureStorage.

Covered behavior:

- ◆ sign up writes login flag
- ◆ sign up rethrows FirebaseAuthException
- ◆ sign in writes login flag
- ◆ sign in rethrows original error code
- ◆ sign out calls GoogleSignIn, FirebaseAuth, and secure storage delete
- ◆ sign out rethrows FirebaseAuthException
- ◆ password reset calls FirebaseAuth
- ◆ Google sign-in cancellation throws google-sign-in-aborted

```

group('AuthService', () {
  test('signUpWithEmail writes logged-in flag on success', () async {
    when(
      firebaseAuth.createUserWithEmailAndPassword(
        email: anyNamed('email'),
        password: anyNamed('password'),
      ),
    ).thenAnswer((_) async => credential);
    when(storage.write(key: anyNamed('key'), value: anyNamed('value')))
      .thenAnswer((_) async {});

    final result = await service.signUpWithEmail('user@example.com', 'pass');

    expect(result, credential);
    verify(storage.write(key: 'is_logged_in', value: 'true')).called(1);
  });

  test('signUpWithEmail rethrows FirebaseAuthException on failure', () async {
    when(
      firebaseAuth.createUserWithEmailAndPassword(
        email: anyNamed('email'),
        password: anyNamed('password'),
      ),
    ).thenThrow(FirebaseAuthException(code: 'email-already-in-use'));

```

```

expect(
  service.signInWithEmail('user@example.com', 'pass'),
  throwsA(isA<FirebaseAuthException>()),
);
});

test('signInWithEmail writes logged-in flag on success', () async {
  when(
    firebaseAuth.signInWithEmailAndPassword(
      email: anyNamed('email'),
      password: anyNamed('password'),
    ),
  ).thenAnswer((_) async => credential);
  when(storage.write(key: anyNamed('key'), value: anyNamed('value')))
    .thenAnswer((_) async {});

  final result = await service.signInWithEmail('user@example.com', 'pass');

  expect(result, credential);
  verify(storage.write(key: 'is_logged_in', value: 'true')).called(1);
});

test('signInWithEmail rethrows FirebaseAuthException with original code',
  () async {
    when(
      firebaseAuth.signInWithEmailAndPassword(
        email: anyNamed('email'),
        password: anyNamed('password'),
      ),
    )

```

```

test('signOut signs out providers and deletes secure storage key',
  () async {
    when(googleSignIn.signOut()).thenAnswer((_) async => null);
    when(firebaseAuth.signOut()).thenAnswer((_) async {});
    when(storage.delete(key: anyNamed('key'))).thenAnswer((_) async {});

    await service.signOut();

    verify(googleSignIn.signOut()).called(1);
    verify(firebaseAuth.signOut()).called(1);
    verify(storage.delete(key: 'is_logged_in')).called(1);
  });

test('signOut rethrows FirebaseAuthException on failure', () async {
  when(googleSignIn.signOut()).thenAnswer((_) async => null);
  when(firebaseAuth.signOut()).thenThrow(
    FirebaseAuthException(code: 'network-request-failed'),
  );

  await expectLater(
    service.signOut(),
    throwsA(
      isA<FirebaseAuthException>().having(
        (error) => error.code,
        'code',
        'network-request-failed',
      ),
    ),
  );
}

```

#test/services/notification_service_test

This file tests NotificationService preference filtering

Covered behavior:

- ◆ Critical alerts OFF allows both critical and non-critical alerts
- ◆ Critical alerts ON allows only Critical, High, or baseScore >= 9.0 alerts
- ◆ All notifications OFF still blocks every alert

```

void main() {
  group('NotificationService preferences', () {
    AppNotification notification({
      String severity = 'MEDIUM',
      double? baseScore,
    }) {
      return AppNotification(
        id: 'notification-$severity-${baseScore ?? 'none'}',
        title: 'Alert',
        body: 'Body',
        severity: severity,
        baseScore: baseScore,
        timestamp: DateTime(2026, 5, 28, 12),
      );
    }

    test('critical alerts off allows both critical and non-critical alerts',
      () {
        final medium = notification(severity: 'MEDIUM', baseScore: 5.0);
        final critical = notification(severity: 'CRITICAL', baseScore: 9.8);

        expect(
          NotificationService.shouldDisplayForPreferences(
            medium,
            allNotificationsEnabled: true,
            criticalAlertsEnabled: false,
            quietHoursEnabled: false,
          ),
        );
      }
    );
  });
}

```

```

test('critical alerts on allows only critical or high alerts', () {
  final medium = notification(severity: 'MEDIUM', baseScore: 5.0);
  final high = notification(severity: 'HIGH', baseScore: 7.5);

  expect(
    NotificationService.shouldDisplayForPreferences(
      medium,
      allNotificationsEnabled: true,
      criticalAlertsEnabled: true,
      quietHoursEnabled: false,
    ),
    isFalse,
  );
  expect(
    NotificationService.shouldDisplayForPreferences(
      high,
      allNotificationsEnabled: true,
      criticalAlertsEnabled: true,
      quietHoursEnabled: false,
    ),
    isTrue,
  );
});

```

Manual Testing and Screenshots:

Notifications:

Manual result:

- ◆ All notifications OFF worked correctly.
- ◆ No foreground/in-app alert was saved or shown.

Issue found:

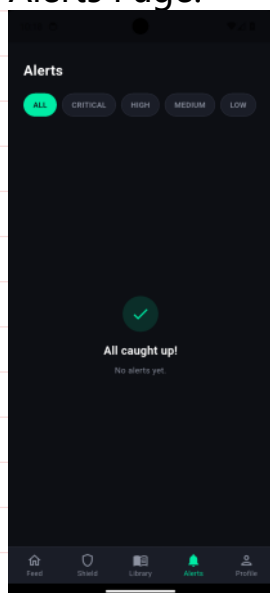
- ◆ Critical alerts ON still allowed Low alerts.
- ◆ Critical alerts OFF hid Critical and High alerts instead of allowing all Firebase alerts.

- ♦ The same preference also hid Critical CVEs from the threat feed.

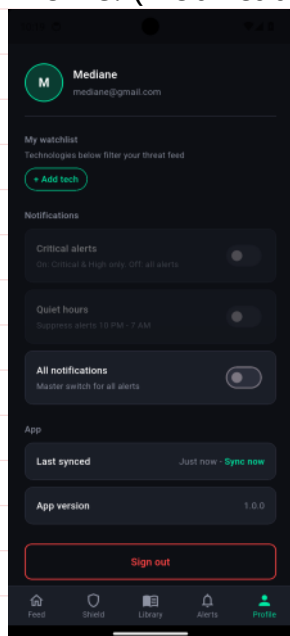
Fix:

- ♦ Critical alerts ON now allows only Critical and High alerts.
- ♦ Low, Medium, and Unknown alerts are suppressed.
- ♦ Critical alerts OFF now allows all severities when All notifications is ON.
- ♦ Critical alerts no longer affects threat feed CVE visibility.

Alerts Page:



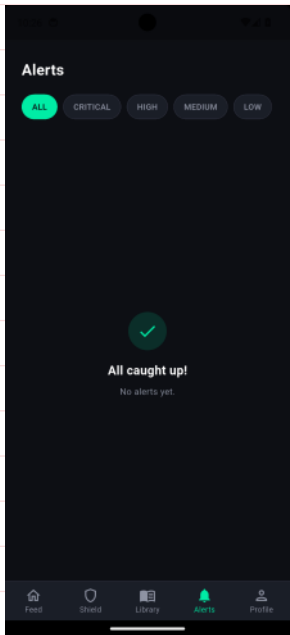
Profile: (Notification Off => critical alerts and quiet hours are automatically Off)



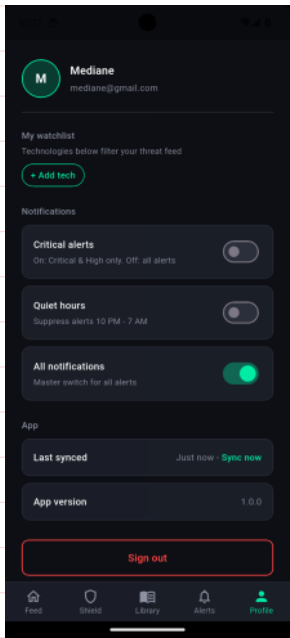
I will send 2 alerts (high and low):

Campaign	Start	End	Status	Target	Last updated	↓	Sends / Impressions	Clicks / Opens
Low Alert VE-2026-4444 CVSS 4.3 - patch now	30 May 2026 10:21:50	-	Active	cybershield_alerts	30 May 2026		<1,000	0%
Critical Alert VE-2026-4444 CVSS 9.5 - patch now	30 May 2026 10:21:36	-	Active	cybershield_alerts	30 May 2026		<1,000	0%

No Alerts:

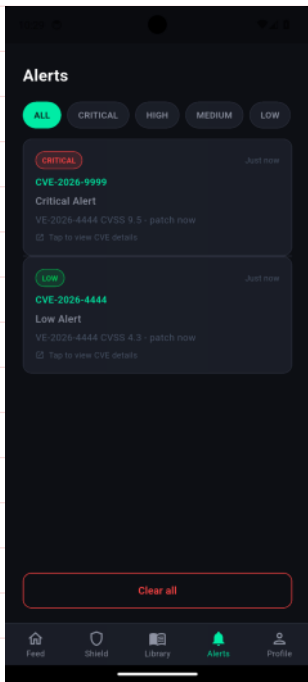


Notifications ON:

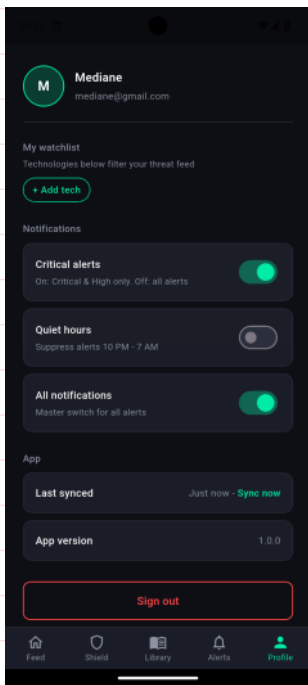


2 new alerts:

Campaign	Start	End	Status	Target	Last updated	↓	Sends / Impressions	Clicks / Opens
Low Alert VE-2026-4444 CVSS 4.3 - patch now	30 May 2026 10:28:41	-	Active	cybershield_alerts	30 May 2026		--	--
Critical Alert VE-2026-4444 CVSS 9.5 - patch now	30 May 2026 10:28:27	-	Active	cybershield_alerts	30 May 2026		--	--



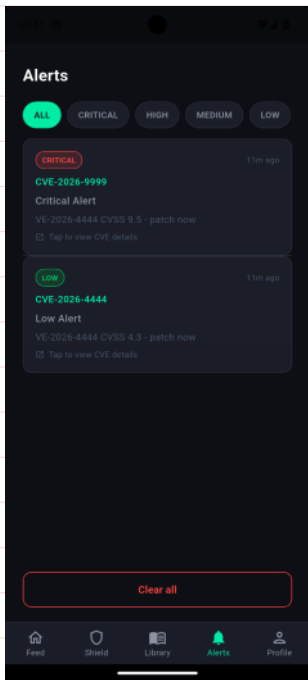
Critical Alerts:



Sending Medium Alert:

	Campaign	Start	End	Status	Target	Last updated	↓	Sends / Impressions	Clicks / Opens
	Medium Alert VE-2026-4444 CVSS 8.5 - patch now	30 May 2026 10:35:37	-	Active	cybershield_alerts	30 May 2026		--	--
	Critical Alert VE-2026-4444 CVSS 9.5 - patch now	30 May 2026 10:28:27	-	Completed	cybershield_alerts	30 May 2026		--	--

After few minutes: (No new alert)



Critical Alert:

	Campaign	Start	End	Status	Target	Last updated	↓	Sends / Impressions	Clicks / Opens
	Critical Alert VE-2026-4444 CVSS 9.5 - patch now	30 May 2026 10:42:00	-	Active	cybershield_alerts	30 May 2026		<1,000	0%
	Medium Alert VE-2026-4444 CVSS 8.5 - patch now	30 May 2026 10:35:37	-	Completed	cybershield_alerts	30 May 2026		<1,000	0%

Alert:



Threat Feed Relation To Watchlist

Issue found:

- ♦ The app still felt like it was showing old repeated generic NVD results.

- ♦ Watchlist relation was not obvious.

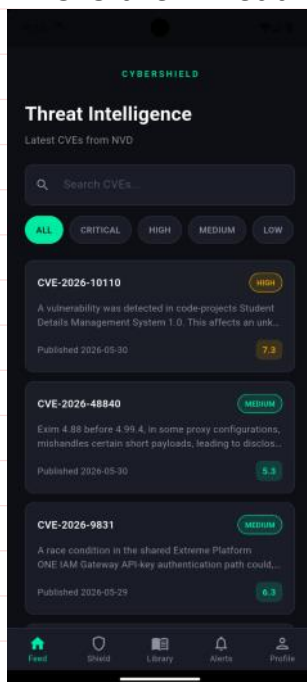
Cause:

- ♦ The first implementation filtered only the already-loaded generic NVD page.
- ♦ If the generic page did not include the user's technologies, the relation was weak.

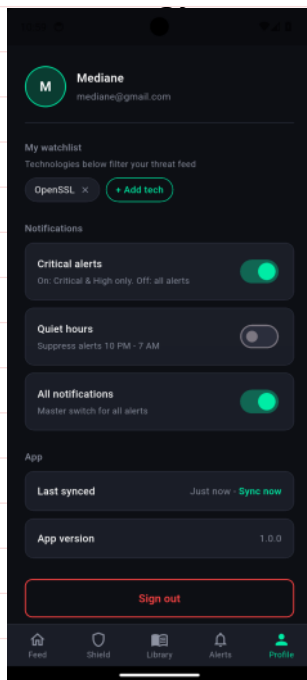
Fix:

- ♦ Watchlist mode now fetches NVD using watchlist keywords.
- ♦ Watchlist mode is active by default in the shell.
- ♦ WATCHLIST chip can disable it for generic feed browsing.

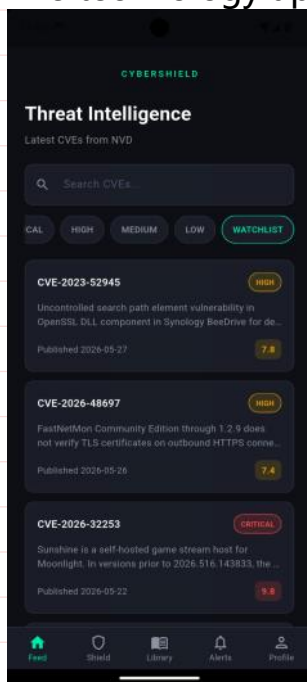
This is the Threat Feed:



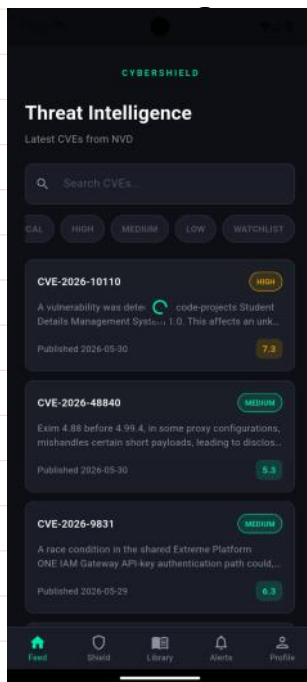
Technology Added:



The technology appears first in the threat feed:



After turning the WATCHLIST chip off:



Flutter Test:

```
rt: PasswordAnalyzer.evaluate hasPassword is false only for empty
00:06 +87: c:/Users/Administrator/Desktop/Internship-2025/Internship-Projects
rt: PasswordAnalyzer.evaluate isVeryStrong is true only for very strong
00:06 +88: All tests passed!
```

Follow-up Improvements

Follow-up improvements for Step 9: Testing & Optimzation

After Deliverable 9, I implemented these follow-up improvements :

- ♦ Use real NVD CPE matching instead of only keywordSearch.
- ♦ Move notification filtering to a backend or data-only FCM payloads for full background suppression.
- ♦ Add UI text explaining that specific watchlist product names give better CVE matches.
- ♦ Add integration tests for real emulator flows.

The goal of Step 9.2 was to make the threat feed more accurate, improve notification filtering for background delivery, explain watchlist behavior more clearly, and add a starting point for emulator integration testing

What I've Done:

- ♦ Added NVD CPE matching before keywordSearch.
- ♦ Added fallback to keywordSearch when no matching CPE is found.
- ♦ Added FCM topic syncing for notification preferences.
- ♦ Added clearer Profile text explaining that specific product names give better CVE matches.
- ♦ Added an emulator-oriented integration test for auth navigation.
- ♦ Added unit tests for CPE ranking and watchlist CPE fetching.
- ♦ Regenerated Mockito mocks after changing ApiService.

Why?

The old watchlist feed depended only on NVD keywordSearch.

That worked, but keywordSearch can be broad. For example, generic terms can match unrelated CVEs because NVD searches text, not exact affected products.

CPE matching is more precise because NVD CVEs are linked to affected platform names. Resolving a watchlist item to a CPE first helps the app fetch CVEs for the actual product when possible.

Notification filtering also needed a stronger background strategy. Client-side Dart filtering works for foreground and in-app handling, but background system notifications can appear before Flutter code runs. Topic syncing gives the Firebase sender a better contract:

- ♦ send general alerts to cybershield_alerts
- ♦ send critical-only alerts to cybershield_critical_alerts
- ♦ let the app subscribe to only the topics that match the user's preferences

HOW?

In ApiService:

- ♦ Added the NVD CPE endpoint.
- ♦ Added resolveCpeNames().
- ♦ Added rankedCpeNamesForKeyword() to choose non-deprecated matching CPEs.
- ♦ Added optional cpeName support to fetchThreats().
- ♦ Updated cache keys so CPE-based and keyword-based results do not collide.

```
Future<List<String>> resolveCpeNames(
    String keyword, {
    int limit = 1,
}) async {
    final trimmedKeyword = keyword.trim();
    if (trimmedKeyword.isEmpty || limit <= 0) return <String>[];
    final cacheKey = trimmedKeyword.toLowerCase();
    final cached = _cpeNameCache[cacheKey];
    if (cached != null) return cached.take(limit).toList();

    try {
        final response = await _dio.get<Map<String, dynamic>>(
            AppStrings.nvdCpeBaseUrl,
            queryParameters: {
                'keywordSearch': trimmedKeyword,
                'resultsPerPage': 20,
                'startIndex': 0,
            },
            options: Options(
                headers: {
                    'apiKey': AppStrings.nvdApiKey,
                },
            ),
        );

        final products =
            response.data?['products'] as List<dynamic>? ?? <dynamic>[];
        final matches = rankedCpeNamesForKeyword(products, trimmedKeyword);

        final matches = rankedCpeNamesForKeyword(products, trimmedKeyword);
        _cpeNameCache[cacheKey] = matches;
        return matches.take(limit).toList();
    } on DioException catch (_) {
        return <String>[];
    } catch (_) {
        return <String>[];
    }
}

@visibleForTesting
static List<String> rankedCpeNamesForKeyword(
    List<dynamic> products,
    String keyword,
) {
    final normalizedKeyword = _normalizeSearchText(keyword);
    if (normalizedKeyword.isEmpty) return <String>[];

    final scored = <ScoredCpeName>[];
    final seen = <String>{};

    for (final product in products) {
        if (product is! Map<String, dynamic>) continue;
        final cpe = product['cpe'];
        if (cpe is! Map<String, dynamic>) continue;
        if (cpe['deprecated'] == true) continue;

        final cpeName = cpe['cpeName']?.toString().trim() ?? '';
        if (cpeName.isEmpty || !canAdd(cpeName)) continue;
    }
}
```

```

if (cpeName.isEmpty || !seen.add(cpeName)) continue;

final titles = cpe["titles"] as List<dynamic>? ?? <dynamic>[];
final titleText = titles
    .map((title) {
        if (title is! Map<String, dynamic>) return '';
        return title["title"]?.toString() ?? '';
    })
    .where((title) => title.trim().isNotEmpty)
    .join(' ');

final normalizedTitle = _normalizeSearchText(titleText);
final normalizedCpe = _normalizeSearchText(cpeName);
var score = 0;

if (normalizedTitle == normalizedKeyword) score += 80;
if (normalizedTitle.contains(normalizedKeyword)) score += 45;
if (normalizedCpe.contains(normalizedKeyword)) score += 25;

if (score > 0) {
    if (cpeName.startsWith('cpe:2.3:a:')) score += 8;
    if (cpeName.startsWith('cpe:2.3:o:')) score += 6;
    scored.add(_ScoredCpeName(cpeName: cpeName, score: score));
}
}

scored.sort((a, b) {
    final scoreComparison = b.score.compareTo(a.score);
    if (scoreComparison != 0) return scoreComparison;
    return a.cpeName.length.compareTo(b.cpeName.length);
});

return scored.map((item) => item.cpeName).toList();
}

```

In ThreatFeedController:

- ◆ Watchlist mode now resolves each watchlist product to CPE names first.
- ◆ If CPEs are found, the controller fetches CVEs using cpeName.
- ◆ If no CPEs are found, it falls back to the previous keywordSearch behavior.
- ◆ Local text filtering is kept for keyword fallback, but CPE results are trusted as NVD product matches.

```

final combined = <ThreatAdvisory>[];
var hasMore = false;
for (final keyword in watchListKeywords) {
    final cpeNames = await _apiService.resolveCpeNames(keyword);
    final results = <ThreatAdvisory>[];

    if (cpeNames.isNotEmpty) {
        _lastRemoteUsedCpeMatches = true;
        for (final cpeName in cpeNames) {
            results.addAll(
                await _apiService.fetchThreats(
                    page: page,
                    cpeName: cpeName,
                    severity: severity,
                ),
            );
            hasMore = hasMore || _apiService.hasMoreThreatPages;
        }
    } else {
        results.addAll(
            await _apiService.fetchThreats(
                page: page,
                keyword: keyword,
                severity: severity,
            ),
        );
        hasMore = hasMore || _apiService.hasMoreThreatPages;
    }
}

```

In NotificationService and ProfileController:

- ◆ Added syncAlertTopicSubscriptions().
- ◆ All notifications OFF unsubscribes from both alert topics.
- ◆ Critical alerts ON subscribes to the critical-only topic.
- ◆ Critical alerts OFF subscribes to the general alerts topic.
- ◆ Profile preference changes now sync Firebase topics.

```
Future<void> syncAlertTopicSubscriptions({
  bool? allNotificationsEnabled,
  bool? criticalAlertsEnabled,
}) async {
  final profile = Get.isRegistered<ProfileController>()
    ? Get.find<ProfileController>()
    : null;
  final allEnabled =
    allNotificationsEnabled ?? profile?.allNotificationsEnabled.value;
  final criticalOnly =
    criticalAlertsEnabled ?? profile?.criticalAlertsEnabled.value;

  if (allEnabled == false) {
    await _unsubscribeFromTopic(AppStrings.fcmAllAlertsTopic);
    await _unsubscribeFromTopic(AppStrings.fcmCriticalAlertsTopic);
    return;
  }

  if (criticalOnly == true) {
    await _unsubscribeFromTopic(AppStrings.fcmAllAlertsTopic);
    await _subscribeToTopic(AppStrings.fcmCriticalAlertsTopic);
    return;
  }

  await _subscribeToTopic(AppStrings.fcmAllAlertsTopic);
  await _unsubscribeFromTopic(AppStrings.fcmCriticalAlertsTopic);
}
```

```
Future<void> _subscribeToTopic(String topic) async {
  await _setTopicSubscription(topic, shouldSubscribe: true);
}
```

```
Future<void> _unsubscribeFromTopic(String topic) async {
  await _setTopicSubscription(topic, shouldSubscribe: false);
}
```

```
Future<void> _setTopicSubscription(
  String topic, {
  required bool shouldSubscribe,
}) {
  final normalizedTopic = topic.trim();
  if (normalizedTopic.isEmpty) return Future<void>.value();

  if (_desiredTopicSubscriptions[normalizedTopic] == shouldSubscribe) {
    return Future<void>.value();
  }
  _desiredTopicSubscriptions[normalizedTopic] = shouldSubscribe;

  _topicQueue = _topicQueue.then(
    (_) => _applyLatestTopicSubscription(normalizedTopic),
  );
  return _topicQueue;
}
```

In ProfileView:

- ◆ Updated the watchlist subtitle to recommend specific products such as OpenSSL or Firebase.

```
void _populateFromCurrentUser() {
  final user = authController.currentUser.value ?? authService.currentUser;
  final email = user?.email ?? '';
  final displayName = (user?.displayName?.trim().isEmpty ?? false)
    ? user?.displayName?.trim()
    : _nameFromEmail(email);
  final initialWatchlist = <String>[
    'Kitty',
    'Firebase',
    'OpenSSL',
  ];
}
```

In tests:

- ◆ Added test/services/api_service_test.dart.
- ◆ Added a watchlist test proving CPE fetching is used before keyword search.
- ◆ Added integration_test/auth_navigation_flow_test.dart for emulator auth navigation smoke testing.

```
void main() {
  group('ApiService CPE ranking', () {
    test('returns matching non-deprecated CPE names in best-match order', () {
      final products = [
        {
          'cpe': {
            'deprecated': false,
            'cpeName': 'cpe:2.3:a:apache:http_server:*:*:*:*:*:*',
            'titles': [
              {'title': 'Apache HTTP Server'},
            ],
          },
        },
        {
          'cpe': {
            'deprecated': true,
            'cpeName': 'cpe:2.3:a:apache:old_http_server:*:*:*:*:*:*',
            'titles': [
              {'title': 'Apache HTTP Server'},
            ],
          },
        },
        {
          'cpe': {
            'deprecated': false,
            'cpeName': 'cpe:2.3:a:example:other:*:*:*:*:*:*',
            'titles': [
              {'title': 'Unrelated Product'},
            ],
          },
        },
      ];
    });
  });
}
```

```
testWidgets('auth screens navigate on an emulator', (tester) async {
  await tester.pumpWidget(const CyberShieldApp());
  await tester.pumpAndSettle();

  expect(find.text('CYBERSHIELD'), findsOneWidget);
  expect(find.text('Sign in'), findsWidgets);

  await tester.tap(find.text('Sign up'));
  await tester.pumpAndSettle();
  expect(find.text('Create account'), findsWidgets);

  await tester.tap(find.text('Sign in').last);
  await tester.pumpAndSettle();
  expect(find.text('Forgot password?'), findsOneWidget);

  await tester.tap(find.text('Forgot password?'));
  await tester.pumpAndSettle();
  expect(find.text('Reset your password'), findsOneWidget);

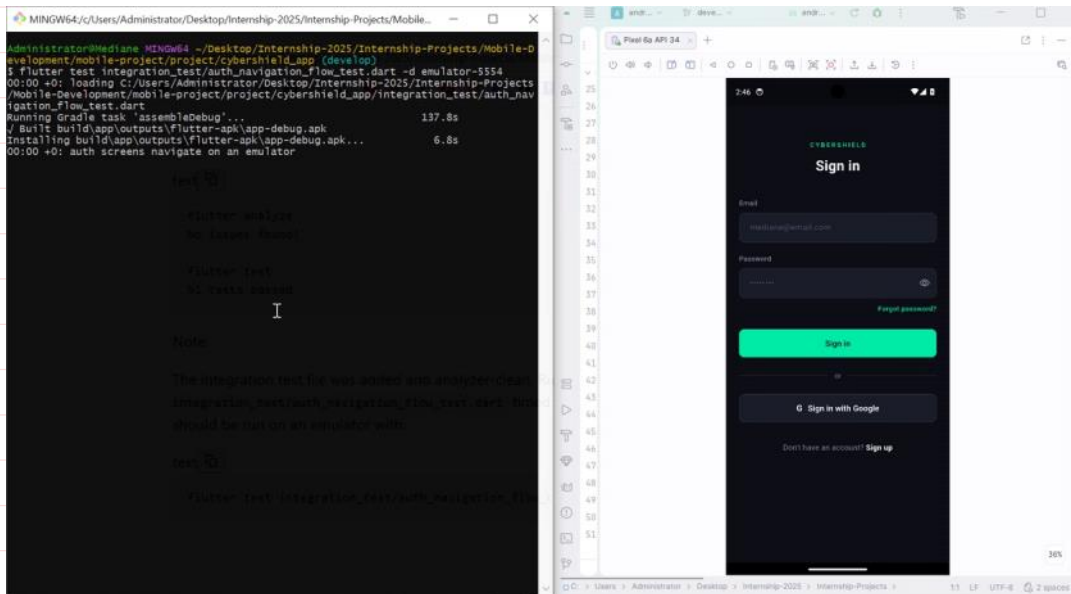
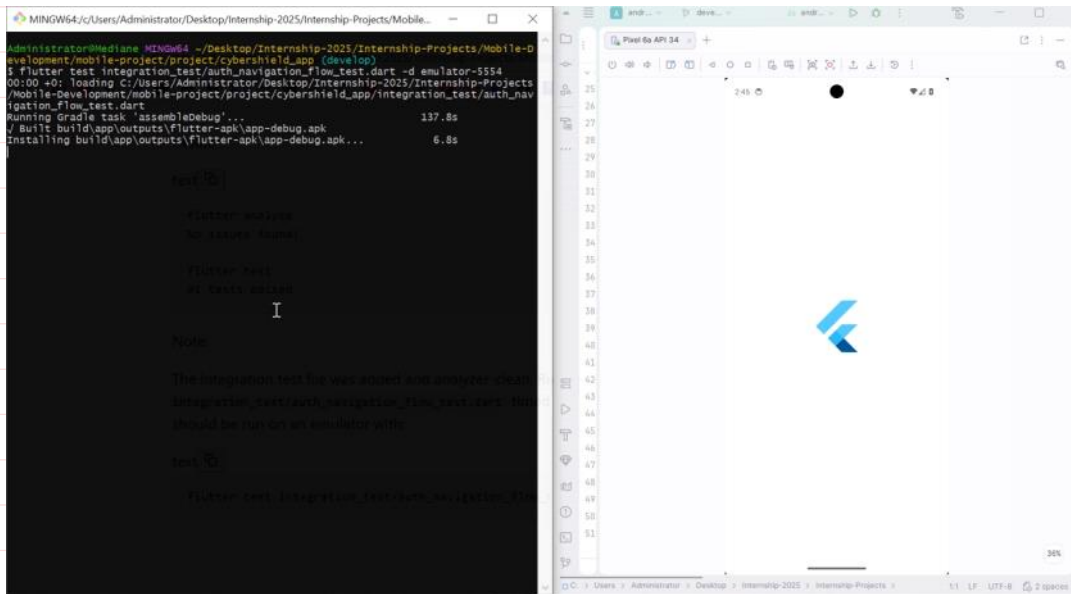
  await tester.tap(find.text('Back to login'));
  await tester.pumpAndSettle();
  expect(find.text('Sign in'), findsWidgets);
});
```

Flutter Test:

```
flutter test integration_test/auth_navigation_flow_test.dart: PasswordAnalyzer.
ong
00:04 +91: All tests passed!
```

Running:

flutter test integration_test/auth_navigation_flow_test.dart -d emulator-5554



2:46

CYBERSHIELD

Sign in

Email

mediana@email.com

Password

Forgot password?

Sign in

or

G Sign in with Google

Don't have an account? [Sign up](#)

CYBERSHIELD

Create account

Full name

Mediana Ozer

Email

mediana@email.com

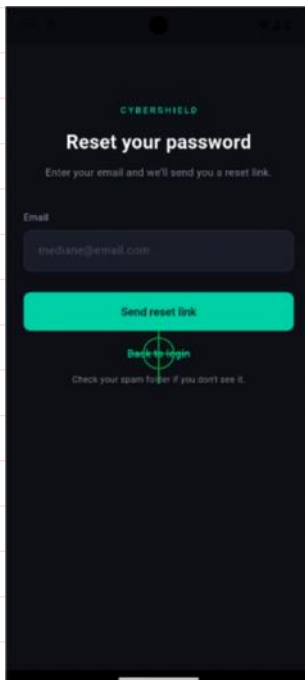
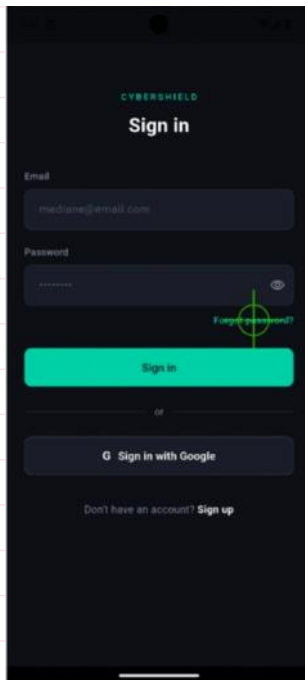
Password

Confirm password

Create account

G Continue with Google

Already have an account? [Sign in](#)



```
Administrator@Mediane MINGW64 ~/Desktop/Internship-2025/Internship-Projects/Mobile-Development/mobile-project/project/cybershield_app (develop)
$ flutter test integration_test/auth_navigation_flow_test.dart -d emulator-5554
00:00 +0: loading C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\Mobile-Development/mobile-project/project/cybershield_app/integration_test/auth_navigation_flow_test.dart
Running Gradle task 'assembleDebug'... 137.8s
✓ Built build\app\outputs\flutter-apk\app-debug.apk
Installing build\app\outputs\flutter-apk\app-debug.apk... 6.8s
00:00 +0: auth screens navigate on an emulator
00:30 +1: (tearDownAll)
00:37 +1: All tests passed!
```